SmartVoice: A Bilingual Cloud-Assisted Windows Desktop Voice Assistant

This project is submitted in partial fulfillment of the
requirements for the degree
of
Bachelor of Science in Computer Science and Engineering

By

Md Fahim Bin Alam
ID: 0812220205101037

Md. Foisal Haque Sohom
ID: 0812220205101057

Nishat Tasnim Chowdhury
ID: 0812220105101022

Under the supervision of
Mohammed Golam Sarwar Bhuyan
Professor & Dean, Faculty of ECE



Department of Computer Science and Engineering
Bangladesh Army University of Engineering & Technology
(BAUET)

July, 2026

KNOWLEDGE & TECHNOLOGY

Bangladesh Army University of Engineering & Technology

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the project entitled "SmartVoice: A Bilingual Cloud-Assisted Windows Desktop Voice Assistant" by "Md Fahim Bin Alam", ID No.: 0812220205101037, Session: 2022-23, "Md. Foisal Haque Sohom", ID No.: 0812220205101057, Session: 2022-23, and "Nishat Tasnim Chowdhury", ID No.: 0812220105101022, Session: 2022-23, has been accepted as satisfactory in partial fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering on July, 2026.

Signature of Supervisor

.....

Mohammed Golam Sarwar Bhuyan

Professor & Dean

Faculty of ECE

Bangladesh Army University of Engineering & Technology

KNOWLEDGE & TECHNOLOGY

Bangladesh Army University of Engineering & Technology

Department of Computer Science and Engineering



DECLARATION

We hereby declare that our project entitled "SmartVoice: A Bilingual Cloud-Assisted Windows Desktop Voice Assistant" is the result of our work. We also ensure that it was not previously submitted or published elsewhere for the award of any degree or diploma. The work has been submitted for the degree of Bachelor of Science in Computer Science and Engineering at Bangladesh Army University of Engineering & Technology (BAUET).

Authors

.....

Md Fahim Bin Alam

.....

Md. Foisal Haque Sohom

.....

Nishat Tasnim Chowdhury

ACKNOWLEDGEMENT

First of all, we express our sincere gratitude to Almighty Allah for giving us the strength, patience, and opportunity to complete this project. We would like to convey our heartfelt appreciation to our respected supervisor, Mohammed Golam Sarwar Bhuyan, Professor & Dean, Faculty of ECE, Bangladesh Army University of Engineering & Technology (BAUET), for valuable guidance, constructive feedback, and continuous encouragement throughout the project. We also acknowledge the support and inspiration of the faculty members and staff of the Department of Computer Science and Engineering, BAUET. Their teaching and encouragement helped us to strengthen the technical and professional foundation required to develop SmartVoice. We are grateful to our friends and well-wishers who provided feedback and support during the development, testing, and documentation stages. Finally, we thank our families for their patience, motivation, and continuous encouragement.

Md Fahim Bin Alam
ID: 0812220205101037
Md. Foisal Haque Sohom
ID: 0812220205101057
Nishat Tasnim Chowdhury
ID: 0812220105101022

ABSTRACT

SmartVoice is a Python-based Windows desktop voice assistant designed to provide bilingual interaction in Bangla and English. The system brings together speech input, language handling, intent routing, task execution, graphical feedback, and text-to-speech output within a single desktop application. Its current implementation uses a PyQt5 graphical user interface, a Chrome and Selenium speech-recognition bridge, Cohere-based intent routing, Groq-assisted chat and content generation, Serper-supported real-time search, Hugging Face image generation, Microsoft Edge text-to-speech, and JSON-based runtime storage. When a user activates the microphone, SmartVoice captures the spoken command, identifies the active language, translates Bangla content into English for routing where necessary, determines the expected task, executes the selected service or automation, presents the result in the interface, and reads an appropriate response aloud. The confirmed system features include general chat, real-time search, opening websites or selected applications, Google and YouTube search, YouTube media control, reminder management, written-content generation, image generation, performance logging, and local chat-log storage. The project is intentionally documented as a cloud-assisted prototype. Although earlier project materials describe a broader privacy-first and offline vision, full offline speech recognition, local intent processing, encrypted database storage, tamper-resistant audit logging, user authentication, and confirmation prompts for sensitive automation are not confirmed in the current codebase. Formal verification reported that the Python environment, safe module imports, local JSON files, and thirteen automated tests completed successfully. Live microphone, browser, API, and external-service workflows remain dependent on network availability, configured API keys, Chrome compatibility, microphone permission, and third-party service quotas. The project demonstrates how modular artificial intelligence services, bilingual voice interaction, desktop automation, and transparent local logging can be combined into a practical assistant prototype while openly documenting its operational limitations and future development priorities.

Keywords: Voice Assistant, Bangla, English, Desktop Automation, Speech Recognition, Intent Routing, PyQt5, Cloud-assisted AI.

TABLE OF CONTENTS

CHAPTER / SECTION	PAGE NO.
CERTIFICATE	II
DECLARATION	III
ACKNOWLEDGEMENT	IV
ABSTRACT	V
TABLE OF CONTENTS	VI
LIST OF FIGURES	VIII
LIST OF TABLES	IX
ABBREVIATIONS AND LIST OF SYMBOLS	XI
CHAPTER 1: INTRODUCTION	12
1.1 INTRODUCTION	12
1.2 MOTIVATION	14
1.3 PROBLEM STATEMENT	14
1.4 OBJECTIVES	15
1.4.1 SPECIFIC OBJECTIVES	15
1.4.2 OVERALL OBJECTIVES	15
1.5 SCOPE AND PROJECT BOUNDARY	16
1.6 ADDRESSING CO, PO, KNOWLEDGE PROFILES, COMPLEX ENGINEERING PROBLEMS AND ACTIVITIES	17
1.7 REPORT LAYOUT	37
1.8 CONCLUSION	37
CHAPTER 2: BACKGROUND STUDY	38
2.1 INTRODUCTION	38
2.2 BACKGROUND OF VOICE ASSISTANT SYSTEMS	38
2.3 RELATED TECHNOLOGIES	39
2.4 COMPARATIVE STUDY	40
2.5 CHALLENGES OF EXISTING SYSTEMS	42
2.6 RESEARCH GAP AND PROJECT POSITIONING	43
2.7 CONCLUSION	43
CHAPTER 3: PROJECT PLANNING AND SCHEDULING	45
3.1 INTRODUCTION	45
3.2 RECOGNITION OF NEED	45
3.3 INFORMATION GATHERING	46
3.4 RELEVANT STAKEHOLDERS	48
3.5 FEASIBILITY STUDY	48
3.6 PROJECT SCHEDULING	51
3.7 RISK MANAGEMENT	52
3.8 CONCLUSION	53
CHAPTER 4: SYSTEM ANALYSIS	54
4.1 INTRODUCTION	54
4.2 REQUIREMENT ANALYSIS	54
4.2.1 FUNCTIONAL REQUIREMENTS	55
4.2.2 NON-FUNCTIONAL REQUIREMENTS	56
4.2.3 DATA AND STORAGE REQUIREMENTS	58
4.2.4 USE-CASE ANALYSIS	59

4.2.5 COST-BENEFIT ANALYSIS	62
4.2.6 SWOT ANALYSIS	65
4.3 CONCLUSION	66
CHAPTER 5: SYSTEM DESIGN	67
5.1 INTRODUCTION	67
5.2 OVERALL ARCHITECTURE	67
5.3 MODULE DESIGN	69
5.4 INTENT ROUTING AND COMMAND FLOW	71
5.5 DATA STORAGE DESIGN	73
5.6 USER INTERFACE DESIGN	75
5.7 SECURITY AND PRIVACY DESIGN	77
5.8 CONCLUSION	78
CHAPTER 6: IMPLEMENTATION AND TESTING	81
6.1 INTRODUCTION	81
6.2 DEVELOPMENT ENVIRONMENT AND EXECUTION	81
6.3 MODULE IMPLEMENTATION	84
6.4 TESTING STRATEGY	92
6.5 VERIFIED TEST RESULTS	94
6.6 END-TO-END DEMONSTRATION TESTS	94
6.7 PERFORMANCE LOGGING	95
6.8 LIMITATIONS DURING TESTING	97
6.9 CONCLUSION	98
CHAPTER 7: CONCLUSION AND FUTURE WORK	99
7.1 INTRODUCTION	99
7.2 PROJECT CONCLUSION	99
7.3 IMPACT ON SOCIETY, ENVIRONMENT AND SUSTAINABILITY	100
7.4 ETHICAL AND SECURITY CONSIDERATIONS	101
7.5 LIMITATIONS	102
7.6 FUTURE WORK	103
7.7 CONCLUSION	105
REFERENCES	106
APPENDIX A: INSTALLATION AND EXECUTION GUIDE	107
APPENDIX B: EVIDENCE SCREENSHOTS AND DEMONSTRATION RECORD	108
APPENDIX C: TEST EVIDENCE AND RESULTS	111
APPENDIX D: CODEBASE EVIDENCE INVENTORY	113

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	Project Work Breakdown Structure	50
Figure 3.2	Proposed Gantt Chart for SmartVoice Development	50
Figure 4.1	SmartVoice System Context Diagram	60
Figure 4.2	Level-0 Data Flow Diagram	61
Figure 4.3	SmartVoice Use Case Diagram	61
Figure 4.4	Scenario-Based Three-Year Cost-Benefit Comparison	64
Figure 5.1	Overall SmartVoice Architecture	68
Figure 5.2	Intent Routing and Response Sequence	70
Figure 5.3	Module Interaction Diagram	71
Figure 5.4	Data Storage and Logging Flow	74
Figure 5.5	Home and Microphone Screen	76
Figure 5.6	Chat Screen	76
Figure 5.7	Security and Privacy Boundary	78
Figure 6.1	SmartVoice Main Window at Launch	85
Figure 6.2	Bangla Voice Command Demonstration	85
Figure 6.3	General Chat Result	86
Figure 6.4	Real-Time Search Result	86
Figure 6.5	Reminder Creation or Alert	87
Figure 6.6	YouTube Play or Control Demonstration	87
Figure 6.7	Image Generation Output	88
Figure 6.8	Code Generation Output	88
Figure 6.9	Content Writing Output	89
Figure 6.10	Application Opening Output	89
Figure 6.11	Multitasking Demonstration	90
Figure 6.12	Setup on First Installation	91
Figure 6.13	Automated Unit-Test Result Showing 18 Tests Passed Successfully	93
Figure 6.14	SmartVoice Runtime Performance Statistics Generated from Logged Interactions	96
Figure 6.15	Comparison of Cohere Intent Routing and Pattern-Based Intent Matching	97
Figure B.1	Environment and Project Root	108
Figure B.2	.env Configuration with Secret Values Hidden	109
Figure B.3	GUI Home/Microphone Screen	109
Figure B.4	English Command Result	109
Figure B.5	Bangla Command Result	110
Figure B.6	Reminder Evidence	110
Figure B.7	Automated Test Output	110

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Project Overview	14
Table 1.2	Scope Boundary: Confirmed Implementation and Future Work	16
Table 1.3	Specific and Overall Objectives	17
Table 1.4	Mapping of COs with BAETE Program Outcomes (POs)	18
Table 1.5	Mapping of COs with Knowledge Profiles (K1-K9)	18
Table 1.6	Mapping of COs with Complex Engineering Problems (P1-P7)	19
Table 1.7	Mapping of COs with Complex Engineering Activities (A1-A5)	19
Table 1.8	PO, CO, Knowledge Profiles, Complex Engineering Problems and Activities Mapping	20
Table 1.9	Mapping Justification with CO1	22
Table 1.10	Mapping Justification with CO2	24
Table 1.11	Mapping Justification with CO3	25
Table 1.12	Mapping Justification with CO4	27
Table 1.13	Mapping Justification with CO5	28
Table 1.14	Mapping Justification with CO6	30
Table 1.15	Mapping Justification with CO7	31
Table 1.16	Mapping Justification with CO8	32
Table 1.17	Mapping Justification with CO9	34
Table 2.1	Related Technical Components	40
Table 2.2	Conceptual Comparison with Common Interaction Approaches	41
Table 2.3	Existing-System Challenges and SmartVoice Response	43
Table 3.1	Stakeholder Analysis	47
Table 3.2	Information-Gathering Sources	47
Table 3.3	Project Work Breakdown and Schedule	49
Table 3.4	Feasibility Summary	51
Table 3.5	Risk Register	52
Table 4.1	Functional Requirements	56
Table 4.2	Non-Functional Requirements	57
Table 4.3	Current Data and Storage Files	58
Table 4.4	Primary Use Cases	59
Table 4.5	Qualitative Cost-Benefit Analysis	63
Table 4.5A	SmartVoice Project Cost Summary and Cost-Benefit Assumptions	63
Table 4.5B	Three-Year Scenario-Based Cost-Benefit Analysis	64
Table 4.6	SWOT Analysis	65
Table 5.1	Architecture Modules and Responsibilities	69
Table 5.2	Frontend Elements	72
Table 5.3	Intent Categories and Actions	72
Table 5.4	Data-Handling Design	74
Table 5.5	Security and Privacy Design	77
Table 6.1	Development Environment and Dependency Status	82
Table 6.2	Implementation Module Summary	83
Table 6.3	Existing Automated Test Coverage	92
Table 6.4	Verified Test-Run Summary	93
Table 6.5	End-to-End Demonstration Test Cases	95

Table 6.6	Performance-Logging Evidence	96
Table 7.1	Social, Environmental and Sustainability Considerations	101
Table 7.2	Ethical and Security Considerations	102
Table 7.3	Current Limitations	103
Table 7.4	Future Work Roadmap	104
Table A.1	Installation and Execution Commands	107
Table C.1	Final Live Test Sign-off Sheet	111
Table D.1	Codebase Evidence Inventory	113

ABBREVIATIONS AND LIST OF SYMBOLS

Abbreviation / Symbol	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
GUI	Graphical User Interface
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
STT	Speech-to-Text
TTS	Text-to-Speech
UI	User Interface
UX	User Experience
IDE	Integrated Development Environment
OS	Operating System
CSV	Comma-Separated Values
QA	Quality Assurance
O(1)	Constant time complexity
PO	Program Outcome
CO	Course Outcome
K	Knowledge Profile
P	Complex Engineering Problem
A	Complex Engineering Activity

Chapter 1
Introduction

1.1 Introduction

SmartVoice is a bilingual Windows desktop voice-assistant prototype developed to simplify common computer-based activities through natural voice interaction. Many routine tasks, such as searching for current information, opening websites or applications, playing media, creating reminders, generating written content, or requesting an image, usually require users to switch repeatedly between multiple applications, browser tabs, search engines, and media platforms. This repeated navigation can be time-consuming and inconvenient, particularly for users who prefer hands-free interaction. SmartVoice aims to provide a unified voice-oriented interaction layer through which users can complete selected desktop tasks using Bangla or English commands.

The system is implemented as a Python-based desktop application with a graphical user interface developed using PyQt5. The interface provides visible microphone control, conversation display, status feedback, and response presentation so that users can understand whether the assistant is listening, processing, responding, or encountering an error. Rather than functioning as only a chatbot, SmartVoice combines several practical capabilities within one environment. These include speech input, bilingual language handling, intent routing, general conversation, real-time web search, browser-based automation, YouTube search and control, reminder management, content writing, image generation, local logging, and text-to-speech feedback.

The workflow of SmartVoice begins when a user activates the microphone and speaks a command. The speech-recognition component captures the spoken input through a Chrome and Selenium-based recognition bridge. The system then identifies the language of the command and supports Bangla-English processing where necessary. After normalisation and translation support, the command is passed to the intent-routing layer. This layer determines whether the user requires a general answer, current information, a reminder, media playback, a search action, automation, written content, image generation, or another supported task. The selected backend module then processes the request and returns a result to the graphical interface, with optional spoken feedback through text-to-speech.

The current implementation is accurately described as a cloud-assisted desktop assistant rather than a fully offline system. Several important features depend on external services, including Cohere for decision and intent routing, Groq for conversational and generated text, Serper for real-time search, Hugging Face for image generation, Chrome Web Speech for speech recognition, translation services for language handling, and Microsoft Edge TTS for spoken responses. This cloud-assisted design enables the prototype to integrate advanced capabilities without requiring the development of separate speech-recognition models, large language models, search engines, or image-generation models from scratch. However, it also means that live system performance can depend on internet connectivity, valid API keys, browser compatibility, microphone permission, service quotas, and third-party provider availability.

SmartVoice is designed as an academic prototype that demonstrates modular software engineering and practical AI-service integration. Its architecture separates the graphical interface, speech processing, language management, intent routing, automation, reminders, media control, image generation, text-to-speech, and local JSON-based logging into distinct modules. This separation improves maintainability and allows future developers to replace individual components without redesigning the entire application. For example, browser-based speech recognition could later be replaced by an offline speech engine, while JSON storage could be upgraded to encrypted database storage.

Although earlier proposal and poster materials described a broader offline and privacy-first vision, the current repository does not confirm the implementation of fully offline speech recognition, local intent processing, encrypted storage, user authentication, tamper-resistant audit logging, or confirmation prompts for sensitive automation. Therefore, SmartVoice should be evaluated as a transparent cloud-assisted Windows desktop voice-assistant prototype. Its main contribution is the practical integration of bilingual voice interaction, graphical feedback, AI-assisted routing, desktop task support, and runtime logging within a single modular application.

.

Table 1.1: Project Overview

Item	Description
Project title	SmartVoice: A Bilingual Cloud-Assisted Windows Desktop Voice Assistant
Project type	Python-based Windows desktop assistant prototype
Primary interaction	Bangla and English voice commands with GUI and text-to-speech feedback
Current architecture	Cloud-assisted, modular desktop application
Core technologies	Python, PyQt5, Selenium/Chrome Web Speech, Cohere, Groq, Serper, Hugging Face, Edge TTS, JSON
Primary users	Students, general desktop users, bilingual Bangla/English users, and users seeking hands-free task support
Evidence base	Inspected source code, README.md, requirements/configuration files, automated tests, runtime JSON files, proposal/poster materials

1.2 Motivation

Desktop users frequently switch among multiple interfaces to search for information, open a website, play a video, create a reminder, obtain a short explanation, or generate content. These repeated interactions can be time-consuming, especially when the user wants a hands-free experience or prefers to speak in Bangla. SmartVoice is motivated by the opportunity to unify common desktop tasks within one assistant interface while supporting bilingual communication.

The project also provides practical engineering learning in areas that are increasingly important for intelligent desktop applications: speech recognition, natural language understanding, language translation, modular application design, browser automation, external API integration, secure configuration handling, testing, and operational logging. The value of the project is not only the visible user interface, but also the coordination of multiple services into a traceable workflow.

1.3 Problem Statement

Routine desktop tasks are often fragmented across different applications and require repeated keyboard and mouse interaction. Users may need to search the web, access media, remember a future task, open a program, or obtain an answer while moving between

unrelated interfaces. General voice tools can provide speech interaction, but a lightweight bilingual desktop assistant that combines conversation, real-time search, browser actions, reminder management, media control, and local activity logging is not always available to the user in a single configurable prototype.

SmartVoice addresses this problem by providing a voice-driven desktop workflow. The system receives user speech, identifies or translates the language where required, routes the command to the appropriate handler, executes the requested action, displays the result, and generates a spoken response. The problem is constrained by the current implementation: several features rely on internet access, service credentials, browser compatibility, microphone permissions, and third-party provider availability.

1.4 Objectives

1.4.1 Specific Objectives

- To develop a desktop graphical interface that supports microphone activation, chat display, system status, and visual feedback.
- To capture user speech through a Chrome and Selenium speech-recognition bridge and process Bangla or English commands.
- To identify language and translate Bangla input into English for intent routing where necessary.
- To route commands through a Cohere-assisted decision module with a local simple-pattern baseline.
- To perform supported tasks including general chat, real-time search, website or application opening, Google and YouTube search, YouTube control, reminders, written-content generation, and image generation.
- To provide audio feedback using text-to-speech and maintain local chat, reminder, performance, and comparison logs in JSON files.
- To verify configuration, reminder logic, imports, and simple decision routing through automated tests and to prepare live end-to-end tests for final demonstration.

1.4.2 Overall Objectives

- To build a practical bilingual voice-assistant prototype for common Windows desktop tasks.
- To demonstrate modular integration of voice input, language handling, AI services, browser automation, GUI output, and local logging.

- To document the difference between the confirmed cloud-assisted implementation and planned privacy-first/offline future enhancements.
- To provide a transparent basis for further development toward stronger privacy, local processing, and more robust desktop automation.

Table 1.2: Scope Boundary: Confirmed Implementation and Future Work

Area	Confirmed Current Scope	Not Confirmed / Future Work
Speech processing	Chrome Web Speech capture via Selenium bridge; microphone workflow	Offline Vosk/CMU Sphinx speech recognition
Language handling	Bangla/English detection and translation for routing	Locally trained bilingual language model
Intent routing	Cohere decision routing with SimplePatternMatcher baseline	Fully local intent engine using Rasa/spaCy
Storage	JSON chat, reminders, performance, and comparison files	SQLite/SQLCipher database and encrypted storage
Security	Environment variables and .gitignore handling for API credentials	Authentication, role-based access control, tamper-resistant audit log
Automation	Selected application/site opening, search, YouTube controls, reminders	Broad file-management automation and confirmation prompts for sensitive actions
Deployment	Windows desktop prototype	Offline packaged product with local-only data processing

1.5 Scope and Project Boundary

The main scope of SmartVoice is voice-based interaction with a Windows desktop environment. The implementation focuses on a single-user prototype rather than a multi-user enterprise platform. It includes cloud-connected modules for language model response, real-time search, and image generation; therefore, its expected operation is bounded by valid credentials, service quotas, and network access. The report does not claim that the application is an offline assistant, a secure personal-data vault, a commercial deployment, or a replacement for accessibility-critical human support.

1.6 Addressing Course Outcomes (CO), Program Outcomes (PO), Knowledge Profiles, Complex Engineering Problems and Activities

The following mappings align the confirmed SmartVoice implementation and documented development activities with the BAETE Program Outcomes (POs), Knowledge Profiles (K1-K9), Complex Engineering Problems (P1-P7), and Complex Engineering Activities (A1-A5). The coloured matrix tables provide a concise visual map, followed by a detailed justification for each course outcome. All claims are based on the inspected codebase, verified environment/test notes, and stated implementation limitations.

Table 1.3: Specific and Overall Objectives

Objective Type	Objective	Evidence in Current Project
Specific	Capture and interpret Bangla/English commands	SpeechToText.py and LanguageManager.py are part of the inspected codebase.
Specific	Route user commands to appropriate services	Model.py and SimpleDecisionMaker.py handle routing logic.
Specific	Execute assistant services and automations	Automation.py, Chatbot.py, RealtimeSearchEngine.py, ReminderManager.py, ImageGeneration.py and YouTubeController.py.
Specific	Provide visual and spoken feedback	Frontend/GUI.py and TextToSpeech.py.
Overall	Offer a cohesive voice-driven desktop interaction layer	main.py orchestrates the modules.
Overall	Maintain transparent evidence and future-work boundaries	Local logs, tests, and documented limitations.

Table 1.4: Mapping of COs with BAETE Program Outcomes (POs)

COs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1												
CO2												
CO3												
CO4												
CO5												
CO6												
CO7												
CO8												
CO9												

Table 1.5: Mapping of COs with Knowledge Profiles (K1-K9)

COs	K1	K2	K3	K4	K5	K6	K7	K8	K9
CO1									
CO2									
CO3									
CO4									
CO5									
CO6									
CO7									
CO8									
CO9									

Table 1.6: Mapping of COs with Complex Engineering Problems (P1-P7)

COs	P1	P2	P3	P4	P5	P6	P7
CO1							
CO2							
CO3							
CO4							
CO5							
CO6							
CO7							
CO8							
CO9							

Table 1.7: Mapping of COs with Complex Engineering Activities (A1-A5)

COs	A1	A2	A3	A4	A5
CO1					
CO2					
CO3					
CO4					
CO5					
CO6					
CO7					
CO8					
CO9					

Table 1.8: PO, CO, Knowledge Profiles (K), Complex Engineering Problems (P) and Complex Engineering Activities (A) Mapping

COs	Mapped Program Outcomes (PO)	Mapped Knowledge Profiles (KP)	Complex Engineering Problems (P) with Evidence	Complex Engineering Activities (A) with Evidence
CO1: Ability to analyse fragmented desktop task workflows, bilingual voice interaction needs, privacy boundaries, and cloud-dependency constraints in a desktop-assistant context.	PO1, PO2, PO4	K1, K2, K3, K8	P1, P2	A1
CO2: Ability to review voice-assistant methods and identify gaps in bilingual interaction, practical desktop automation, privacy, and transparent cloud-service integration.	PO2, PO4	K4, K8	P1, P3	A2
CO3: Ability to design and implement a modular voice-assistant workflow that integrates speech capture, language handling, intent routing, task execution, GUI output, and text-to-speech feedback.	PO3	K5	P2, P4, P5, P7	A1, A3
CO4: Ability to apply modern tools including Python, PyQt5, Selenium/Chrome, service APIs, JSON storage, and automated-test tools to develop and maintain the system.	PO5	K2, K6	P2, P6	A3
CO5: Ability to verify configuration, reminder logic, safe module imports, and simple decision routing, and to prepare repeatable end-to-end test scenarios for the live system.	PO4, PO5	K2, K6, K8	P1, P3, P7	A1, A3
CO6: Ability to plan, coordinate, document, and manage cross-module development activities while considering responsible use of cloud services, APIs, and local runtime data.	PO8, PO10	K9	P5, P6	A1, A2

COs	Mapped Program Outcomes (PO)	Mapped Knowledge Profiles (KP)	Complex Engineering Problems (P) with Evidence	Complex Engineering Activities (A) with Evidence
CO7: Ability to communicate the SmartVoice architecture, workflow, test evidence, limitations, and future roadmap to technical and non-technical stakeholders.	PO9	K7, K9	P6	A4
CO8: Ability to assess privacy, user-data, accessibility, cloud-service, safety, societal, and sustainability implications of the SmartVoice design.	PO6, PO7	K5, K7, K9	P2, P5, P6	A4
CO9: Ability to identify limitations and propose evidence-based enhancements, including offline speech, local intent routing, encrypted storage, stronger logging, and safer automation controls.	PO11, PO12	K8	P1	A3, A5

Justification For Program Outcomes (PO), Complex Engineering Problems (P) and Complex Engineering Activities (A), Knowledge Profiles (K)

CO1: Problem Identification & Analysis – Analyse fragmented desktop task workflows, bilingual voice interaction requirements, cloud-service dependencies, privacy boundaries, and practical limitations in a Windows desktop-assistant context.

Table 1.9: Mapping Justification with CO1

Program Outcomes	Application in this Project
PO1: Engineering Knowledge Application	Applied software-engineering, speech-processing, language-handling, browser-automation, and desktop-UI knowledge to understand how SmartVoice receives a command, selects an action, executes a task, and returns visual and spoken feedback. The analysis was grounded in the confirmed Python modules, the PyQt5 interface, Selenium/Chrome speech bridge, and modular backend services.
PO2: Problem Analysis	Identified the practical problem of fragmented desktop interaction: a user commonly switches between applications, search pages, media controls, reminders, and chat tools. Analysed the further requirements of bilingual Bangla/English interaction, cloud dependency, privacy exposure, API reliability, and limited automation coverage. These constraints informed the bounded scope of the prototype.
PO4: Investigation	Used codebase inspection and documented verification evidence to check environment readiness, valid JSON runtime files, dependency consistency, configuration handling, imports, reminder logic, and simple command-routing behaviour. Live microphone, browser, and network-service tests remain a separate final demonstration requirement.
K1: Natural sciences	Applied human communication, speech, language, and human-computer interaction concepts to understand user voice input, bilingual language switching, and spoken feedback in a desktop environment.
K2: Mathematics	Applied logical decision-flow reasoning, timing concepts for reminders, and basic data validation principles for JSON runtime records and reproducible test observations.
K3: Engineering fundamentals	Applied modular software architecture, file-based state management, error-aware configuration, and integration principles across GUI, speech capture, routing, automation, and output modules.

Program Outcomes	Application in this Project
K8: Engagement of research	Reviewed the practical capabilities and boundaries of modern voice, search, chat, text-to-speech, and image-generation services, while distinguishing confirmed implementation from planned offline/privacy-first concepts.
P1: Depth of Knowledge Required	Required integrated knowledge of desktop software engineering, AI-service use, speech interaction, language handling, browser automation, and local runtime data management to analyse system behaviour and risks.
P2: Range of Conflicting Requirements	Balanced responsiveness and usability against cloud dependency, service cost/quota, data privacy, API reliability, bilingual handling, and safe automation limits.
A1: Range of resources	Coordinated a broad set of resources: Python modules, GUI assets, microphone/browser tools, external APIs, local JSON files, configuration variables, automated tests, and project documentation.

CO2: Literature Review & Research Gap Analysis – Review voice-assistant approaches and identify gaps relating to bilingual interaction, practical desktop automation, cloud-service transparency, privacy, and reliable local control.

Table 1.10: Mapping Justification with CO2

Program Outcomes	Application in this Project
PO2: Problem Analysis	Compared the requirements of a practical desktop assistant with common limitations in fragmented GUI workflows, browser-only tools, cloud voice systems, and proposed offline privacy-first alternatives. The analysis identified a need for a transparent description of what SmartVoice currently implements and what remains future work.
PO4: Investigation	Investigated the repository structure, README guidance, configuration files, service integrations, tests, proposal material, and poster material. This evidence was used to distinguish confirmed modules from features that are described only as planned design goals.
K4: Engineering specialization	Applied specialised knowledge of voice-assistant pipelines, API-based AI services, browser automation, language routing, speech-to-text, text-to-speech, and task-oriented command classification.
K8: Engagement of research	Considered current approaches to speech interfaces, multilingual interaction, desktop automation, privacy-aware design, and cloud/offline trade-offs to identify justified enhancement opportunities.
P1: Depth of Knowledge Required	Required multidisciplinary understanding of intelligent-assistant design, service integration, user interaction, and software limitations to identify meaningful research and implementation gaps.
P3: Depth of analysis required	Required careful analysis of the difference between the implemented cloud-assisted system and the earlier/planned offline privacy-first design so that the report does not overstate current capabilities.
A2: Level of interactions	Managed interactions between speech capture, language translation, Cohere routing, Groq response generation, Serper search, automation modules, GUI updates, and text-to-speech output.

CO3: Solution Design & Implementation – Design and implement a modular bilingual desktop-assistant workflow integrating speech capture, language handling, intent routing, task execution, user-interface feedback, and voice output.

Table 1.11: Mapping Justification with CO3

Program Outcomes	Application in this Project
PO3: Design/ Development of Solutions	Designed a practical modular solution in which main.py orchestrates microphone activation, speech processing, language handling, command routing, automation/search/chat/image/reminder actions, GUI updates, and text-to-speech feedback. The architecture separates responsibilities so individual modules can be improved without rewriting the full application.
K5: Engineering design and support	Designed a lightweight file-based prototype using local JSON storage rather than a database server, while keeping the modules separate for maintainability. The design supports future substitution of components such as offline speech recognition, local intent detection, encrypted storage, and stronger permission controls.
P2: Range of Conflicting Requirements	Balanced feature breadth against reliability, usability against command complexity, local simplicity against cloud-service dependence, and response quality against latency or API quotas.
P4: Familiarity with issues in practice	Addressed practical integration issues including microphone/browser behaviour, Chrome/WebDriver compatibility, asynchronous GUI responsiveness, API configuration, external service availability, and file-based state communication.
P5: Extent of applicable codes	Applied multiple technology conventions and provider requirements across desktop UI programming, browser automation, API-based AI services, JSON data handling, and Windows execution environments.
P7: Interdependence	Managed the interdependent flow from voice capture through translation/routing and action execution to GUI/TTS output and logging, where a failure in one external module can affect the complete user experience.
A1: Range of resources	Integrated frontend assets, Python modules, Chrome/Selenium tooling, cloud-service APIs, environment variables, local JSON files, and test materials.

Program Outcomes	Application in this Project
A3: Innovation	Combined bilingual voice interaction, AI-assisted command routing, browser automation, real-time search, reminders, media control, image generation, and text-to-speech within one Windows desktop prototype.

CO4: Modern Engineering Tools Application – Apply modern software-development, desktop-UI, browser-automation, AI-service, configuration, and automated-testing tools to build and maintain SmartVoice.

Table 1.12: Mapping Justification with CO4

Program Outcomes	Application in this Project
PO5: Modern Tool Usage	Used Python as the core language; PyQt5 for the desktop interface; Selenium/Chrome for speech capture and selected browser interaction; Cohere, Groq, Serper, Hugging Face, and Edge TTS for cloud-assisted capabilities; JSON files for local persistence; and unittest for automated checks.
K2: Mathematics	Applied logical categorisation, timing and delay handling for reminders, simple performance-record interpretation, and data-validation reasoning during runtime-file and test inspection.
K6: Engineering practice (technology)	Applied environment configuration through .env, dependency management through Requirements.txt, .gitignore protection for secret files, modular source organisation, safe import checks, and unit-test execution practices.
P2: Range of Conflicting Requirements	Balanced tool capability with practical limitations such as Windows/Chrome requirements, external API keys, network access, provider quotas, local dependency consistency, and need for a simple GUI workflow.
P6: Extent of stakeholder involvement and conflicting requirements	Considered the needs of end users, project supervisors, developers, cloud-service providers, and privacy-conscious stakeholders while keeping the prototype feasible for a final-year project.
A3: Innovation	Applied a contemporary toolchain to bridge desktop interaction with cloud AI capabilities, enabling multiple supported command categories through a unified interface.

CO5: Experimental Analysis & Data Evaluation – Verify deterministic modules, inspect runtime data, document test evidence, and prepare repeatable end-to-end demonstration scenarios for SmartVoice.

Table 1.13: Mapping Justification with CO5

Program Outcomes	Application in this Project
PO4: Investigation	Conducted documented verification through Python and virtual-environment checks, pip consistency checking, JSON validation, safe import tests, and the formal unittest suite. The reported automated suite completed thirteen tests successfully; live API, microphone, browser, and GUI behaviour must be demonstrated separately.
PO5: Modern Tool Usage	Used Python unittest discovery and controlled command-line checks to validate configuration, reminder parsing, import side effects, and local simple-pattern decision routing without launching potentially costly or interactive services.
K2: Mathematics	Used structured test-case design, expected-versus-observed outcome logic, timing interpretation for reminders, and basic record analysis for JSON logs.
K6: Engineering practice (technology)	Applied repeatable test commands, isolated safe imports, requirements checking, and structured logging so evidence can be reproduced before final demonstration.
K8: Engagement of research	Used evidence-based performance and reliability reasoning, recognising that no live accuracy, latency, user-study, or external-service success-rate claim should be made without measured results.
P1: Depth of Knowledge Required	Required knowledge of software testing, module dependencies, runtime data, external-service boundaries, and error isolation to interpret what the automated tests do and do not prove.
P3: Depth of analysis required	Required analysis of test coverage gaps between local deterministic checks and full end-to-end voice/browser/API behaviour, leading to a separate live demonstration plan.
P7: Interdependence	Recognised that external APIs, Chrome/WebDriver, microphone permissions, GUI event flow, and TTS playback must all function together for a successful user interaction.
A1: Range of resources	Used virtual-environment tooling, Python tests, local JSON data, configuration files, safe import checks, and a structured demonstration checklist.

Program Outcomes	Application in this Project
A3: Innovation	Created a transparent evidence model that separates verified automated behaviour from tasks that must still be validated live, reducing the risk of unsupported performance claims.

CO6: Project Management & Leadership: Plan, coordinate, document, and manage cross-module development activities while maintaining responsible and transparent use of cloud services and local runtime data.

Table 1.14: Mapping Justification with CO6

Program Outcomes	Application in this Project
PO8: Ethics	Applied responsible project-management principles by documenting cloud dependency, data-handling limits, missing controls, provider/service risks, and the boundary between current implementation and future features. The project avoids presenting planned offline or encrypted features as implemented.
PO10: Communication	Organised technical content into implementation modules, report chapters, test evidence, diagrams, tables, demonstration steps, and limitations so that project work can be reviewed by both technical and non-technical stakeholders.
K9: Ethics, inclusive behavior and conduct	Promoted transparent claims, privacy awareness, bilingual access, responsible handling of secret values, and clear disclosure of system limitations and external-service data flows.
P5: Extent of applicable codes	Managed multiple platform and provider constraints, including Windows execution, Chrome/Selenium behaviour, API configuration, service-specific requests, environment variables, and local file formats.
P6: Extent of stakeholder involvement and conflicting requirements	Balanced developer constraints, user expectations, supervisor assessment requirements, privacy concerns, and third-party service limitations during planning and documentation.
A1: Range of resources	Coordinated source code, APIs, GUI assets, configuration, tests, logs, documentation, and demonstration evidence across the project lifecycle.
A2: Level of interactions	Managed interactions among modules and participants by defining responsibilities, evidence boundaries, validation steps, and clear handoffs between implementation, testing, and report preparation.

CO7: Effective Communication & Presentation: Communicate SmartVoice architecture, workflow, source-code evidence, test results, limitations, and future plans clearly to diverse audiences.

Table 1.15: Mapping Justification with CO7

Program Outcomes	Application in this Project
PO9: Individual & Team Work	Prepared structured technical content that supports collaboration and review: module summaries, system-flow diagrams, requirement tables, test cases, evidence placeholders, and a concise supervisor demonstration sequence. The documentation helps team members present a coherent account of a multi-module system.
K7: Role of engineering	Explained how engineering design choices affect user productivity, bilingual accessibility, privacy, reliability, cloud dependence, and responsible automation in a desktop-assistant setting.
K9: Ethics, inclusive behavior and conduct	Used transparent and inclusive communication by avoiding unsupported claims, identifying privacy and data-sharing implications, and recognising that Bangla/English usability should be validated with real users.
P6: Extent of stakeholder involvement and conflicting requirements	Communicated technical content for users, supervisors, developers, and privacy-conscious stakeholders whose expectations can differ regarding speed, simplicity, features, and data protection.
A4: Consequences to society and the environment	Presented the potential social benefits and practical risks of voice-driven desktop interaction, including accessibility, data sharing, AI answer reliability, external-service dependence, and future sustainability considerations.

CO8: Ethical & Social Impact Recognition – Assess the societal, environmental, safety, accessibility, privacy, and data-governance implications of a cloud-assisted bilingual desktop voice assistant.

Table 1.16: Mapping Justification with CO8

Program Outcomes	Application in this Project
PO6: Engineer & Society	Considered how SmartVoice may reduce repetitive desktop interaction and support Bangla/English access while recognising that voice data and user queries may pass through external services. The project therefore requires clear user communication and should not be relied on for high-stakes decisions without verification.
PO7: Environment & Sustainability	Recognised that external cloud services, repeated browser automation, and network/API usage have resource and continuity implications. The current report does not claim a measured environmental impact; future local/offline components may reduce recurring network dependence.
K5: Engineering design and support	Evaluated design trade-offs including lightweight local JSON storage, modular replacement of components, future offline alternatives, maintenance requirements, and the need for safer command confirmation controls.
K7: Role of engineering	Addressed the role of engineering in creating convenient but responsible human-computer interaction, considering privacy, accessibility, correct disclosure, and limitations of generative AI responses.
K9: Ethics, inclusive behavior and conduct	Promoted data-minimisation awareness, secret protection, privacy disclosure, bilingual usability, fair user treatment, and avoidance of overstated security or privacy claims.
P2: Range of Conflicting Requirements	Balanced convenience and automation against data privacy, user consent, external AI-service use, cost, reliability, error risk, and the need for understandable user control.
P5: Extent of applicable codes	Applied provider and platform rules, local configuration controls, .gitignore practices, and Windows/browser execution constraints while recognising that security controls are currently limited.

Program Outcomes	Application in this Project
P6: Extent of stakeholder involvement and conflicting requirements	Considered users, supervisors, service providers, developers, and people concerned with privacy or accessibility, whose priorities may conflict.
A4: Consequences to society and the environment	Assessed potential benefit in desktop productivity and bilingual access alongside risks of cloud data handling, misinformation, unsafe automation, service outage, and resource dependence.

CO9: Lifelong Learning & Technology Adaptation – Identify current SmartVoice limitations and propose responsible, evidence-based adaptations to evolving voice, AI, privacy, testing, and desktop-automation technologies.

Table 1.17: Mapping Justification with CO9

Program Outcomes	Application in this Project
PO11: Project Management & Finance	Recognised the continuing management needs of a cloud-assisted system, including API quotas and potential costs, dependency updates, Chrome/WebDriver compatibility, test maintenance, secure configuration, and prioritisation of future development work.
PO12: Lifelong Learning	Demonstrated continued learning by analysing the current architecture, recognising service and security limitations, and identifying feasible future enhancements: offline speech recognition, local intent handling, encrypted storage, stronger audit logging, confirmation prompts, and more complete live testing.
K8: Engagement of research	Engaged with evolving knowledge in voice interfaces, multilingual interaction, cloud/offline AI trade-offs, privacy-aware system design, testing practice, and safe desktop automation to guide future decisions.
P1: Depth of Knowledge Required	Required continued learning across changing APIs, browser tools, speech technologies, language models, security practices, and practical deployment constraints.
A3: Innovation	Proposes a modular improvement path that can replace or extend individual components without discarding the current architecture, allowing the assistant to move toward stronger privacy and reliability.
A5: Familiarity	Extends conventional desktop programming by integrating voice interaction, cloud AI services, browser automation, bilingual language handling, local runtime data, and responsible future-security planning.

List of Activities for SmartVoice

The following list presents the team responsibility allocation for the SmartVoice project. The intended contribution balance is based on the technical complexity and scope of the assigned activities.

Create a Team – Teamwork (FA, FS, NC)

1. Idea sharing – Teamwork
2. Project idea selection – Teamwork
3. Project scope definition – Teamwork
4. Project proposal documentation
    i. Problem statement, objectives and expected outcome – NC
    ii. System architecture and core methodology – FA
    iii. Technology stack and feasibility analysis – FS
5. Literature and background study – Teamwork
6. Requirement gathering – Teamwork
7. Existing system and feature analysis – FS
8. Proposed system architecture – FA
9. Functional and non-functional requirement specification – FS
10. Risk, ethics and privacy assessment – NC
11. Cost-benefit analysis – NC
12. Project scheduling and milestone planning – Teamwork
13. Backend module decomposition – FA
14. Main assistant workflow design – FA
15. Threading and service orchestration – FA
16. Speech recognition workflow design – FA
17. Selenium and Chrome Web Speech integration – FA
18. Microphone state and profile handling – FA
19. Bangla-English language detection – FA
20. Query translation and normalization – FA
21. Cohere intent-routing model integration – FA
22. Intent classification prompt and decision logic – FA
23. Simple pattern-matcher baseline – FS
24. Groq chatbot response pipeline – FA

25. Serper real-time search integration – FA
26. Search response synthesis – FA
27. Automation executor development – FS
28. Application and website launch automation – FS
29. Google and YouTube search automation – FS
30. YouTube playback controller – FA
31. YouTube pause, resume, volume and timer controls – FS
32. Reminder parser and JSON storage – FS
33. Due reminder checking and alert loop – FA
34. Hugging Face image-generation worker – FS
35. Image-generation parameters and output handling – FS
36. Edge TTS and voice playback – FS
37. PyQt home screen implementation – FS
38. PyQt chat screen and status-view implementation – FS
39. GUI and backend file-state integration – FS
40. GUI graphics, icons and GIF asset arrangement – NC
41. JSON chat, reminder and performance-storage structure – FA
42. Performance and comparison logging design – FA
43. Statistics and comparison export utilities – FS
44. Environment template and dependency setup – FS
45. Runtime data and secret ignore strategy – FA
46. Unit-test strategy and test planning – FA
47. Configuration and reminder test cases – FS
48. Import-side-effect and pattern-routing tests – FA
49. Unit-test documentation and result tables – NC
50. UI and usability-test documentation – NC
51. Architecture, data-flow, use-case and workflow diagrams – NC
52. Screenshot and evidence collection – NC
53. User manual and installation guide – NC
54. Report formatting, references and appendix preparation – NC
55. Poster and presentation layout – NC
56. Final system integration and bug fixing – Teamwork
57. Final demonstration and presentation rehearsal – Teamwork

58. Final documentation review and submission preparation – Teamwork

Legend:

- Md Fahim Bin Alam (FA)
- Md. Foisal Haque Sohom (FS)
- Nishat Tasnim Chowdhury (NC)
- Teamwork (TW)

1.7 Report Layout

The report is organized into seven chapters. Chapter 1 introduces the project, objectives, scope, and educational mappings. Chapter 2 presents the background of voice assistants, related technologies, existing-system challenges, and the project position. Chapter 3 describes planning, stakeholders, feasibility, scheduling, and risk management. Chapter 4 analyses functional requirements, non-functional requirements, storage, use cases, cost-benefit considerations, and SWOT factors. Chapter 5 presents the architecture, modules, routing flow, data handling, interface design, and privacy boundary. Chapter 6 documents implementation, setup, testing, verified results, demonstration tests, and performance logging. Chapter 7 concludes the report and presents social impact, ethical considerations, limitations, and future work.

1.8 Conclusion

SmartVoice is a modular and transparent desktop assistant prototype. It brings together a number of contemporary tools, but its operational boundaries must be stated clearly. The current product is a cloud-assisted system with verified unit tests and safe imports, while live API and hardware-dependent workflows should be demonstrated and documented before final submission.

Chapter 2
Background Study

2.1 Introduction

The background study establishes the technical and practical context of SmartVoice as a bilingual Windows desktop assistant. A voice assistant is not a single algorithm; it is a coordinated interaction system that must acquire speech, transform it into usable text, identify the user's intended action, invoke the appropriate capability, and return understandable feedback. In SmartVoice, these responsibilities are distributed across a PyQt5 desktop interface, a Chrome/Selenium speech-recognition bridge, language handling, an intent-routing layer, specialised task modules, and text-to-speech output. This modular construction is important because the requirements of a desktop assistant are wider than those of a normal question-answering chatbot. The system must also communicate state clearly, respond safely when a browser or API service fails, and avoid treating every spoken sentence as a command that should be executed automatically. The current source-code evidence therefore provides the central basis for the discussion in this chapter [12].

The study also distinguishes the implemented prototype from the earlier offline-first design described in the proposal and poster. The present codebase uses cloud-assisted services for major capabilities, including Cohere for decision routing, Groq for content generation, Serper for current web-search results, Hugging Face for image generation, Chrome Web Speech for speech capture, and Edge TTS for spoken feedback. This does not reduce the value of the prototype; rather, it defines the real engineering boundary that must be stated accurately in the report. SmartVoice should be evaluated as an integration-oriented desktop assistant that combines local orchestration with selected online intelligence services. The background discussion therefore examines both its strengths - rapid functionality, bilingual routing, modular design, and service integration - and the limitations that follow from external network, browser, API, privacy, and quota dependencies.

2.2 Background of Voice Assistant Systems

Voice-assistant systems normally follow a pipeline in which an acoustic input is captured, converted into text, interpreted, mapped to an answer or action, and returned to the user through visual or spoken feedback. Each stage introduces a distinct engineering problem. Speech capture must manage microphone state, browser support, ambient noise, accent

variation, and user hesitation. Natural-language interpretation must identify whether a sentence is a request for information, a command, a reminder, a search, a media-control instruction, or a casual conversation. The action layer must then select a suitable tool rather than attempting to solve every request through a single language model. Finally, the interface must explain what the assistant is doing so that users are not confused when a request is being processed, sent to a web service, or unable to be completed. The Web Speech API separates recognition from application-level processing, which supports this layered approach [4].

SmartVoice follows this general model but adapts it for a Windows desktop use case. The project starts from a microphone toggle inside the PyQt5 interface, receives recognised text through the browser-based speech bridge, and uses language management to support Bangla and English interaction. After normalisation and possible translation for routing, the decision model determines the required path. General questions are directed toward a chatbot response, current-information requests toward real-time search, command-like requests toward automation or YouTube control, and time-based requests toward the reminder manager. The response is displayed through the user interface and can also be synthesised through Edge TTS. This workflow allows one assistant to coordinate several task types while retaining a visible separation between voice recognition, decision logic, automation, and feedback.

2.3 Related Technologies

The SmartVoice implementation is built on Python because it provides a practical environment for desktop development, file processing, web automation, and AI-service integration [1]. PyQt5 supplies the graphical desktop interface, including the home screen, chat screen, microphone control, custom top bar, and visual assets [2]. Selenium is used as a browser-automation bridge for Chrome-based speech recognition and for parts of the YouTube-control workflow [3]. This approach enabled the project to demonstrate live browser-assisted interaction without building a custom speech-recognition engine from scratch. However, it also means that compatibility with Chrome, the microphone, a suitable WebDriver, and network access must be considered during installation and live testing.

The decision and response capabilities are also modular. Cohere is used as a first-layer intent-routing service, while a local SimplePatternMatcher provides a basic comparison baseline. Groq supports conversational and generated-content responses, and Serper supplies web-search results that can be summarised for time-sensitive queries. Hugging Face inference

is used for image-generation requests, while Edge TTS and audio playback provide spoken output. LanguageManager coordinates Bangla/English detection and routing support, and ReminderManager maintains time-based notifications through local JSON data. The system does not currently use an SQL database, REST server, Docker deployment, or offline ASR/NLU stack. Instead, it uses file-based storage and direct Python-module coordination. The chosen technologies demonstrate a realistic prototype architecture in which independent services can be replaced later without redesigning the entire user interface.

Table 2.1: Related Technical Components

Component	Role in SmartVoice	Key Consideration
Python	Core application language and module orchestration	Environment compatibility and dependency management
PyQt5	Desktop interface with home/microphone and chat-oriented screens	UI responsiveness and thread-safe updates
Selenium and Chrome	Browser-assisted speech recognition and web/media control	Driver compatibility, browser availability, dynamic page behavior
Web Speech recognition	Speech input bridge	Network/browser support and recognition variability
Cohere	Intent/decision routing	API key, quota, service response, data sharing
Groq	General chat and generated text	API availability, rate limits, answer quality
Serper	Real-time search retrieval	Freshness, quota, provider terms
Hugging Face	Image generation service	Model availability, request time, quota
Edge TTS	Spoken output	Voice availability, network/service dependency
JSON files	Runtime chat, reminders, logs, and comparison data	No encryption, concurrent-write and backup limitations

2.4 Comparative Study

The comparison in Table 2.2 is intentionally conceptual rather than a performance benchmark. SmartVoice has not been tested against commercial assistants under identical hardware, network, microphone, language, or task conditions. The purpose of the comparison

is to show how the prototype combines interaction modes that users otherwise access separately. A traditional graphical desktop workflow normally requires keyboard and mouse navigation across several windows. A basic web search interface can provide information but does not directly coordinate reminders, media control, or desktop actions. A generic cloud voice tool may recognise spoken language but may not expose a transparent modular route for custom Windows automation. SmartVoice positions itself between these approaches by using a desktop GUI as the visible control surface while accepting Bangla or English voice input and selecting a task-specific module based on detected intent.

The most meaningful difference is therefore architectural integration rather than a claim of superior intelligence. SmartVoice brings together chat, search, reminders, media control, generated content, image generation, and limited automation in a single Python desktop prototype. Its strengths are modularity, direct project-level access to routing logic, local JSON logging, and the ability to tailor the user experience for the target academic demonstration. Its limitations are equally important: external APIs are required for several features, results may vary with network availability and quota, browser automation can be fragile, and security controls remain limited compared with a production-grade assistant. A future comparative study should define identical task scripts, establish measurable indicators such as completion time and success rate, and collect user feedback under controlled conditions.

Table 2.2: Conceptual Comparison with Common Interaction Approaches

Criteria	Traditional GUI Interaction	Web Search / Chat Interface	Typical Cloud Voice Tool	SmartVoice Prototype
Primary input	Keyboard and mouse	Typed text	Voice	Bangla/English voice plus GUI
Desktop task integration	Manual switching between applications	Usually browser-centred	Varies by platform	Selected browser, search, media, reminder, and app/site actions
Real-time search	User performs search manually	Available through search service	Varies	Supported through Serper-assisted module
Local logging	Application-specific	Service-specific	Service-specific	Local JSON logs are used
Offline privacy	Can be local depending on app	Usually, cloud connected	Usually, cloud connected	Not confirmed; current system is

Criteria	Traditional GUI Interaction	Web Search / Chat Interface	Typical Cloud Voice Tool	SmartVoice Prototype
				cloud-assisted
Customization	Manual workflows	Prompt based	Platform dependent	Modular Python codebase and environment configuration

2.5 Challenges of Existing Systems

Voice-driven desktop systems face both input and execution challenges. Speech recognition can be affected by microphone sensitivity, environmental noise, accent, speaking speed, language switching, browser configuration, and the availability of a compatible speech-recognition service. An utterance may also be ambiguous: the sentence 'play relaxing music' requires a media-search route, whereas 'what is relaxing music' could be interpreted as a general-information question. The assistant must therefore make an intent decision before launching a browser, calling an API, or producing an answer. Automation creates an additional reliability issue because actions may depend on application names, installed software, browser state, focused windows, or web-page layout. Selenium-based operations can fail when a page changes or when a matching browser driver is not available. These are normal prototype risks and should be tested through controlled end-to-end demonstrations.

Privacy and security are also central challenges. In the current implementation, several user requests can be transmitted to third-party cloud services. Local runtime files are JSON-based and are not confirmed to be encrypted, while the codebase does not contain a full user-authentication or role-management mechanism. For these reasons, the assistant should not be positioned as suitable for sensitive personal, financial, medical, or institutional data. A responsible deployment message should tell users when a request may use an external provider and should avoid unnecessary retention of chat or performance records. The remaining engineering challenge is to preserve usability while adding confirmation prompts, stronger error messages, safe allow-lists for automation, encrypted storage, and offline alternatives. Table 2.3 records the current response and the unresolved boundary for each challenge.

Table 2.3: Existing-System Challenges and SmartVoice Response

Challenge	Current SmartVoice Response	Remaining Limitation
Fragmented desktop tasks	Routes multiple task types from one voice command	Coverage is limited to implemented actions
Bangla/English input	Language detection and translation are included	Accuracy under varied accents/noise requires live testing
Need for fresh information	Uses Serper real-time search module	Search quality and quota depend on provider
Natural-language ambiguity	Cohere routing plus simple pattern baseline	No formal intent-accuracy study confirmed
Repeated reminders/media actions	Includes reminder and YouTube-control modules	Live browser workflow was not verified in the safe inspection
Data traceability	Maintains local JSON logs	Files are not encrypted or tamper-resistant
Offline privacy expectation	Proposal/poster describe privacy-first direction	Current code is not fully offline

2.6 Research Gap and Project Positioning

Many tools already provide individual capabilities such as speech recognition, web search, chatbot interaction, reminder scheduling, text-to-speech, browser control, or image generation. The gap addressed by SmartVoice is not the invention of any single underlying service. Instead, it is the practical integration of these capabilities into a coherent bilingual Windows desktop workflow that a user can operate from one graphical interface. The project demonstrates how a voice request can move through several decision points: recognition, language handling, intent classification, selection of a specialised module, task execution, GUI feedback, spoken feedback, and optional logging. This integration problem is relevant because a fragmented collection of tools requires the user to repeatedly decide which application to open, while an assistant must make that selection transparently and safely.

2.7 Conclusion

This chapter has shown that SmartVoice is located at the intersection of voice-interface engineering, desktop GUI design, browser automation, cloud-service integration, bilingual language handling, and local runtime logging. The technologies selected for the prototype make it possible to demonstrate a broad range of functions quickly: voice capture, intent

routing, chat, current-information search, reminders, media control, image generation, and spoken feedback. The key technical contribution is the coordination of these functions through a single Windows-oriented application rather than treating them as unrelated scripts. The module separation also creates a maintainable foundation for future replacement of individual services.

At the same time, the review establishes boundaries that must remain visible throughout the report. SmartVoice currently depends on network services, API credentials, Chrome, Selenium-compatible browser automation, and a microphone-enabled environment. It does not yet provide a verified offline processing pipeline, encrypted data layer, multi-user access control, or completed production-grade safety policy for sensitive actions. These limitations do not invalidate the project; they define the next research and development steps. The following chapter converts the background findings into a planning, feasibility, schedule, and risk-management framework that matches the system actually represented by the current codebase.

Chapter 3
Project Planning and Scheduling

3.1 Introduction

Planning SmartVoice requires more than arranging source files because the prototype combines multiple technologies with different reliability conditions. The PyQt5 interface, microphone state, Chrome speech-recognition bridge, language manager, intent model, cloud APIs, automation modules, YouTube controller, text-to-speech service, local JSON files, and test suite must operate as a coordinated system. A failure in one component can affect the user experience even if the remaining code is correct. For example, a valid general-chat request may still fail if the API quota is exhausted; a media-control request may fail if Chrome or the target page state changes; and a voice request may never reach the routing logic if microphone capture is not available. Planning therefore includes dependency identification, task breakdown, test preparation, risk handling, evidence collection, and documentation alongside coding work.

The project plan uses an iterative structure. Early activities define the functional boundary and the user scenarios. Middle activities build and connect the GUI, speech path, language handling, intent routing, task modules, feedback mechanisms, and file-based storage. Later activities focus on formal tests, live demonstration tests, risk reduction, performance logging, screenshots, technical documentation, and report preparation. The 28-week Gantt chart has been adapted from the project proposal so that its phase names better reflect the cloud-assisted implementation currently present in the repository. It should be read as a disciplined development and reporting schedule rather than proof that every planned feature is already complete.

3.2 Recognition of Need

The need addressed by SmartVoice arises from the repetitive nature of ordinary desktop interaction. A user may need to open a browser, look up a current topic, play or pause a video, create a short reminder, ask for a general explanation, generate draft content, or navigate to a web site. Individually, these tasks are simple; however, they require context switching among applications, search pages, media platforms, reminder tools, and keyboard or mouse operations. For users who prefer hands-free interaction, have temporary difficulty using conventional input devices, or want to reduce repetitive navigation, a voice-driven

desktop interface can make these actions more direct. Bangla/English handling is particularly relevant because a bilingual user may naturally switch language while describing a task.

The need should not be overstated as a claim that SmartVoice replaces all desktop controls or existing commercial assistants. The prototype focuses on a selected set of supported actions and is designed for an academic project demonstration. Its value lies in demonstrating that one modular interface can identify several classes of user intent and coordinate different services from a common workflow. The recognised need also includes transparency: the user should understand when the assistant is listening, when it is processing, whether a request is being treated as chat or automation, and whether an external service is required. In a final demonstration, the team should collect a small amount of user feedback on command clarity, response visibility, perceived usefulness, and areas where users expected an unsupported action.

3.3 Information Gathering

Information gathering for SmartVoice was conducted primarily through structured inspection of the project repository and its supporting documentation. The team reviewed the README file, dependency list, environment template, main orchestration file, frontend source, backend modules, test files, runtime JSON files, proposal material, poster material, and Git history. This evidence provides a factual description of what the system currently contains: a Python desktop GUI, browser-assisted speech recognition, bilingual routing support, Cohere and Groq integration, Serper search, Hugging Face image generation, Edge TTS, reminders, YouTube control, JSON storage, logging helpers, and a formal unit-test suite. It also identifies what is not confirmed, including offline ASR/NLU, encrypted SQL storage, REST services, Docker deployment, and authenticated multi-user access.

Repository evidence should be complemented by user-centred information before final submission. A short questionnaire or guided demonstration can ask participants whether the microphone state is understandable, whether voice commands are natural, how often they would use reminders or media control, whether Bangla/English switching feels reliable, and which actions they believe should require confirmation. The resulting feedback should be recorded as actual observations rather than assumed satisfaction scores. The project can also use direct test logs, screenshot evidence, error messages, and performance-log entries as information sources. Tables 3.1 and 3.2 summarise relevant stakeholders and sources so that future decisions remain traceable to either code evidence, test evidence, or user evidence.

Table 3.1: Stakeholder Analysis

Stakeholder	Interest / Need	Interaction with SmartVoice
Primary desktop user	Quick hands-free access to common tasks	Provides voice command and receives visual/spoken result
Bangla/English user	Bilingual command support	Uses language detection and translation route
Project team	Maintainable modular code and demonstrable results	Implements, configures, tests, and documents modules
Academic supervisor	Evidence-based final-year project quality	Reviews report, design, test evidence, and demonstration
Third-party service providers	Authenticated API use and terms compliance	Provide language, search, generation, speech, and hosted model services
Browser and operating-system environment	Compatibility and permissions	Provides Chrome, microphone access, desktop and media environment

Table 3.2: Information-Gathering Sources

Source	What It Provides	Status
README.md	Installation, run, and demo guidance	Confirmed from codebase review
Requirements.txt and .env.example	Dependencies and configuration variables	Confirmed from codebase review
Backend and frontend modules	Actual features and architecture	Confirmed from codebase review
tests/*.py	Unit-test scope and verification evidence	Confirmed: 13 tests reported passed
Data/*.json	Runtime storage presence and structure type	Confirmed: local JSON lists/files
Project proposal and poster	Planned offline/privacy-first direction	Referenced only as planned/not fully implemented
User interviews/surveys	User needs and usability feedback	Missing or unclear; add only if actually conducted

3.4 Relevant Stakeholders

SmartVoice has a compact but diverse stakeholder environment. The primary stakeholder is the desktop user who expects simple voice interaction, clear feedback, and reliable completion of supported tasks. The user benefits when the system distinguishes information requests from automation commands and explains errors without exposing technical details. The development team is another stakeholder because each module must remain understandable enough to debug, test, and extend. The supervisor and department require a report that accurately maps the implemented project to academic outcomes, avoids unsupported performance claims, and provides reproducible evidence. External service providers also indirectly influence the system because their API availability, quotas, policy changes, and response formats can affect live functionality.

Stakeholder expectations can conflict. Users may desire more automation, while responsible design requires caution before opening applications, controlling media, or executing actions that could have side effects. The team may prefer rapid integration through cloud services, while privacy-conscious users may prefer local processing. The supervisor may expect a complete final demonstration, while the project must acknowledge when a live test depends on network access, a browser driver, or an API key. These competing needs justify the use of clear scope statements, test records, consent-oriented notices, and future-work planning. In a later version, stakeholders could be engaged through pilot testing with students, accessibility users, and technical reviewers, but the present report treats these as recommended activities unless direct feedback evidence is inserted.

3.5 Feasibility Study

The feasibility of SmartVoice is assessed across technical, operational, economic, legal, and schedule dimensions. Technically, the repository has a usable Python 3.10.10 environment, an operational virtual environment, valid local JSON files, a detected Chrome installation, safe module imports, and a formal test suite that completed thirteen tests successfully during the documented verification run. These results support the feasibility of the modular prototype. The application is not a separate web server and does not require a database server, which reduces installation complexity. However, live features remain dependent on a microphone, Chrome, Selenium/WebDriver compatibility, network access, API keys, external-service quotas, and user-facing GUI/browser windows. Technical

feasibility is therefore high for a controlled demonstration but conditional for unrestricted deployment.

Operational feasibility is supported by the graphical interface, voice-trigger workflow, status feedback, reminder management, and task-specific module routing. Users can interact through a familiar desktop window rather than a command-line interface. Economic feasibility is analysed in more detail in Section 4.2.5 using the project budget of BDT 15,025 and scenario-based benefit assumptions. Legal and ethical feasibility requires careful disclosure because user queries may be transmitted to third-party services and local JSON files are not verified as encrypted. The project should therefore avoid sensitive data, protect `.env` credentials, and obtain permission before collecting user feedback. Schedule feasibility is supported by the 28-week plan and the division of work among team members. Tables 3.3 and 3.4 provide the detailed work breakdown and feasibility summary.

Table 3.3: Project Work Breakdown and Schedule

Phase	Major Activities	Expected Evidence
Planning	Define scope; identify services; collect project requirements	Proposal, scope table, task list
Environment setup	Create virtual environment; configure requirements and environment variables	README-aligned setup evidence
User interface	Build PyQt5 home/microphone and chat screens	GUI screenshots
Speech and language	Implement browser speech bridge, language detection and translation	Voice demonstration screenshots
Intent and task modules	Implement decision routing, chat, search, automation, reminders, image and media functions	Module inventory and demo results
Storage and logging	Create JSON chat, reminder, performance and comparison storage	Data file evidence
Testing	Run unit tests, safe imports, JSON validation, and live demonstration tests	Test output and test-case table
Documentation	Prepare diagrams, report, references, appendices, and presentation	Final report and slide evidence

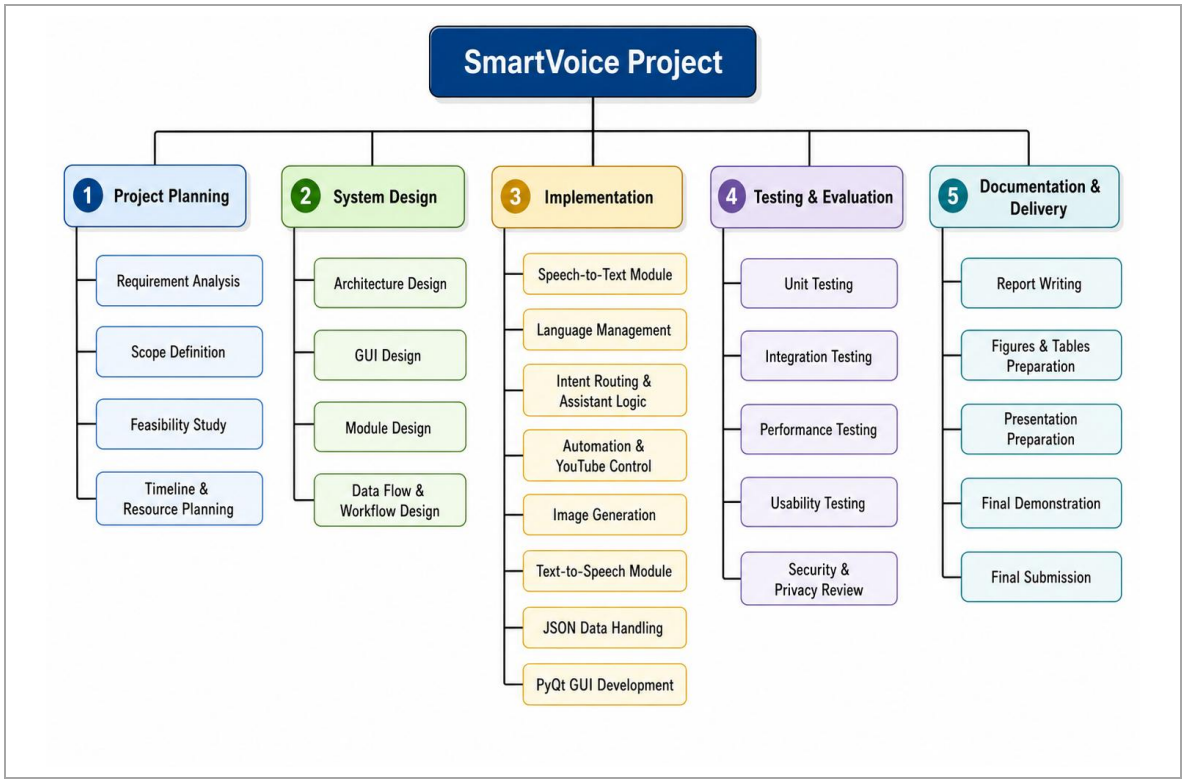


Figure 3.1: Project Work Breakdon Structure

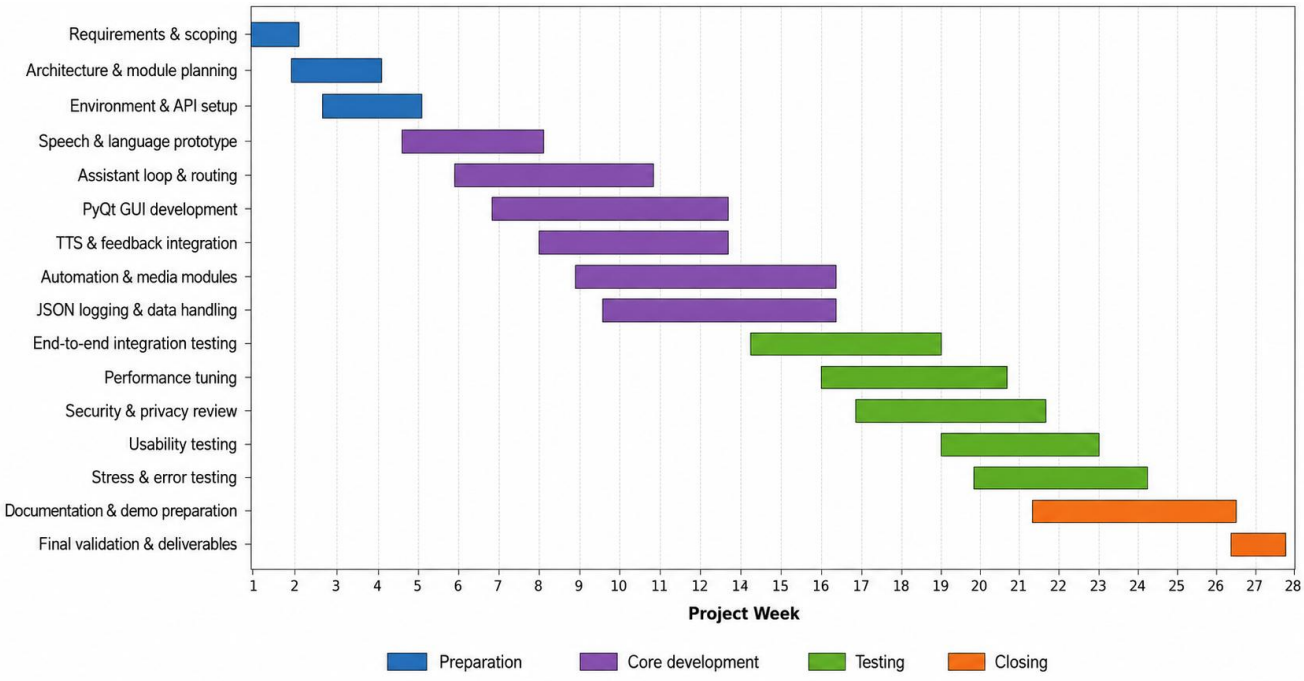


Figure 3.2: Proposed Gantt Chart for SmartVoice Development

Table 3.4: Feasibility Summary

Feasibility Area	Assessment	Evidence / Condition
Technical	Feasible as a modular Python prototype	Python 3.10.10, virtual environment, imports, local JSON files, and test suite were verified.
Operational	Feasible for a controlled user demo	Requires microphone permission, Chrome, valid API keys, and internet connection.
Economic	Variable operating cost	Cloud APIs may have quotas or usage costs; actual prices must be checked at deployment time.
Schedule	Feasible for iterative development	Existing modules and tests provide a working baseline; live demo evidence still needs capture.
Legal and ethical	Requires careful handling	Users should not submit sensitive data; third-party service terms and data practices apply.
Security	Partially addressed	Environment variables and. gitignore are present; no encryption/authentication/audit integrity confirmed.

3.6 Project Scheduling

The SmartVoice schedule is organised over twenty-eight weeks to make integration manageable and to reserve adequate time for testing and documentation. The early phase focuses on requirements, user scenarios, technical scope, module planning, and environment setup. This is followed by a speech and language prototype, main assistant-loop development, intent routing, chat/search integration, text-to-speech, GUI development, automation, media control, reminders, and logging. The middle of the schedule is where integration risk is highest because a feature may work in isolation but fail when connected to a live browser, microphone, external provider, or GUI state. For this reason, end-to-end testing is not left until the final week; it begins after the main modules are connected.

The final stages focus on performance review, privacy and security checks, usability testing, stress/error testing, screenshots, report writing, user manual preparation, poster preparation, and final demonstration rehearsal. The Gantt chart on the following landscape page uses colour to distinguish preparation, core development, testing, and closing activities. It is based on the sequence in the project proposal but has been revised to use the confirmed SmartVoice modules rather than unsupported offline-only modules. The chart

should be treated as the project-management plan. Any completed dates, changes in scope, or delays can be recorded later in the final presentation or appendix without altering the structure of the schedule.

3.7 Risk Management

Risk management is necessary because SmartVoice depends on both local and external components. The most significant technical risks are speech-recognition failure, ambiguous intent selection, browser-driver incompatibility, API quota exhaustion, network interruption, differences in external API responses, and errors in Selenium-based media control. Operational risks include user confusion about microphone state, unsupported commands, unexpected web-page behaviour, and incomplete evidence collection before submission. Privacy and security risks include accidental disclosure of API keys, sending sensitive prompts to third-party services, storing unnecessary text in local logs, and performing an action without the user fully understanding what will occur. The risk register in Table 3.5 identifies likelihood, impact, and practical mitigation for these conditions.

Mitigation begins with scope control. The assistant should execute only known and tested actions, return explicit status messages, and avoid silently failing when an external service is unavailable. The `.env` file should remain outside version control, and screenshots must never expose secret values. The project team should prepare a controlled demonstration sequence with known commands, a stable internet connection, a suitable microphone, and previously tested browser settings. Where possible, fallback behaviour should be simple: if real-time search is unavailable, the GUI should report the limitation; if YouTube control fails, the command should not be presented as successfully completed; and if speech recognition is unreliable, the team should document the test condition rather than claim a passing result. Longer-term mitigation includes confirmation prompts, stricter allow-lists, encrypted storage, local speech/NLU alternatives, and systematic integration tests.

Table 3.5: Risk Register

Risk	Likelihood	Impact	Mitigation
API quota, outage, or billing limit	Medium	High	Use clear GUI error messages; prepare local fallback explanation; test before presentation.
Chrome/WebDriver	Medium	High	Verify installed Chrome version and driver

Risk	Likelihood	Impact	Mitigation
mismatch			availability before the final demo.
Microphone permission or speech-recognition failure	Medium	High	Test device permissions; prepare typed fallback and pre-recorded evidence.
Unreliable network	Medium	High	Use stable network; perform service checks before demo; document cloud dependence.
Unexpected browser page layout changes	Medium	Medium	Test YouTube/search flow close to demonstration date; catch exceptions.
Sensitive query exposure	Medium	High	Warn users not to provide sensitive data; disclose third-party processing.
JSON corruption or accidental edit	Low	Medium	Back up Data/ files and validate JSON before demonstration.
Feature overstatement in report	Medium	High	Use confirmed-from-code labels and keep proposal-only features in future work.

3.8 Conclusion

The project-planning chapter confirms that SmartVoice is feasible as a controlled desktop-assistant prototype when its real dependencies are acknowledged. The codebase supports modular development, the documented environment has passed basic runtime and import checks, and the formal test suite provides an initial quality baseline. The project is operationally understandable because it uses a GUI and distinct task paths, while its economic boundary can be assessed using the documented student-project budget. The plan also makes clear that successful demonstration depends on testing the full voice, browser, network, and API flow on the intended machine.

The Gantt schedule, stakeholder analysis, feasibility assessment, and risk register provide a practical framework for completing the remaining evidence. They also protect the report from overstating features that were planned in the proposal but are not confirmed in the repository. The next chapter translates this planning work into functional and non-functional requirements, data needs, use cases, cost-benefit calculations, and strategic analysis.

Chapter 4
System Analysis

4.1 Introduction

System analysis converts the SmartVoice concept into clear operational requirements and measurable constraints. The central requirement is not simply that the assistant should answer questions. It must accept a user interaction, determine whether the request is a conversation, current-information search, reminder, automation instruction, media command, content-generation request, image-generation request, language command, or exit instruction, and then route that request to the correct module. The assistant must return an understandable result through the GUI and, where appropriate, speech synthesis. Each of these steps depends on preconditions such as valid configuration, network access, available API credentials, a functioning browser, a microphone, and reliable runtime files. The analysis therefore considers both what the system should do and the environment in which it can do it.

The requirements in this chapter are grounded in the confirmed repository structure and documented verification results. They intentionally separate implemented functions from future enhancements. For example, local JSON storage, reminder handling, UI interaction, basic tests, and service integration are confirmed, whereas offline-only processing, encrypted relational storage, user authentication, tamper-resistant logging, and comprehensive confirmation prompts are not confirmed. This distinction is important for a final-year report because functional requirements must be linked to code or demonstration evidence. Tables and diagrams in the following sections provide a structured representation of user needs, system behaviour, storage, use cases, cost-benefit assumptions, and strategic strengths and risks.

4.2 Requirement Analysis

The requirement analysis is organised into functional, non-functional, data, use-case, economic, and strategic categories. Functional requirements describe the observable actions the assistant must perform, such as launching the GUI, receiving speech input, selecting a route, producing a response, and maintaining reminders or logs. Non-functional requirements describe conditions under which those actions must operate, including platform compatibility, response clarity, privacy awareness, dependency management,

maintainability, and testability. For SmartVoice, a requirement is considered confirmed only when it is represented by the source code, configuration, test suite, or documented runtime evidence. A requirement may still be included as a future target, but it must be labelled as such rather than represented as an existing feature.

The analysis also recognises that a cloud-assisted assistant has external dependencies. A user cannot expect the same behaviour when network services are unavailable, API quotas are exhausted, or browser automation fails. Consequently, error handling and transparent feedback are essential requirements, not optional design polish. The requirement tables below provide identifiers that can be used during testing and demonstration. Each later screenshot or test record should reference the appropriate requirement identifier so that the final submission has a traceable path from requirement to implementation evidence and observed result.

4.2.1 Functional Requirements

The functional requirements in Table 4.1 define the actions SmartVoice is expected to perform in the current prototype. The assistant shall launch a PyQt5 desktop interface, provide a microphone state control, receive recognised speech through the Chrome/Selenium bridge, process Bangla or English input for routing, choose an intent category, and invoke a relevant backend module. Depending on the command, the system shall provide a general chat response, perform a real-time search, initiate a supported automation action, search or control YouTube, create a reminder, generate text, generate an image, change language behaviour, or close the assistant. The assistant shall display results through the GUI and may speak the response using Edge TTS. Local chat, reminder, performance, and comparison files shall be maintained where the relevant modules are used.

Functional requirements must be demonstrated in a controlled way because several actions depend on external services. A request should not be marked as passed merely because a module exists in the repository; the team should record the exact voice command, expected route, observed output, date, machine, network condition, and pass/fail result. Some functions, such as chat-log creation or reminder parsing, can be examined through files and unit tests. Other functions, such as live speech capture, real-time search, image generation, YouTube control, and text-to-speech, require live demonstration evidence. This requirement approach ensures that the report communicates both the breadth of the implementation and the limits of the available validation.

Table 4.1: Functional Requirements

ID	Requirement	Description
FR-01	Launch desktop GUI	The system shall launch the PyQt5 interface and display the main interaction screen.
FR-02	Microphone toggle	The user shall be able to activate/deactivate the microphone state from the GUI.
FR-03	Speech capture	The system shall capture speech through the configured Chrome/Selenium bridge.
FR-04	Language handling	The system shall handle Bangla and English input and translate Bangla for routing where needed.
FR-05	Intent routing	The system shall classify the request into a supported command category.
FR-06	General chat	The system shall generate a general response through the chat module when appropriate.
FR-07	Real-time search	The system shall request current search data through the search module when required.
FR-08	Automation	The system shall open supported websites/apps and perform available search actions.
FR-09	Media control	The system shall support YouTube search and supported playback-control commands.
FR-10	Reminder management	The system shall parse, store, check, and announce reminders.
FR-11	Image generation	The system shall request image generation when configured API access is available.
FR-12	Text-to-speech	The system shall convert appropriate assistant output into spoken feedback.
FR-13	Local logging	The system shall maintain supported local JSON chat, reminder, performance, and comparison files.

4.2.2 Non-Functional Requirements

The non-functional requirements define the quality conditions that make SmartVoice usable as a desktop prototype. The system is intended for a Windows environment using Python 3.10 or newer, an installed Chrome browser, a microphone, a speaker or headphones, a working virtual environment, and valid configuration values. The user interface should communicate states clearly: idle, listening, processing, success, or error. The code should be modular so that a failure in one service can be located and corrected without rewriting the entire application. Configuration should be separated from source

code through environment variables, while `.gitignore` should prevent accidental repository exposure of secret files. Runtime JSON files should remain readable and recoverable for prototype-level troubleshooting.

Performance and reliability must be treated honestly. The current source code and safe verification run confirm an executable environment and automated tests, but they do not establish a final voice-recognition accuracy, end-to-end latency, memory footprint, or user-satisfaction score. These metrics require repeatable live tests under defined conditions. Privacy is also a non-functional requirement because some tasks transmit user content to cloud providers. The GUI and documentation should therefore discourage entry of sensitive information and should explain external-service dependence. Future non-functional targets include offline alternatives, encrypted storage, stronger error recovery, explicit confirmation for sensitive automation, and more complete accessibility testing.

Table 4.2: Non-Functional Requirements

ID	Area	Requirement
NFR-01	Platform	Windows desktop environment; Python 3.10 or newer recommended by project setup.
NFR-02	Browser dependency	Google Chrome must be installed for browser-assisted speech and selected automation/media features.
NFR-03	Connectivity	Internet access is required for external AI, search, translation, and hosted generation services.
NFR-04	Security	Secrets shall be stored in environment variables and excluded from version control.
NFR-05	Usability	The GUI should make microphone state and response state visible to the user.
NFR-06	Reliability	The system should handle unavailable services and invalid commands without crashing.
NFR-07	Maintainability	Backend services shall remain separated into modules with identifiable responsibilities.
NFR-08	Performance	Response time should be measured in live tests; no benchmark value is claimed in this report.
NFR-09	Privacy transparency	The application should disclose that user content may be sent to third-party cloud services.
NFR-10	Testability	Core configuration, reminders, imports, and simple routing should be covered by automated tests.

4.2.3 Data and Storage Requirements

SmartVoice uses a file-based storage design rather than a relational database. Confirmed runtime artefacts include `Data/Chatlog.json`, `Data/reminders.json`, `Data/performance_log.json`, and `Data/comparison_log.json`, as well as voice-related HTML, audio, image, temporary, and GUI state files. This approach is suitable for an academic prototype because it is lightweight, transparent, and easy to inspect without installing a database server. It supports fast development and simplifies demonstration of local persistence. The documented inspection found the JSON files to be structurally valid, with the performance log containing records and the chat/reminder files available for runtime use. The data tables in this chapter describe each path, purpose, and observed status.

The same design also creates important constraints. JSON files do not automatically provide encryption, concurrent-write control, retention policies, audit integrity, user separation, or query capabilities comparable to a structured database. A corrupt or incomplete runtime file could affect a feature if the application does not handle the error gracefully. The project should therefore store only the minimum necessary information, avoid recording sensitive prompts, keep API credentials in `.env` rather than JSON, and clearly explain the limitation in the security section. Future versions can migrate persistent data to SQLite or another local database, add encryption, define retention and deletion policies, and include schema validation. The current choice is appropriate for the demonstrated prototype, but not sufficient on its own for a high-trust production deployment.

Table 4.3: Current Data and Storage Files

Path / File	Purpose	Observed Status During Inspection
Data/Chatlog.json	Stores local chat-log entries	Valid JSON list; 0 entries at inspection
Data/reminders.json	Stores reminders	Valid JSON list; 0 entries at inspection
Data/performance_log.json	Stores performance-related records	Valid JSON list; 33 entries at inspection
Data/comparison_log.json	Stores comparison-related records	Valid JSON list; 0 entries at inspection
Data/Voice.html	Supports speech-related	Present in repository evidence

Path / File	Purpose	Observed Status During Inspection
	browser/runtime activity	
Frontend/Files/	Runtime file communication area	Currently empty during inspection; code expects/writes .data files
.env	Stores service credentials and assistant configuration	Required; secret values not printed or included in report

4.2.4 Use-Case Analysis

Use-case analysis expresses SmartVoice behaviour from the user's perspective. The primary actor is a desktop user who starts the application, activates the microphone, speaks a Bangla or English request, and receives visual and spoken feedback. The user may ask a general question, request current information, open a site or application, search Google or YouTube, control YouTube playback, set a reminder, request generated text, request an image, change language mode, or exit. Each use case has a precondition: the program must be running, configuration must be available, and the particular feature must have its required services. For example, real-time search requires valid network/API access, while YouTube control requires an accessible browser and compatible page state.

The diagrams reserved in Figures 4.1 to 4.3 should show the boundary between the user, the GUI, the routing layer, specialised backend modules, external APIs, and local JSON files. The diagrams should not depict unimplemented modules as though they are operational. The most useful system-context diagram will show external cloud services separately from local code, while the data-flow diagram will show speech text, intent labels, API responses, action results, and logs moving through the system. The use-case diagram should distinguish user-visible actions from internal support functions. Together, these models help a supervisor understand the system without reading code and provide a logical basis for later test cases.

Table 4.4: Primary Use Cases

Use Case	Actor	Precondition	Main Outcome
Ask general question	Desktop user	App running; API configuration available	Chat answer is displayed and can be spoken.
Ask Bangla command	Desktop user	Microphone, browser speech bridge, translation	Input is handled for route selection and action.

Use Case	Actor	Precondition	Main Outcome
		configuration	
Request current information	Desktop user	Internet and Serper service configured	Search result is obtained and presented.
Set a reminder	Desktop user	App running	Reminder is parsed and stored in reminders JSON.
Play/control YouTube	Desktop user	Chrome/browser control available	Video search/playback or supported control is attempted.
Generate image	Desktop user	Hugging Face service key/quota available	Image-generation request is sent and result is made available.
Review performance	Project team	Log file exists	Logged performance/statistics can be reviewed through helper scripts.

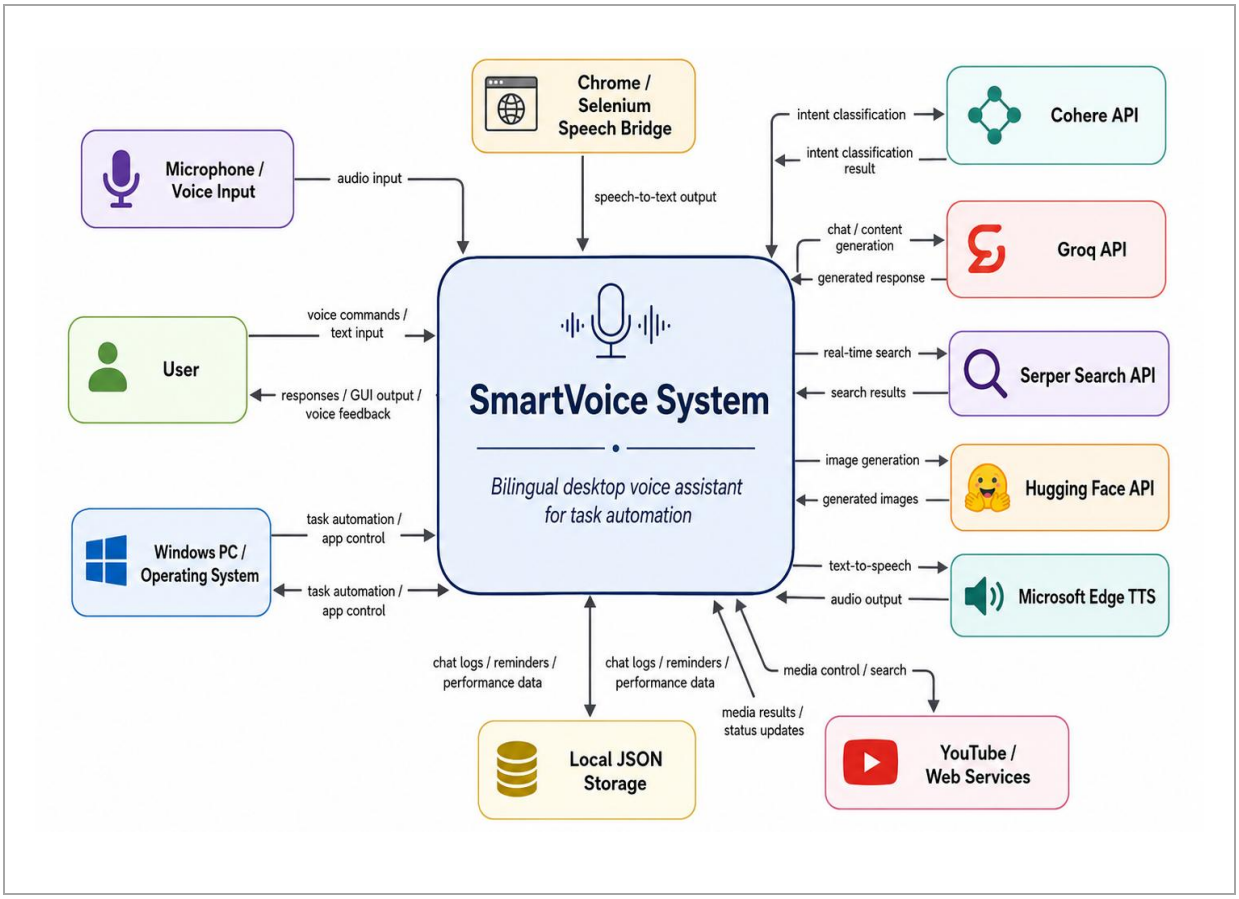


Figure 4.1: SmartVoice System Context Diagram

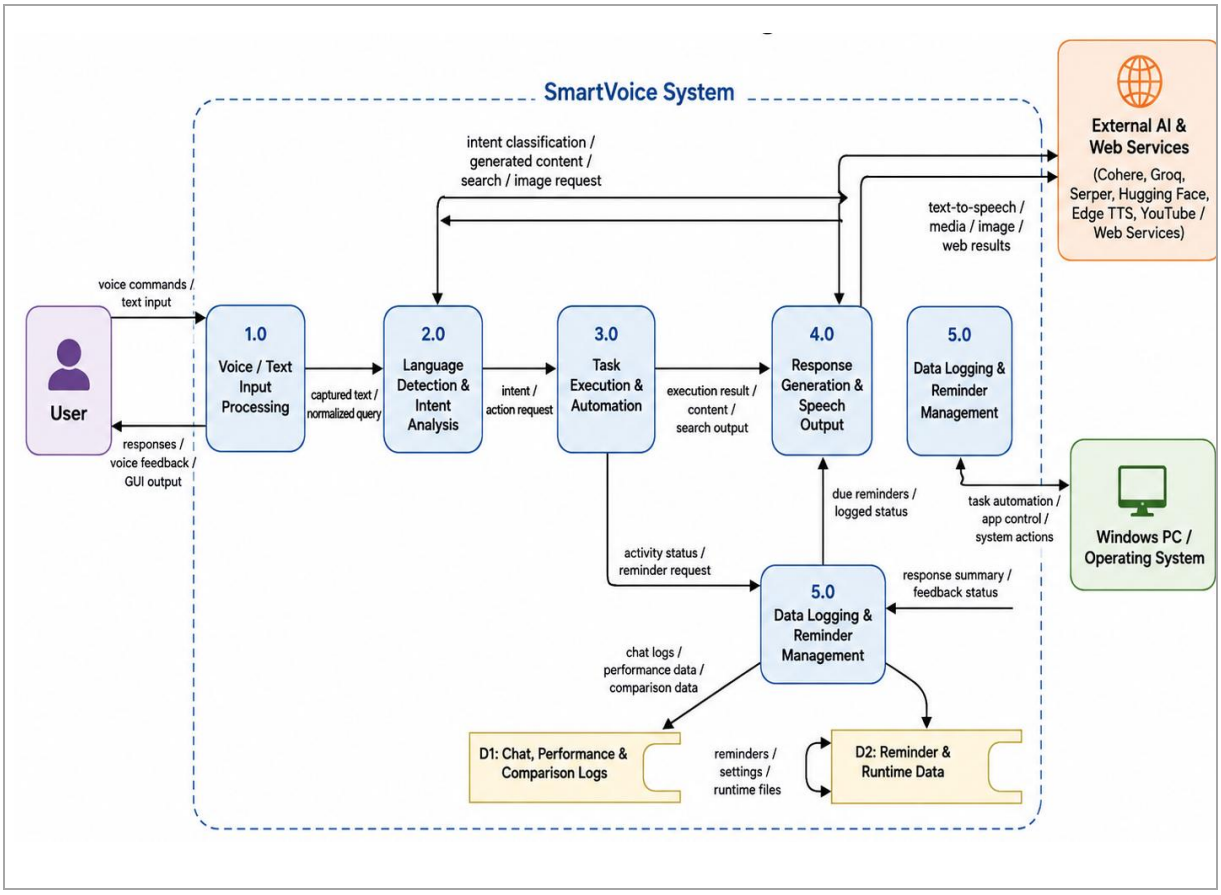


Figure 4.2: Level-0 Data Flow Diagram

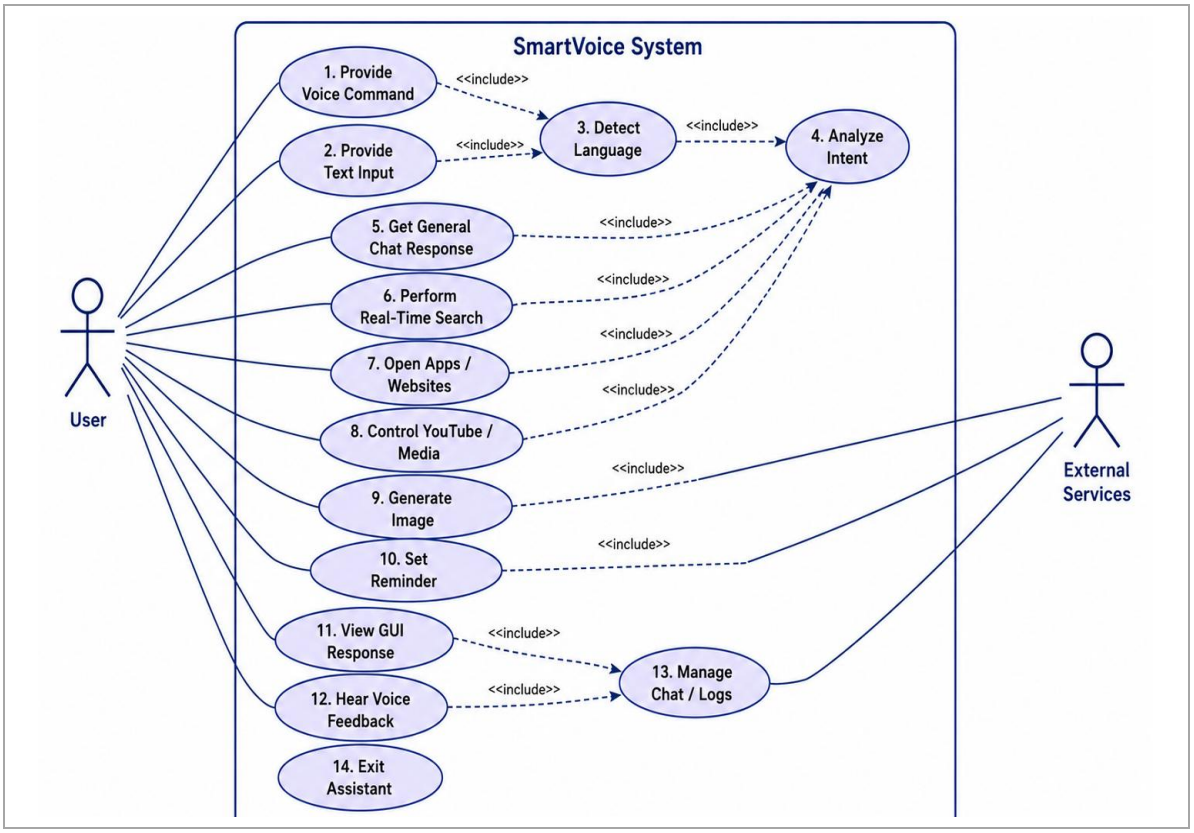


Figure 4.3: SmartVoice Use Case Diagram

4.2.5 Cost-Benefit Analysis

The SmartVoice budget supplied by the team totals BDT 15,025. It includes no material or equipment purchase because the project is developed using existing personal computer resources. The main cost is BDT 14,025 for five paid AI subscription services used during research, development, documentation, coding support, debugging, system design, implementation, and testing. Printing and binding are estimated at BDT 500, and miscellaneous expenses are estimated at BDT 500. The direct initial project cost is therefore calculated as C0 = 14,025 + 500 + 500 = BDT 15,025. This figure is a documented student-project cost, not a commercial production budget. It does not include unpaid student labour, personal computer depreciation, personal internet bills, or future enterprise deployment costs.

Because SmartVoice is an academic prototype with no confirmed sales revenue, the financial benefit is assessed through a transparent productivity-time-saving scenario. The base-case calculation assumes ten regular users, an average saving of eight minutes per user per working day, 250 working days per year, and a conservative value of BDT 60 per productive hour. The estimated annual benefit is Bannual = N x (m/60) x D x V = 10 x (8/60) x 250 x 60 = BDT 20,000. A maintenance reserve of BDT 1,500 per year is included for connectivity, API, and update contingencies. Therefore the annual net benefit is BDT 18,500, the three-year net benefit is BDT 40,475 after total three-year costs of BDT 19,525, the benefit-cost ratio is 60,000 / 19,525 = 3.07, and the simple payback period is 15,025 / 18,500 = 0.81 years, or about 9.7 months. These are illustrative feasibility estimates only; they must not be presented as measured savings until a user study is completed.

Table 4.5: Qualitative Cost-Benefit Analysis

Item	Cost / Benefit Type	Current Assessment
Python, PyQt5, Selenium, JSON tools	Development cost	Low direct license cost; development time and maintenance remain required.
Cloud language/search/image services	Operating cost	Variable cost/quota; must be measured from current provider plans and usage.
Chrome and microphone	Operational requirement	Existing hardware/software may reduce initial cost, but compatibility testing is required.
Reduced manual task switching	User benefit	Potential efficiency gain; no formal time study has been completed.
Bilingual voice	User benefit	Can improve accessibility for Bangla/English

Item	Cost / Benefit Type	Current Assessment
interaction		users; requires live usability evaluation.
Local logs	Project benefit	Supports debugging and demonstration; no encryption or audit integrity confirmed.
External dependency	Operational risk	May increase ongoing cost and reduce availability during internet/service disruption.

Quantitative method and assumptions: Let N be the number of regular users, m be average minutes saved per user per working day, D be working days per year, and V be the estimated value of one productive hour. The annual productivity benefit is Bannual = N x (m/60) x D x V. For the base scenario, N = 10, m = 8, D = 250, and V = BDT 60; therefore Bannual = 10 x (8/60) x 250 x 60 = BDT 20,000. A maintenance and contingency reserve of BDT 1,500 per year is used only for feasibility modelling. These calculations are illustrative and must not be reported as measured commercial revenue or verified user savings.

Table 4.5A: SmartVoice Project Cost Summary and Cost-Benefit Assumptions

Component	Formula / Basis	Amount (BDT)	Status
Paid AI subscription services	Five paid subscriptions used for research, development, documentation, coding support, debugging, design, implementation and testing	14,025	Team budget
Printing and binding	Required project-report printing and binding	500	Team budget
Miscellaneous	Maximum allocated miscellaneous amount	500	Team budget
Initial project cost (C0)	14,025 + 500 + 500	15,025	Documented total
Annual maintenance reserve	Illustrative provision for connectivity, API and update contingency	1,500 / year	Scenario assumption
Base annual productivity benefit	10 users x (8/60) hour/day x 250 days/year x BDT 60/hour	20,000 / year	Scenario assumption

Table 4.5B: Three-Year Scenario-Based Cost-Benefit Analysis

Scenario	Benefit Assumption	Annual Benefit	3-Year Benefit	3-Year Cost	Net Benefit	BCR	Simple Payback
Conservative	5 users, 5 min/day, 250 days/year, BDT 60/hour	6,250	18,750	19,525	-775	0.96	3.16 years
Base	10 users, 8 min/day, 250 days/year, BDT 60/hour	20,000	60,000	19,525	40,475	3.07	0.81 years
Optimistic	15 users, 10 min/day, 250 days/year, BDT 60/hour	37,500	112,500	19,525	92,975	5.76	0.42 years

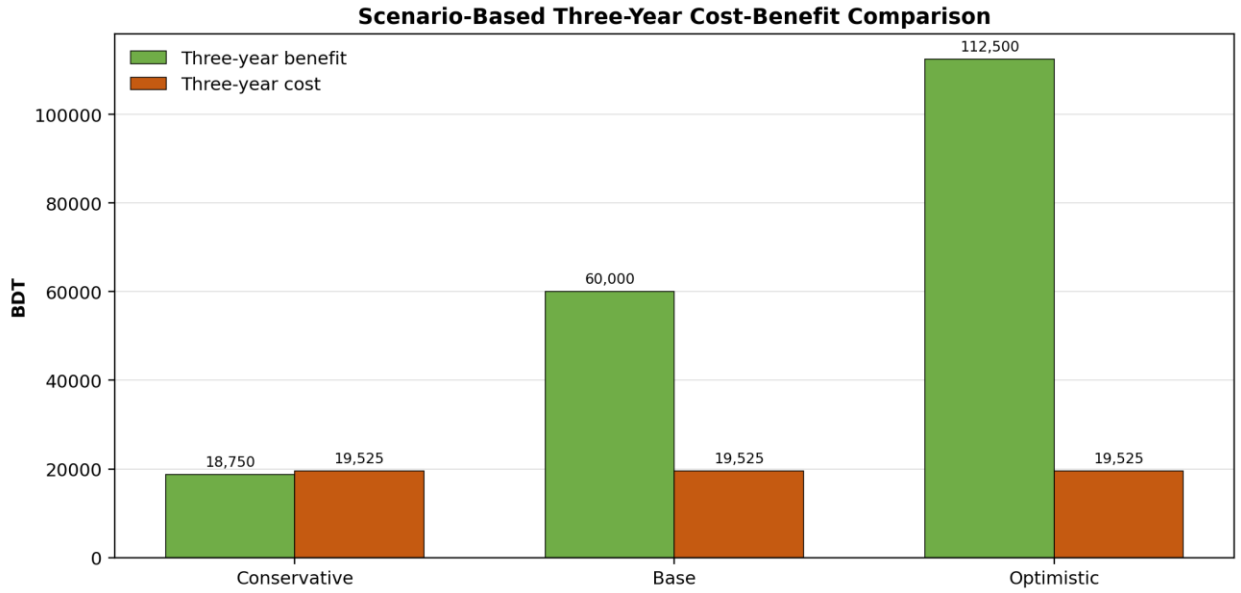


Figure 4.4: Scenario-Based Three-Year Cost-Benefit Comparison

Base-case result: Total three-year cost = C0 + (3 x 1,500) = 15,025 + 4,500 = BDT 19,525. Total three-year benefit = 3 x 20,000 = BDT 60,000. Net benefit = 60,000 - 19,525 = BDT 40,475. Benefit-cost ratio (BCR) = 60,000 / 19,525 = 3.07. Simple payback = 15,025 / (20,000 - 1,500) = 0.81 years, approximately 9.7 months. At a 10% discount rate, the estimated three-year net present value is approximately BDT 30,982. These figures show that the base case is economically favourable only if the assumed adoption and time saving are realised.

4.2.6 SWOT Analysis

The SWOT analysis assesses SmartVoice as a prototype rather than as a finished commercial product. Its principal strengths are its modular Python architecture, bilingual command flow, PyQt5 graphical interface, separation of specialised modules, local JSON runtime storage, test coverage for selected deterministic functions, and broad demonstrated feature scope. The project brings multiple user-facing capabilities together in one desktop environment, making it suitable for an academic demonstration of integration engineering. It also benefits from a clear repository structure in which configuration, speech, routing, chat, search, automation, reminders, image generation, media control, logging, and tests are represented by distinct modules.

Its weaknesses include dependence on cloud APIs, Chrome/WebDriver compatibility, external quotas, unknown live accuracy and latency, non-encrypted JSON storage, limited safety confirmation, and the absence of a confirmed authenticated multi-user architecture. Opportunities include replacing individual services with offline alternatives, extending Bangla support, adding encrypted storage, building stronger confirmation policies, conducting user studies, packaging the application, and using performance logs for reproducible evaluation. Threats include API pricing changes, service outage, browser updates, privacy concerns, accidental disclosure of credentials, and user overconfidence in automation. The value of the SWOT analysis is that it prevents the report from describing the prototype only through its strengths; it directly links strategic choices to the implementation and future-work roadmap.

Table 4.6: SWOT Analysis

Area	Analysis
Strengths	Modular architecture; bilingual command flow; single desktop interface; integration of chat, search, reminders, media, image generation and text-to-speech; automated unit-test baseline.
Weaknesses	Cloud dependence; no encrypted database; no user authentication; no verified end-to-end live benchmark; browser/WebDriver sensitivity; feature limits driven by external services.
Opportunities	Offline/local speech models; local intent routing; stronger privacy controls; packaging; usability study; richer Windows automation; improved bilingual support.
Threats	API price/quota changes; service outages; browser changes; incorrect speech recognition; sensitive data exposure; misinformation from AI/search results; platform policy changes.

4.3 Conclusion

The system analysis establishes that SmartVoice requires a combination of functional features, environmental conditions, data-management rules, user interactions, economic assumptions, and strategic controls. The assistant is feasible as a prototype because its requirements map to identifiable modules and the basic development environment has been verified. However, the analysis also demonstrates that the system cannot be evaluated solely by the number of integrated features. Reliability depends on the entire path from microphone input to routing, external service response, browser automation, GUI feedback, and data logging. The requirement tables provide a practical checklist for testing this path.

The cost-benefit calculation shows that the project can be economically justifiable under a clearly stated base scenario, but the outcome depends on regular use and actual time saved. The resulting figures are scenario-based rather than measured commercial results. The SWOT analysis and risk discussion reinforce the need for transparent privacy notices, cautious automation, and proof through live evidence. The next chapter presents the design architecture that connects the requirements to the code modules and the visual elements that will be inserted by the team.

Chapter 5
System Design

5.1 Introduction

The SmartVoice design is based on modular orchestration. Rather than placing speech recognition, language handling, intent classification, chat generation, real-time search, automation, reminders, media control, image generation, text-to-speech, and GUI rendering in one large script, the project separates these responsibilities into dedicated modules. The central `main.py` file coordinates their workflow. This design choice improves readability, testing, debugging, and future replacement of individual components. For example, the current Chrome/Selenium speech-recognition path could later be replaced with an offline speech engine without requiring the entire user interface or reminder system to be rebuilt. Similarly, the file-based storage layer can later be migrated to an encrypted database while retaining the same user-facing workflow.

A good system design must also make boundaries visible. Some modules operate locally, including the PyQt5 interface, JSON handling, reminder logic, basic pattern matching, and orchestration. Other capabilities depend on external providers or browser services. The design diagrams in this chapter should therefore distinguish local execution from cloud-assisted execution, data storage from API calls, and user-visible screens from hidden backend modules. This prevents an inaccurate presentation of SmartVoice as an offline-only system and gives a clearer explanation of where failures, privacy risks, and future enhancements belong.

5.2 Overall Architecture

The overall architecture can be explained through five logical layers. The first is the interaction layer, which contains the PyQt5 GUI, graphical assets, home screen, chat screen, microphone state, and displayed feedback. The second is the speech and language layer, where the Chrome/Selenium bridge captures speech text and LanguageManager assists with Bangla/English handling and routing support. The third is the decision layer, which uses the Cohere model and the SimplePatternMatcher baseline to select an intent category. The fourth is the service and action layer, which contains Chatbot, RealtimeSearchEngine, Automation, ReminderManager, ImageGeneration,

YouTubeController, and TextToSpeech. The fifth is the storage and evidence layer, which maintains JSON runtime files and performance/comparison logs.

The main orchestration loop connects these layers. Once a command is captured, the application normalises it, determines the active language, routes it to the appropriate module, receives a text or action result, updates the GUI, triggers speech output where required, and records data when the relevant logging function is invoked. A request may involve both local and external steps. For example, a real-time query starts locally in the GUI and routing layer, uses Serper and Groq externally, then returns to the local interface and optional Edge TTS output. The architecture diagram should represent this round trip with separate boundaries for the local application, browser services, cloud APIs, and runtime files.

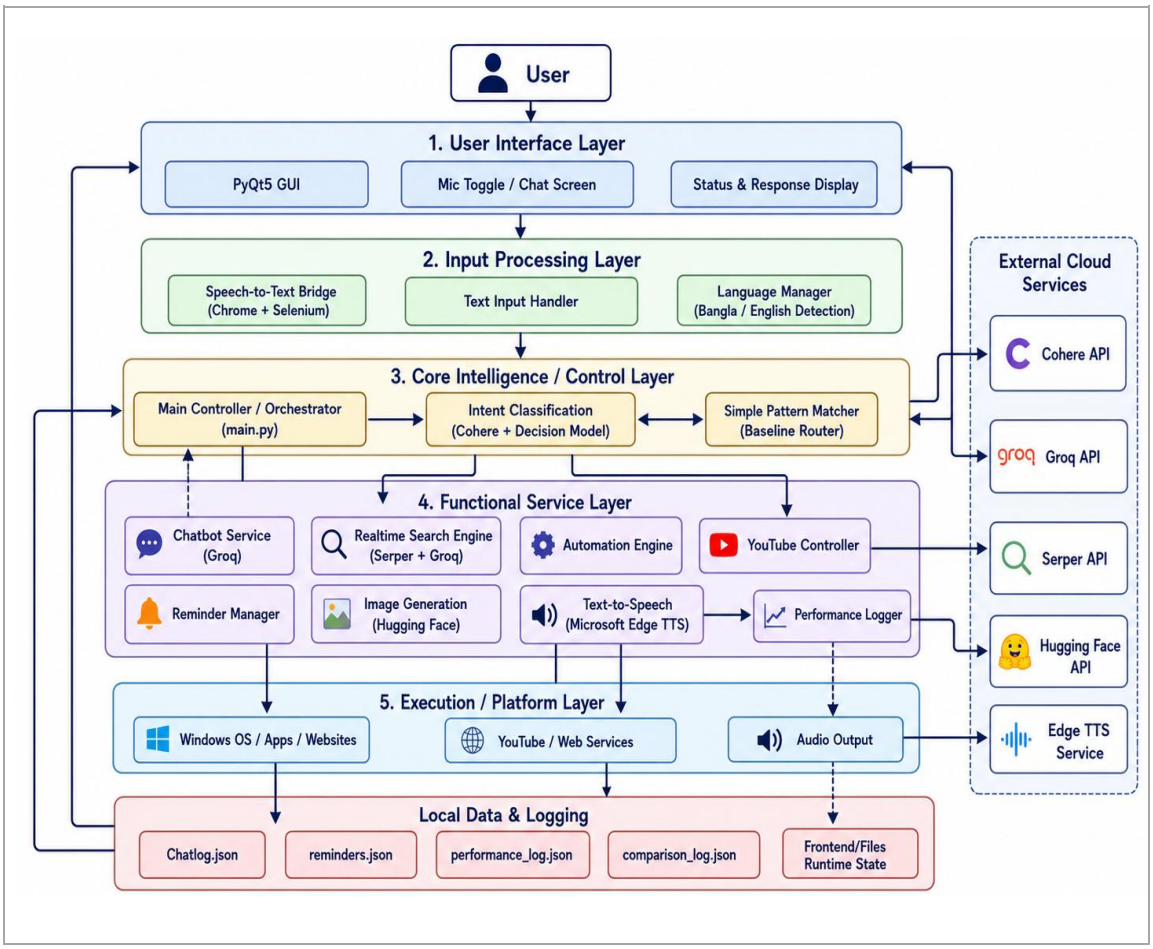


Figure 5.1: Overall SmartVoice Architecture

Table 5.1: Architecture Modules and Responsibilities

Module / File	Primary Responsibility	Key Inputs / Outputs
main.py	Orchestrates startup, state files, mic toggle, routing, action execution, GUI and TTS flow	User state, command text, action result, status updates
Frontend/GUI.py	Renders desktop interface and visual state	User interaction, chat/status display, assets
Backend/SpeechToText.py	Captures user speech through browser-based mechanism	Microphone input to text
Backend/LanguageManager.py	Detects/handles Bangla and English, supports translation route	Text to normalized/translated text
Backend/Model.py	Cohere-assisted command decision/routing	Command text to intent labels
Backend/SimpleDecisionMaker.py	Local simple pattern-based routing baseline	Command text to simple route
Backend/Chatbot.py	Generates general assistant response through Groq	Question to text response
Backend/RealtimeSearchEngine.py	Retrieves real-time search information and forms response	Current query to search-supported answer
Backend/Automation.py	Opens/selects applications/sites and executes supported actions	Automation command to task result
Backend/ReminderManager.py	Parses, stores, checks, and returns due reminders	Reminder command to JSON reminder state
Backend/ImageGeneration.py	Requests image generation through Hugging Face	Prompt to image output
Backend/TextToSpeech.py	Generates spoken output	Response text to speech audio
Backend/YouTubeController.py	Supports YouTube search/play/control behavior	Media command to browser/media action
Backend/PerformanceLogger.py	Stores performance/comparison-related data	Event/result to log record

5.3 Module Design

The module design follows a responsibility-based approach. `Config.py` loads environment settings and provides clear messages when required values are missing. `SpeechToText.py` manages the browser-assisted recognition bridge. `LanguageManager.py` coordinates language state and translation support. `Model.py`

invokes the Cohere decision model, while `SimpleDecisionMaker.py` provides a lightweight pattern-based reference route. `Chatbot.py` provides general generated responses, and `RealtimeSearchEngine.py` combines current search results with answer generation. `Automation.py` handles supported open/close/search/content/system actions, whereas `YouTubeController.py` addresses playback and control behaviour. `ReminderManager.py` parses, stores, and retrieves due reminders. `ImageGeneration.py` handles image requests, and `TextToSpeech.py` synthesises spoken responses. `PerformanceLogger.py` and the helper scripts support local evidence review.

The practical advantage of this separation is fault isolation. If image generation fails because a provider is unavailable, the chat and reminder functions do not need to stop. If a new service is introduced, it can be implemented as a module with a clear input/output contract. The design also enables unit tests to focus on deterministic functions such as configuration loading, reminder parsing, import side effects, and pattern routing. The figure placeholders in this section should show module interactions in a way that mirrors the repository rather than copying an earlier proposal diagram that includes unimplemented offline databases or policy components.

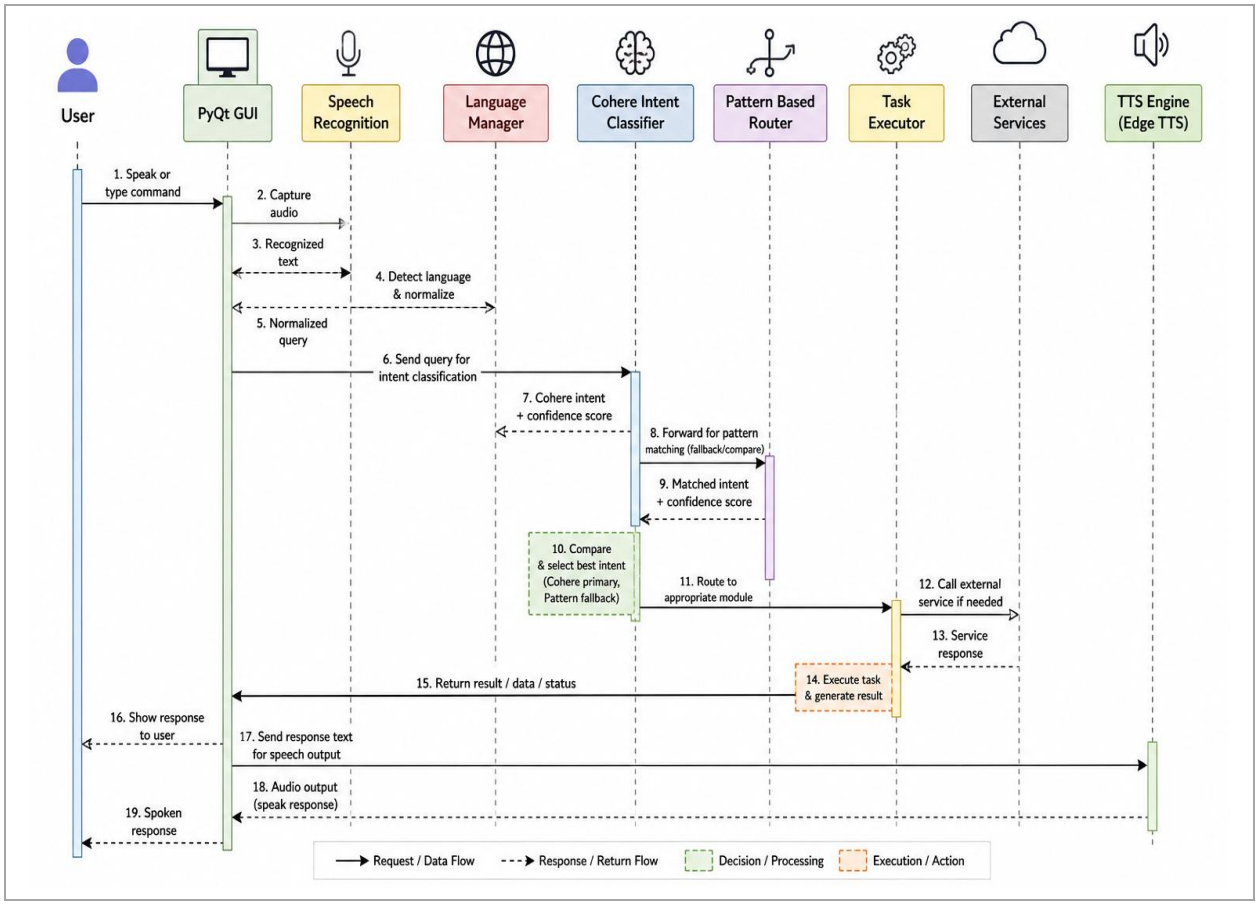


Figure 5.2: Intent Routing and Response Sequence

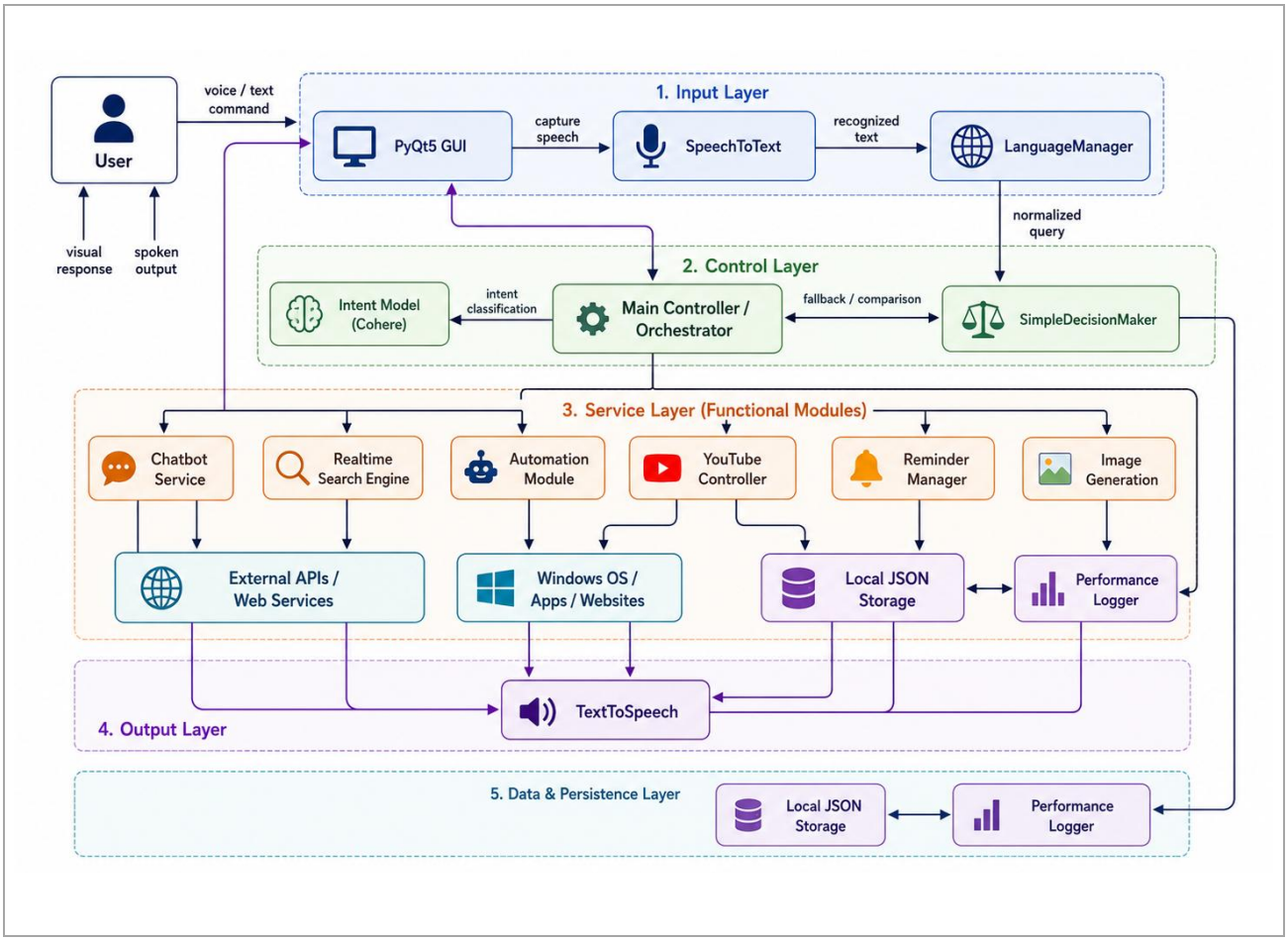


Figure 5.3: Module Interaction Diagram

5.4 Intent Routing and Command Flow

Intent routing is the decision point that transforms recognised speech text into an actionable system path. SmartVoice uses a Cohere-based decision model for routing, supported by a SimplePatternMatcher baseline. The route labels in the implementation include general, realtime, open, close, play, system, content, Google search, YouTube search, YouTube control, reminder, generate image, language, and exit. This design is necessary because one natural-language input can imply very different system behaviour. A general question should produce an explanatory response, while a time-sensitive query should search the web, a reminder request should be parsed into a time and stored, and a YouTube command should activate browser-control logic. The route therefore acts as the bridge between language understanding and specialised execution.

The full command flow begins when the user activates the microphone through the GUI. The speech module produces text, the language manager helps identify the language and any required translation/normalisation, and the decision layer assigns an intent. The selected backend module processes the request and returns a result. The result may be a

text response, a status message, a reminder confirmation, a browser action, or an image path. The GUI presents the result, and TextToSpeech may speak it aloud. Performance information may be logged depending on the route. The design should include error branches: an unsupported intent, missing configuration, unavailable network service, failed API call, missing browser driver, or invalid reminder syntax must produce a visible response rather than a silent failure.

Table 5.2: Frontend Elements

Screen / Element	Purpose	Evidence to Insert
Home/microphone screen	Starting screen that presents user-facing voice/microphone state	Insert screenshot in Figure 5.5
Chat screen	Displays user input, assistant response, and conversation flow	Insert screenshot in Figure 5.6
Custom top bar	Provides desktop window branding/control presentation	Insert in GUI screenshot if visible
Status area	Shows listening, thinking, or response-related state	Insert in suitable GUI screenshot
Graphics assets	GIFs and other visual resources used by GUI	Optional asset inventory screenshot

Table 5.3: Intent Categories and Actions

Intent Category	Expected Action
general	Generate conversational response.
realtime	Retrieve current information through search module.
open	Open supported website/application.
close	Close supported application where implemented.
play	Play requested media through browser/media flow.
system	Handle system-related request or state action.
content	Generate written content.
google search	Perform a Google search action.
youtube search	Search YouTube.
youtube control	Attempt supported media controls such as pause/resume/volume.
reminder	Parse/store/check reminder.
generate image	Request image generation through Hugging Face.
language	Change or manage language handling.

Intent Category	Expected Action
exit	End assistant workflow.

5.5 Data Storage Design

The current SmartVoice storage design uses JSON and runtime files. This is a deliberate prototype-level choice because it allows data to be inspected directly and avoids the complexity of running a separate database service. Chat history, reminders, performance entries, and comparison records are represented in local files under the `Data/` directory. Voice/browser runtime state, generated media, temporary audio, and interface coordination artefacts may also be written to project folders. The data-storage diagram should show which modules read or write each record type: ReminderManager interacts with reminder data, chat and response flows may use chat history, PerformanceLogger writes evaluation data, and the GUI or speech components use temporary runtime state.

A secure storage design requires more than persistence. The current source evidence does not confirm encryption, user accounts, role permissions, tamper-evident logs, retention limits, or deletion rules. Therefore the report should state that file-based storage is appropriate for an individual academic prototype but should not be treated as a privacy-complete production design. The recommended future direction is to introduce a structured local database, validate schemas, encrypt sensitive records, define a retention period, protect write operations, and give users a clear way to export or delete their data. This boundary is particularly important because a voice assistant can receive personal queries and because performance records may reveal usage patterns.

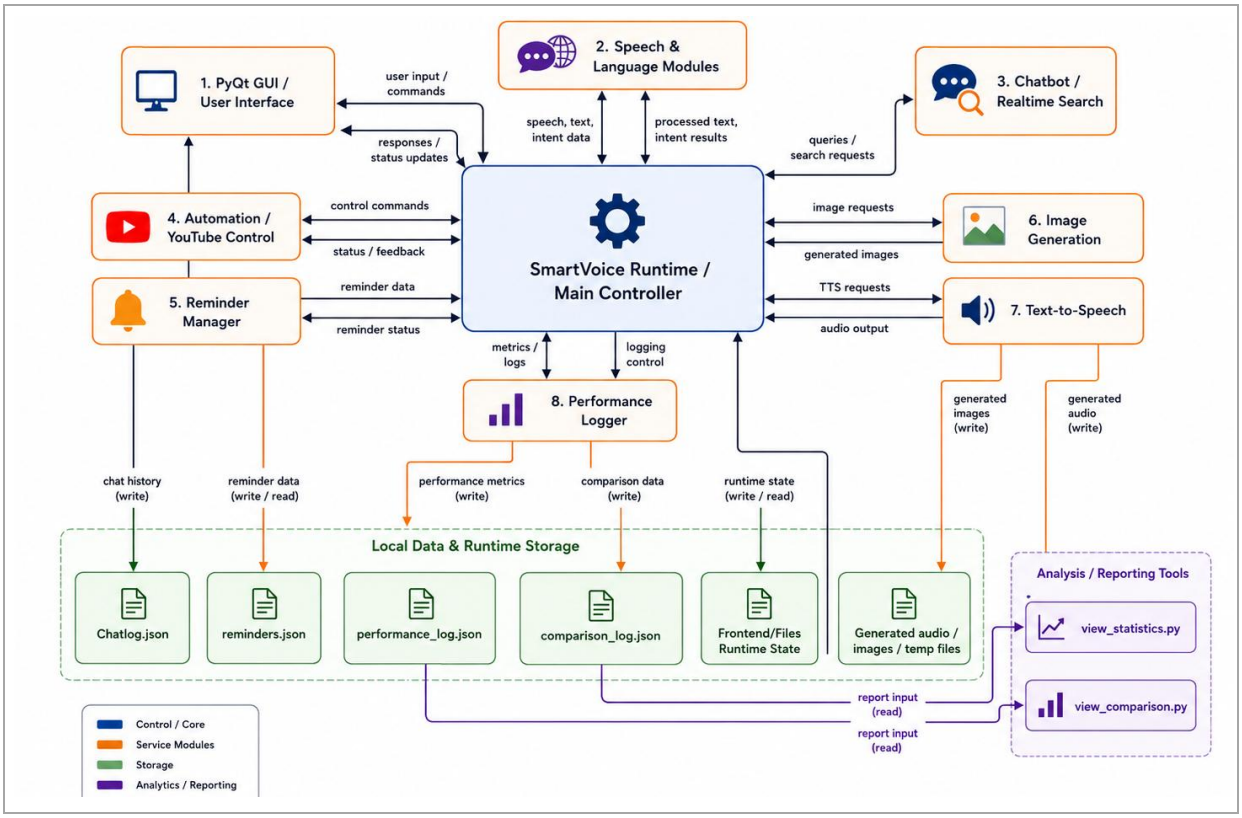


Figure 5.4: Data Storage and Logging Flow

Table 5.4: Data-Handling Design

Data Type	Current Storage	Design Rationale	Limitation
Chat history	Data/Chatlog.json	Simple local history for continuity and review	No encryption or retention policy confirmed
Reminders	Data/reminders.json	Lightweight persistence for due-checking	No calendar synchronization or multi-user access
Performance data	Data/performance_log.json	Records performance-related runtime evidence	Exact schema/metric interpretation must be documented from code
Comparison data	Data/comparison_log.json	Supports comparison-related evidence	No benchmark conclusion should be claimed without methodology
Runtime state	Frontend/Files and related .data files	Enables file-based UI/backend coordination	Directory was empty during inspection; runtime behavior needs live confirmation
Secrets	.env	Keeps API keys out of source code	Not encryption; local device security is still required

5.6 User Interface Design

The SmartVoice user interface is designed around visibility of state. A voice assistant must not leave the user unsure whether the microphone is disabled, listening, processing a command, waiting for a network response, or completing an action. The PyQt5 GUI provides a home/microphone screen and a chat-oriented interface supported by custom graphics and a top bar. The user interaction should begin with a clear microphone control and should display text feedback before or alongside speech output. The interface should also make errors understandable by reporting whether a service is unavailable, a command is unsupported, a reminder was not recognised, or a browser action could not be completed. Screenshots inserted in Figures 5.5 and 5.6 should show these states with real evidence from the running application.

Usability should be considered for both technical and non-technical users. Command examples should be simple, short, and visible in documentation. The system should use consistent language in buttons, notifications, and error messages. Bangla/English handling should not force the user to understand the internal translation path. Visual elements should remain readable on ordinary desktop screens, while spoken output should be optional enough that users can use the application silently when needed. Future interface work could add an activity history, explicit privacy notice, quick-command suggestions, a text-input fallback, accessibility settings, selectable voice settings, and confirmation dialogs for actions with side effects.

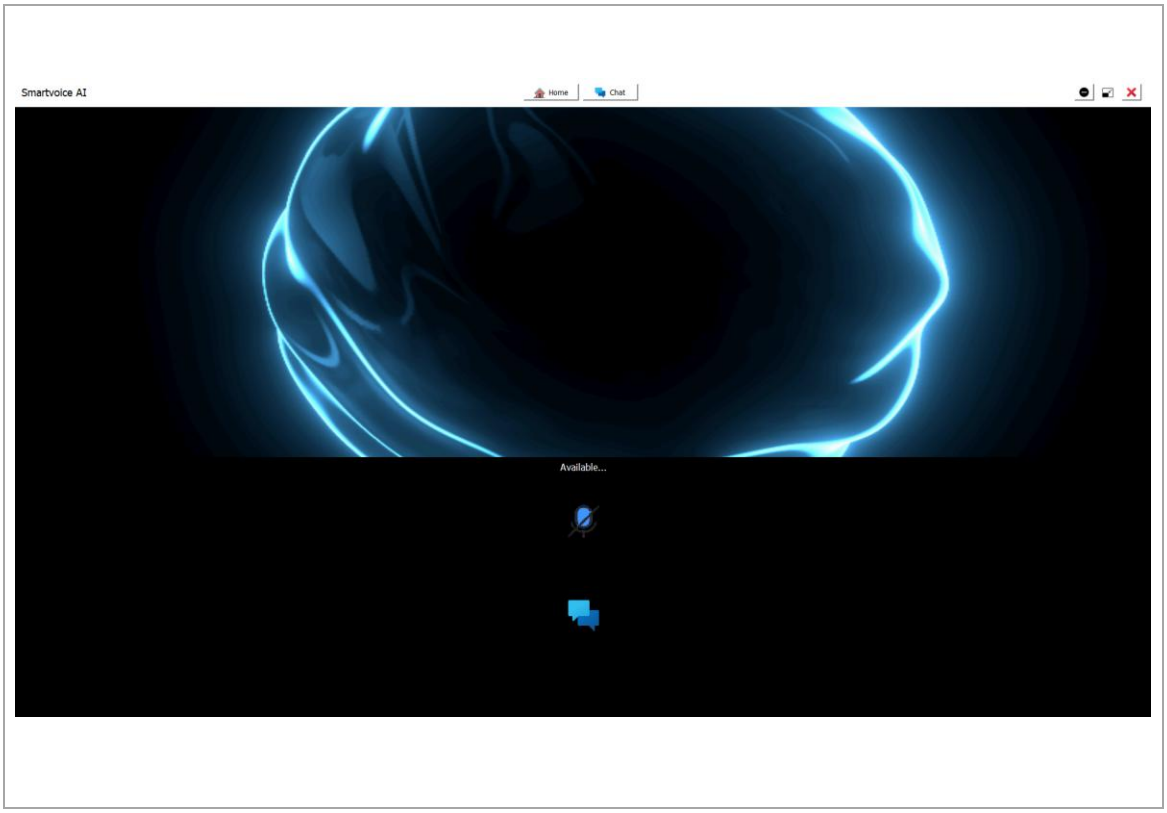


Figure 5.5: Home and Microphone Screen

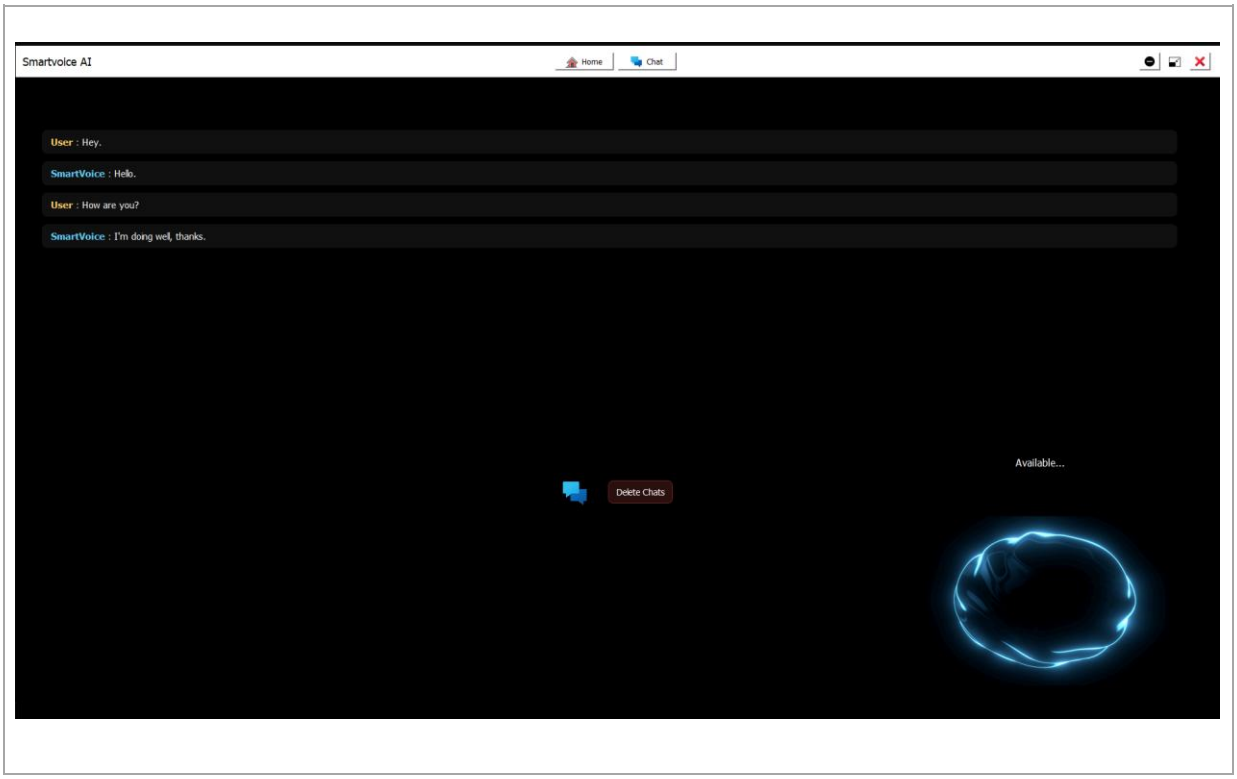


Figure 5.6: Chat Screen

5.7 Security and Privacy Design

The present security design begins with configuration separation. API keys and settings are expected to be supplied through `.env`, while `.gitignore` excludes this file from repository tracking. This is an important first control because it reduces the risk of accidentally publishing credentials with source code. The system also separates task modules, which makes it easier to inspect which component contacts an external service. However, these controls are limited. The source code does not confirm user authentication, role-based authorisation, encrypted JSON storage, tamper-resistant logs, consent workflows for every external call, or a full allow-list/confirmation policy for sensitive automation. The report must therefore describe security as a current boundary plus a future design direction.

Privacy is especially important because speech text, search queries, generated content prompts, and image-generation prompts may be transmitted to third-party services. Users should be advised not to enter sensitive data, and the documentation should identify when a feature relies on cloud processing. Local logs should contain only information required for development and testing, and screenshots should be reviewed to ensure that they do not reveal tokens, account details, or private user content. The future design should include explicit user consent, confirmation before externally sending a request, data minimisation, local/offline alternatives, encryption at rest, secret rotation, audit trails, and safe command controls. Table 5.5 and Figure 5.7 provide a clear comparison between current measures and the controls required for a higher-trust version.

Table 5.5: Security and Privacy Design

Area	Current Control	Gap / Future Control
API credentials	Environment variables; .gitignore excludes .env	Use secret manager and rotation process for deployment
User data	Local JSON runtime files	Encrypt sensitive data and define retention/deletion policy
Third-party processing	Cloud services are configured for key features	Show privacy notice and minimize sent content
Automation safety	Task modules route commands	Add confirmation prompts and allow-list sensitive actions
User identity	No confirmed authentication	Add login/profile/role controls if system becomes multi-user
Auditability	Performance/comparison JSON	Add tamper-evident audit records and

Area	Current Control	Gap / Future Control
	logs	structured event metadata
Error handling	Some configuration errors are handled	Add user-friendly failure states and consistent error reporting

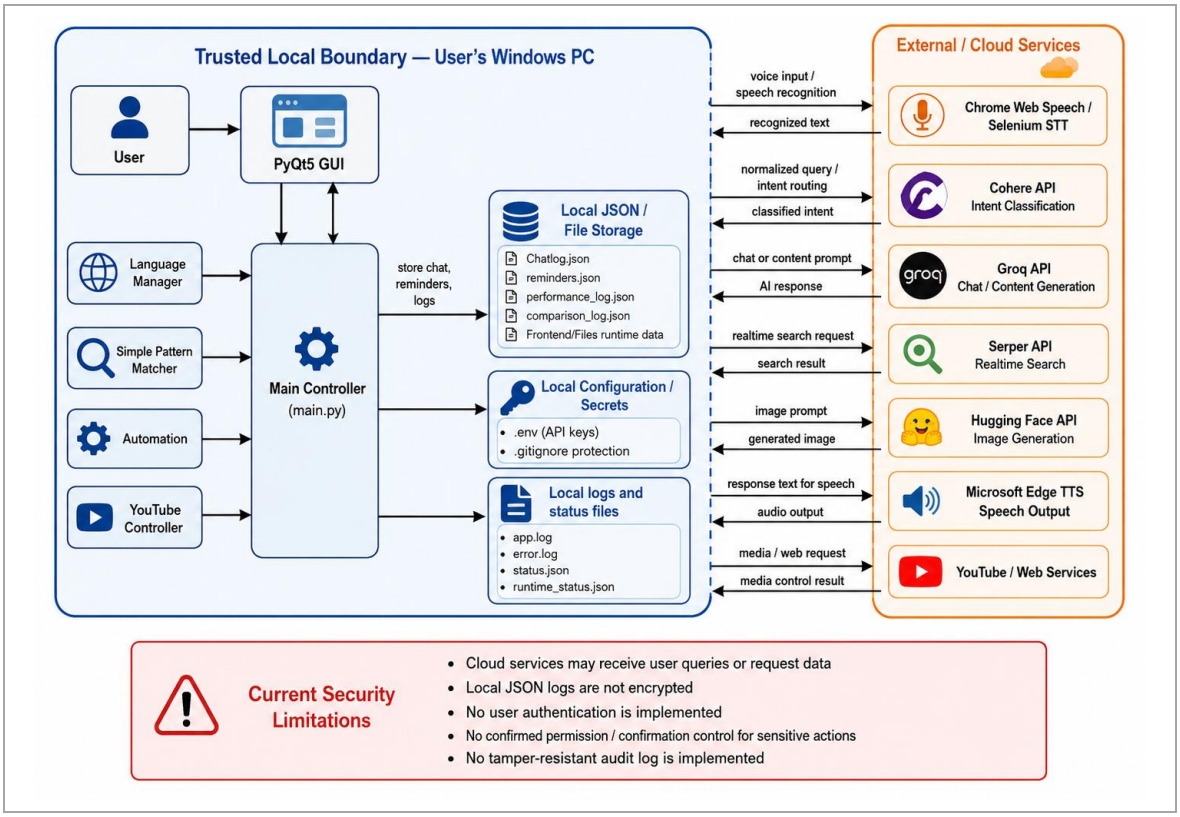


Figure 5.7: Security and Privacy Boundary

5.8 Conclusion

The system design of SmartVoice has been developed around the principles of modularity, maintainability, traceability, and controlled integration. Instead of placing all functionalities within a single large program, the design separates the system into distinct layers and modules responsible for user interaction, speech recognition, language processing, intent routing, task execution, response generation, text-to-speech output, and local data storage. This structure reflects the actual Python repository, where modules such as SpeechToText.py, LanguageManager.py, Model.py, Chatbot.py, RealtimeSearchEngine.py, Automation.py, ReminderManager.py, ImageGeneration.py, TextToSpeech.py, and PerformanceLogger.py perform specialised responsibilities. The central main.py component coordinates these modules and manages the overall workflow of the assistant.

The design allows SmartVoice to process a user request through a structured sequence. A user first interacts with the PyQt5 graphical interface and activates the microphone. The speech-recognition component captures the command, while the language-management module supports Bangla and English processing. The intent-routing layer then determines the most suitable action path, such as general conversation, real-time search, reminder creation, image generation, YouTube control, browser automation, or content generation. After the selected service completes its task, the result is returned to the graphical interface and may also be spoken through the text-to-speech module. This organised flow makes the system easier to understand, test, debug, and extend.

Another important strength of the design is its flexibility for future improvement. Since the current application is divided into separate modules, individual services can be replaced without redesigning the entire system. For example, the browser-assisted speech-recognition mechanism can later be replaced with an offline speech-recognition engine. Similarly, the Cohere-based cloud intent-routing component can be supplemented or replaced with a local machine-learning or rule-based model. The current JSON-based storage system can also be migrated to an encrypted database solution, such as SQLite with additional security controls, while preserving the existing reminder, chat-log, and performance-logging workflow. This modular approach provides a practical foundation for future development toward a more private, reliable, and independently deployable assistant.

The design also clearly identifies the current technical boundaries of SmartVoice. Several visible capabilities depend on cloud APIs, browser automation, microphone permissions, Chrome compatibility, network connectivity, API credentials, and external-service availability. Therefore, the system should not be represented as a fully offline assistant or a complete privacy-preserving solution. Although .env configuration and .gitignore provide basic protection for secret keys, the present implementation does not confirm encrypted user data, user authentication, role-based access control, tamper-resistant audit logs, or confirmation prompts for sensitive automation actions. These limitations are important because a voice assistant may process personal requests, search queries, reminders, and generated content.

Overall, the SmartVoice design provides a suitable architecture for a final-year software engineering prototype. It demonstrates how a bilingual desktop assistant can combine graphical interaction, speech input, language handling, AI-supported routing,

browser-based automation, cloud services, local logging, and spoken feedback within one coordinated Windows application. The design balances practical implementation with future extensibility while clearly documenting the distinction between confirmed current capabilities and planned enhancements. The next chapter explains how this architecture was implemented, configured, tested, and evaluated through automated tests, live demonstrations, and runtime performance evidence.

.

Chapter 6
Implementation and Testing

6.1 Introduction

This chapter documents what can be confirmed from the SmartVoice implementation and what still requires live demonstration. The codebase contains the planned modular structure: a Python main loop, PyQt5 GUI, speech-recognition bridge, language manager, Cohere decision model, pattern-based router, Groq chatbot, Serper real-time search, automation, reminders, image generation, Edge TTS, YouTube control, local JSON data files, and tests. The documented verification run also established that the Python environment and virtual environment are available, package consistency was checked, selected modules imported safely, and thirteen formal tests completed successfully. These are meaningful implementation findings because they show that the source code can be loaded and that selected deterministic behaviours have automated coverage.

Not every implementation claim can be verified through static inspection. Live speech recognition, browser control, YouTube interaction, real-time search, API calls, image generation, text-to-speech playback, and GUI behaviour depend on a device, network, browser, microphone, API credentials, quotas, and visible runtime state. The report therefore distinguishes safe verification from end-to-end demonstration. Reserved figures and tables in this chapter should be completed using genuine screenshots and test results collected by the team on the final demonstration machine. This approach is more professional than presenting a static code module as proof that a live service succeeded.

6.2 Development Environment and Execution

The documented development environment uses Python 3.10.10 with an existing `.venv` virtual environment. The project includes `Requirements.txt`, `.env.example`, and `README.md` to support installation and configuration. During the safe verification run, `pip check` reported no broken requirements, and selected imports of `main`, `Frontend.GUI`, `Backend.SpeechToText`, and `Backend.ImageGeneration` completed successfully. Chrome was detected in the Windows Program Files location, although `chrome.exe` was not directly available through the standard command lookup. The repository also contains two packages listed in the requirement file that were not present in the inspected virtual environment: `duckduckgo_search` and `screen_brightness_control`.

The current active module path uses `ddgs`, and no active code import of the brightness-control package was confirmed, so these are dependency-alignment issues rather than confirmed application failures.

The main demonstration command is `python main.py` after activating the virtual environment. This command should be used for the full assistant because it starts the GUI and worker orchestration loop. Running `python Frontend\GUI.py` starts only the frontend and does not provide the complete assistant pipeline. Individual backend files can be invoked during development, but they do not represent the integrated product. First-time browser use may require `webdriver-manager` to download or select a compatible ChromeDriver, and cloud features require valid keys in `.env`. The team should document the exact computer, operating system, Python version, Chrome version, microphone, network condition, and date used for the final live test.

Table 6.1: Development Environment and Dependency Status

Item	Verified Status	Comment
Python runtime	Python 3.10.10	Verified in environment
Virtual environment	.venv available and working	Verified in environment
Package consistency	pip check passed	No broken requirements reported
Google Chrome	Installed in Program Files	Get-Command did not resolve chrome.exe; Selenium runtime should be tested live
Formal unit tests	13 tests passed in 5.441 seconds	Reported test run completed successfully
Safe imports	main, Frontend.GUI, Backend.SpeechToText, Backend.ImageGeneration imported	Safe import checks passed
Runtime JSON files	Valid JSON lists	Chatlog=0, reminders=0, performance=33, comparison=0 at inspection
Missing declared packages	duckduckgo_search and screen_brightness_control	Dependency mismatch to resolve/document
External services	Not verified live	Require network/API call/browser/microphone/visible UI

Table 6.2: Implementation Module Summary

Module	Implementation Summary	Confirmation Status
Config.py	Loads configuration/environment variables and provides configuration handling	Confirmed from code inspection
GUI.py	Implements PyQt5 GUI with home/mic and chat-related interface	Confirmed from code inspection
SpeechToText.py	Uses browser/Chrome/Selenium speech workflow	Confirmed from code inspection; live microphone not tested
Model.py	Uses Cohere for decision/intent routing	Confirmed from code inspection; live API not tested
SimpleDecisionMaker.py	Provides simple local pattern routing baseline	Confirmed; covered by tests
LanguageManager.py	Manages Bangla/English detection and translation route	Confirmed from code inspection; live language accuracy not tested
Chatbot.py	Uses Groq for general chat/content responses	Confirmed from code inspection; live API not tested
RealtimeSearchEngine.py	Uses Serper and response generation for up-to-date queries	Confirmed from code inspection; live API not tested
Automation.py	Executes browser/system task actions	Confirmed from code inspection; live action result not tested
ReminderManager.py	Parses/stores/checks reminders	Confirmed; covered by tests
ImageGeneration.py	Calls Hugging Face image generation	Confirmed from code inspection; live API not tested
TextToSpeech.py	Provides Edge TTS spoken response	Confirmed from code inspection; live audio not tested
YouTubeController.py	Supports YouTube search/control	Confirmed from code inspection; live browser/media control not tested
PerformanceLogger.py	Maintains performance/comparison data	Confirmed from code inspection

6.3 Module Implementation

Implementation begins in `main.py`, which coordinates startup state, microphone interaction, recognised text, language handling, decision routing, specialised task execution, GUI updates, and spoken feedback. The frontend is implemented in `Frontend/GUI.py`, while the backend is split into focused services. `Config.py` handles configuration; `SpeechToText.py` manages the Chrome/Selenium bridge; `LanguageManager.py` supports bilingual routing; `Model.py` and `SimpleDecisionMaker.py` perform route selection; `Chatbot.py` and `RealtimeSearchEngine.py` provide conversational and current-information responses; `Automation.py` and `YouTubeController.py` handle supported actions; `ReminderManager.py` maintains reminders; `ImageGeneration.py` handles visual-generation requests; and `TextToSpeech.py` uses Edge TTS and playback mechanisms. `PerformanceLogger.py` and helper scripts support inspection of runtime evidence.

This implementation is notable for its direct module integration rather than for a custom trained machine-learning model. Cohere, Groq, Serper, Hugging Face, and Chrome Web Speech provide external intelligence or media capability, while Python code coordinates the decision and response flow. The image placeholders in Figures 6.1 to 6.7 should show real results from the GUI launch, a Bangla command, a general response, a real-time search, a reminder, YouTube control, and image generation. Captions must state exactly what was demonstrated and should avoid asserting that a feature is universally reliable from a single screenshot.

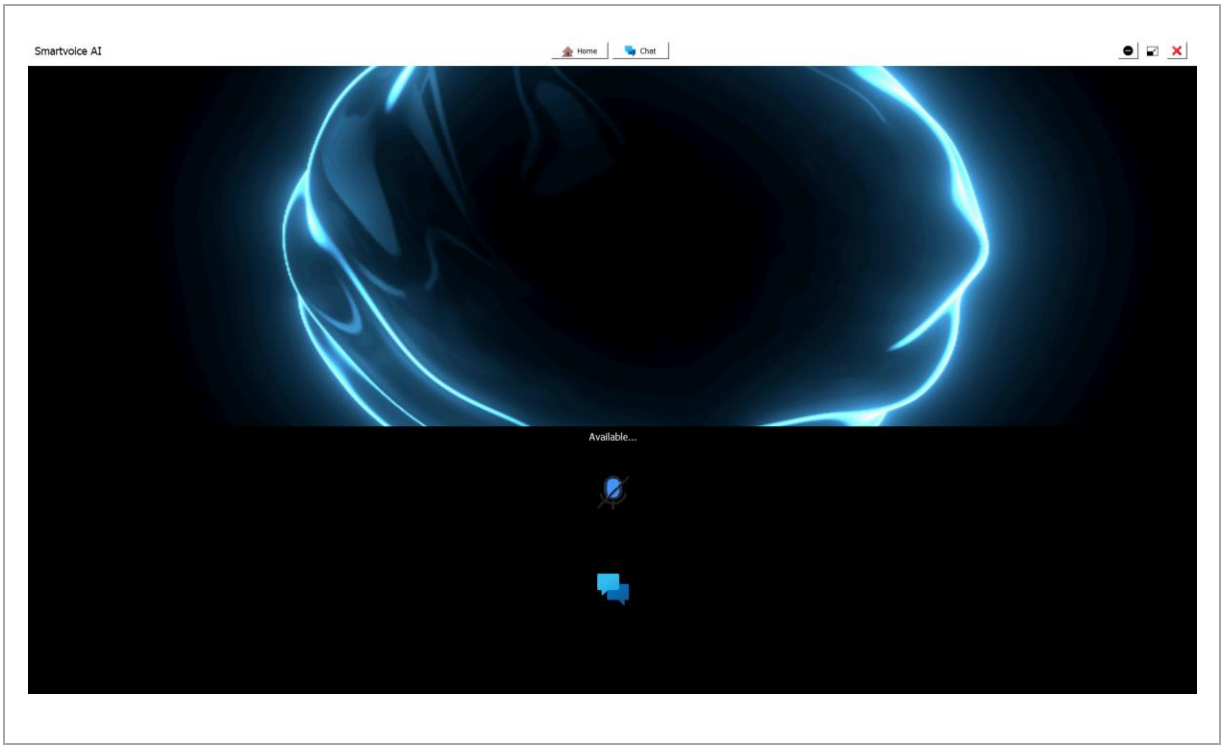


Figure 6.1: SmartVoice Main Window at Launch

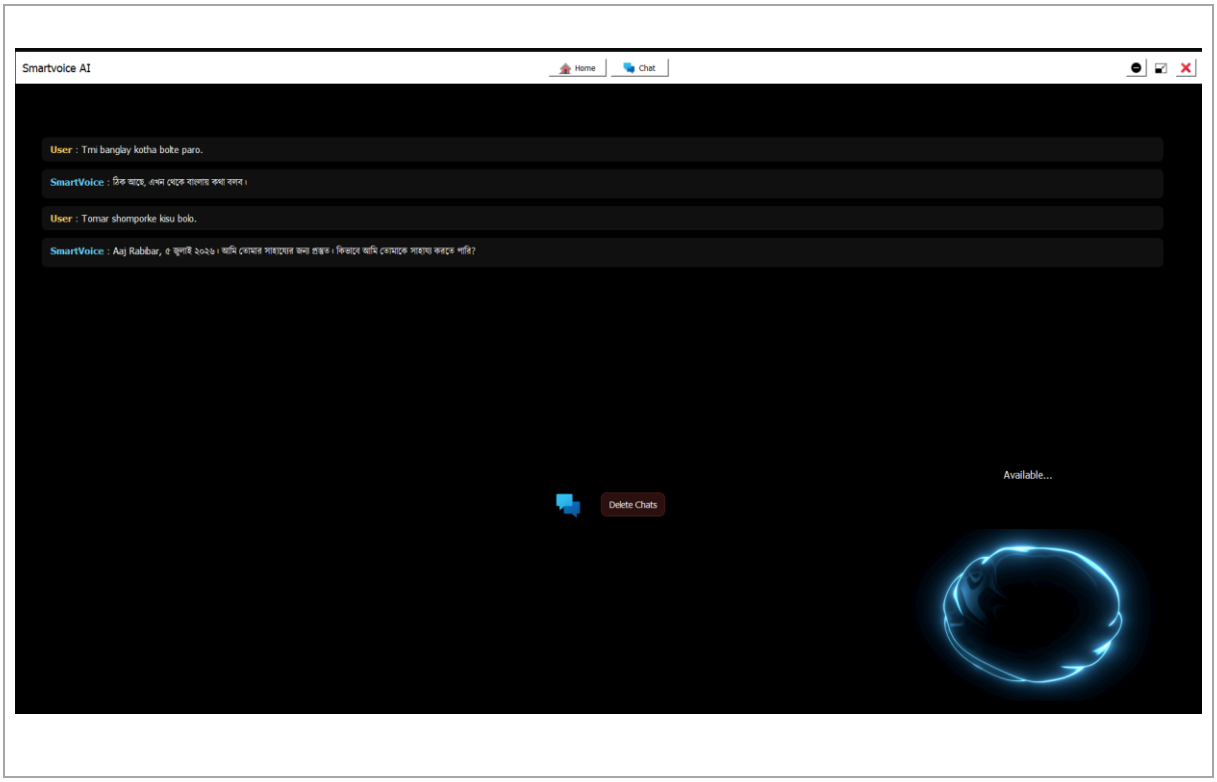


Figure 6.2: Bangla Voice Command Demonstration

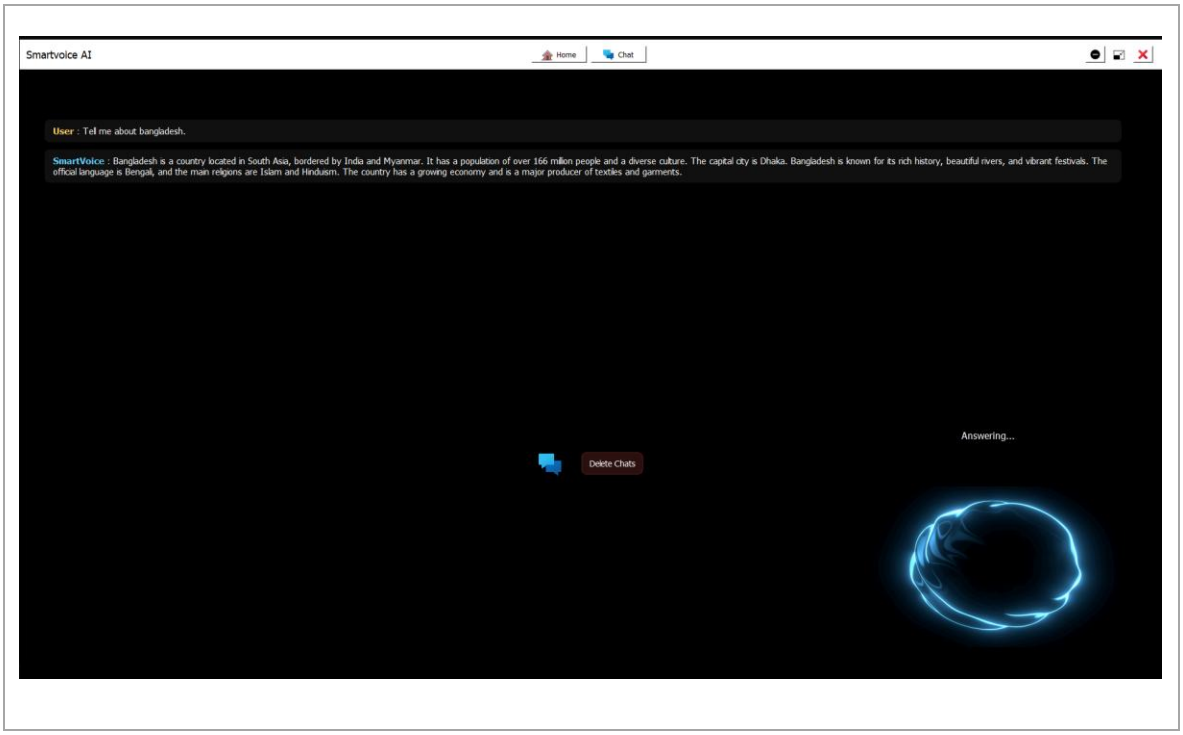


Figure 6.3: General Chat Result

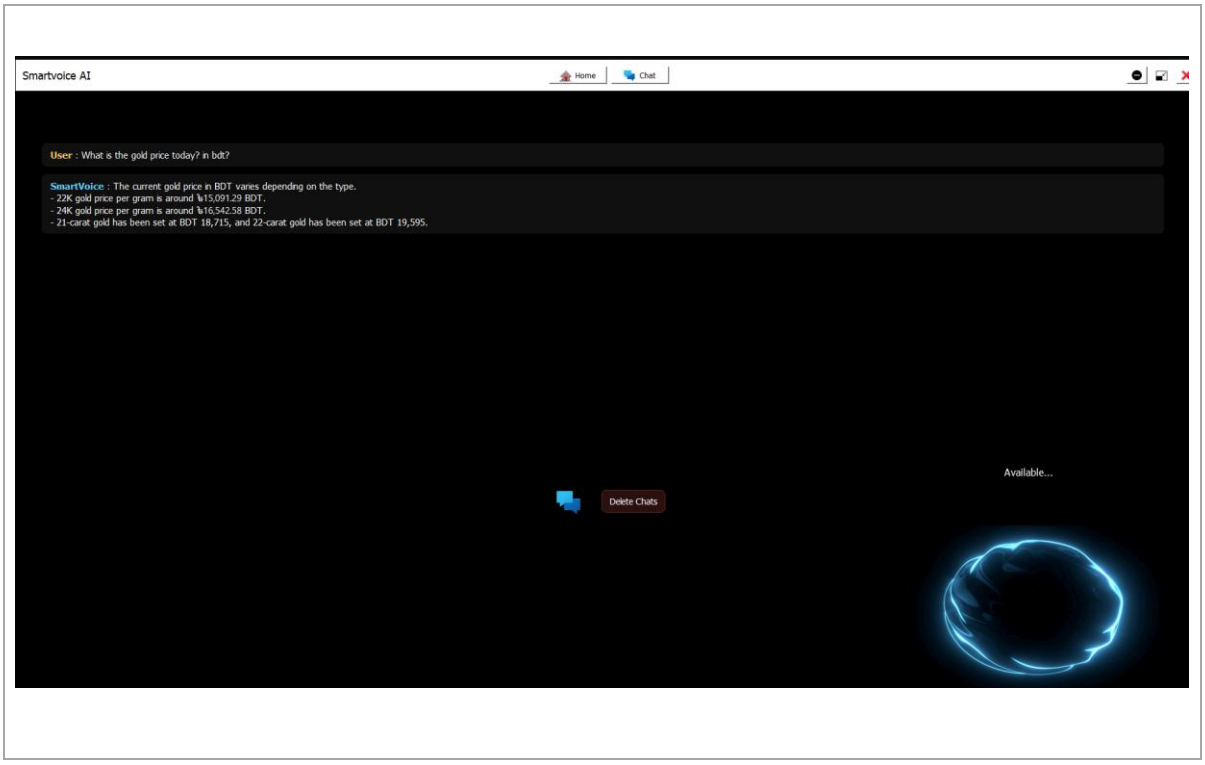


Figure 6.4: Real-Time Search Result

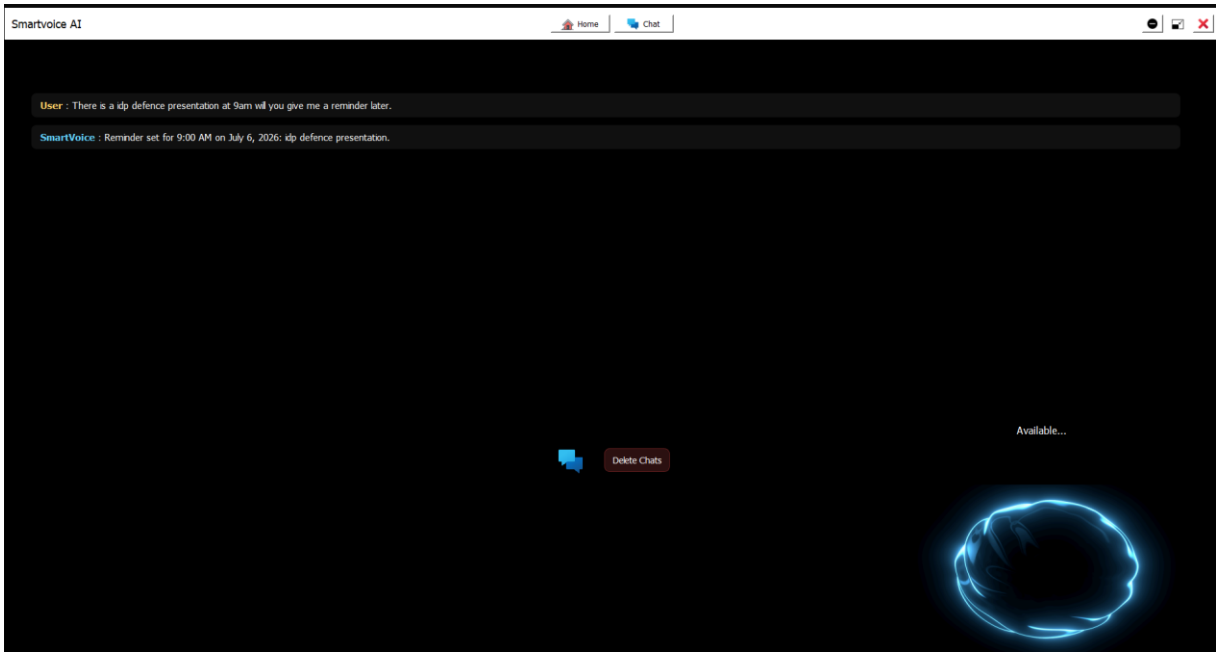


Figure 6.5: Reminder Creation or Alert

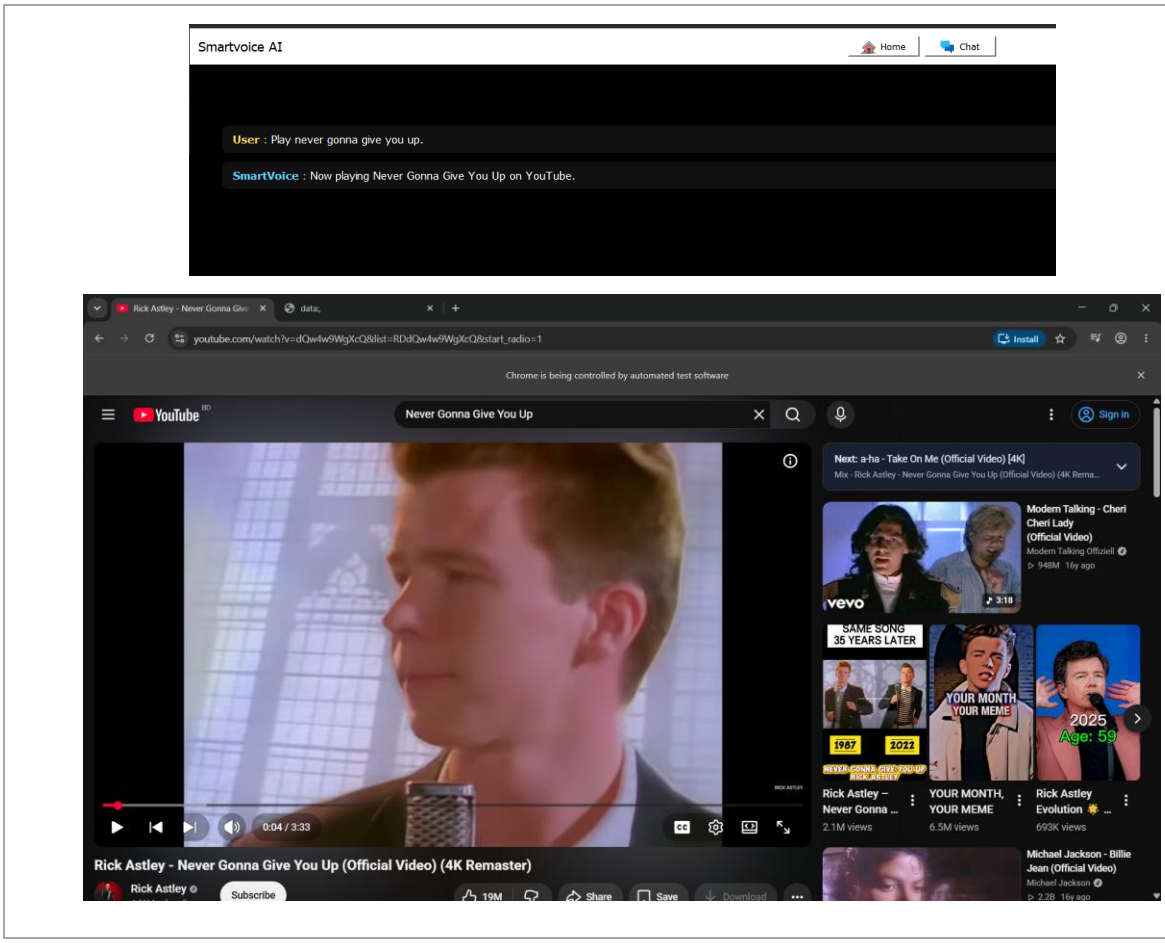


Figure 6.6: YouTube Play or Control Demonstration

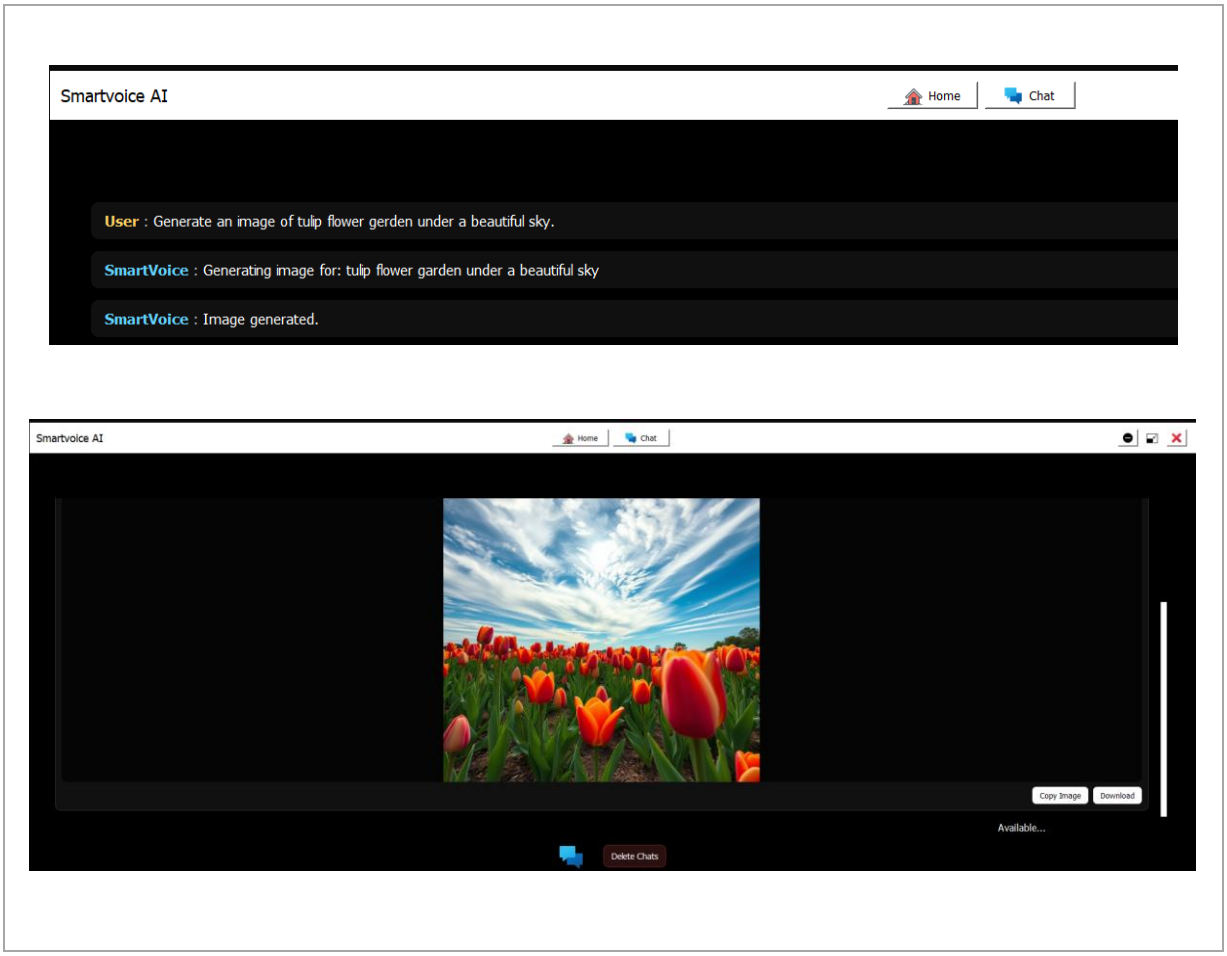


Figure 6.7: Image Generation Output

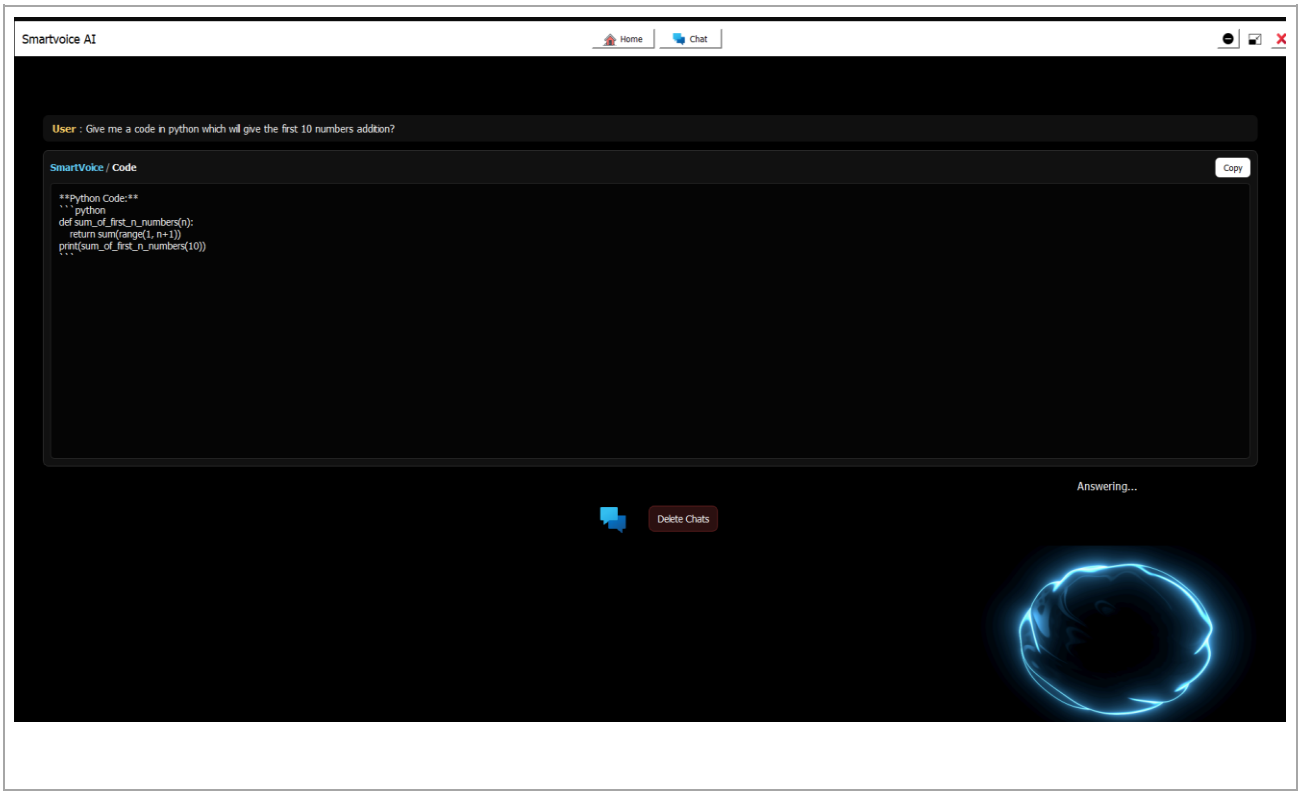


Figure 6.8: Code Generation Output

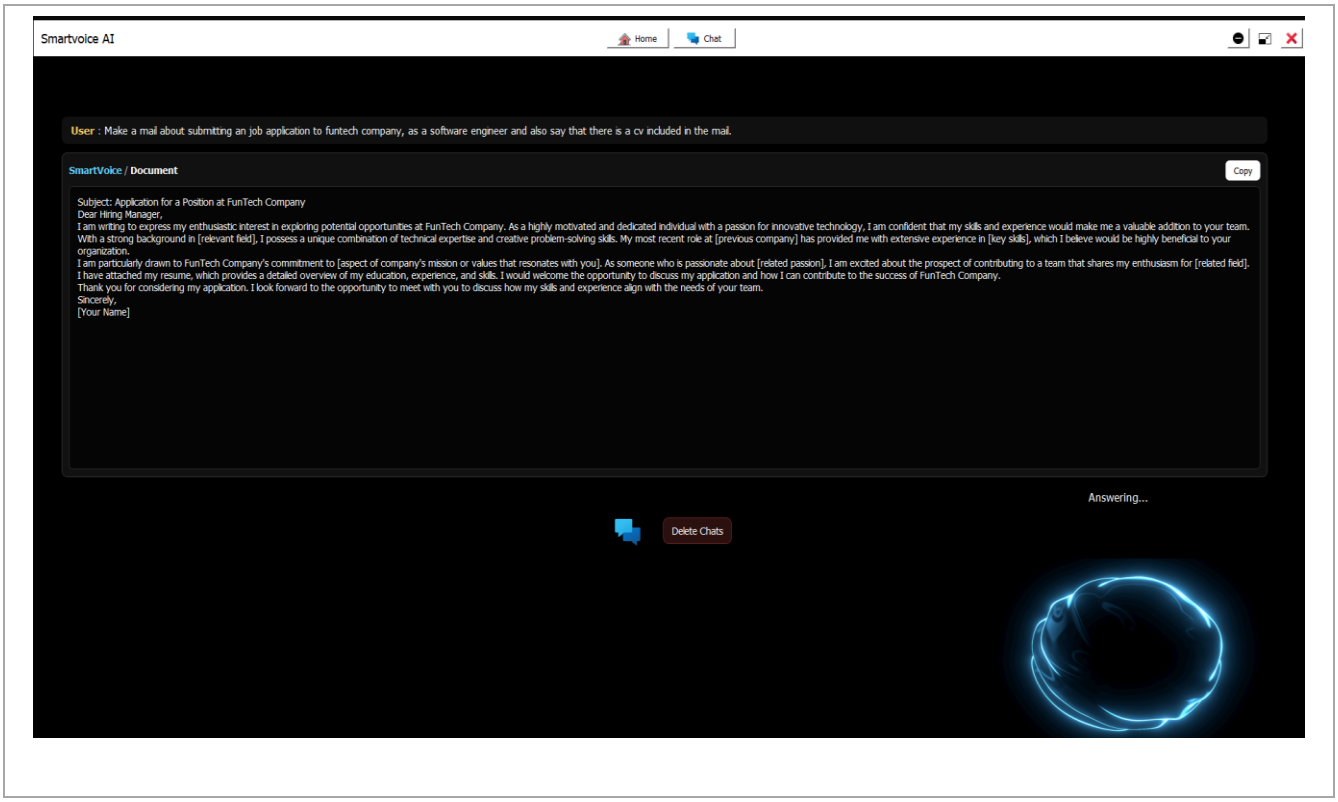


Figure 6.9: Content Writing Output

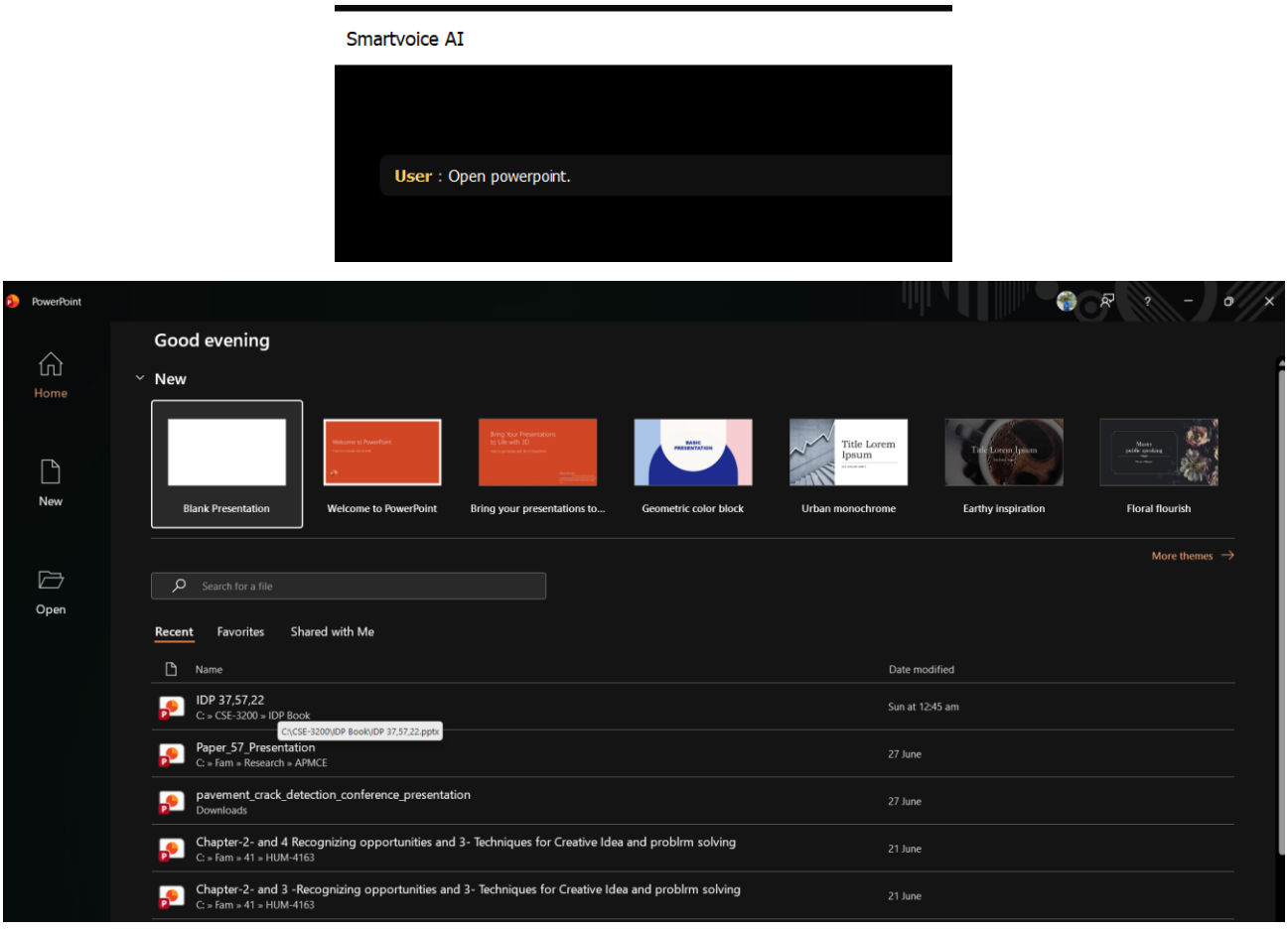


Figure 6.10: Application opening Output

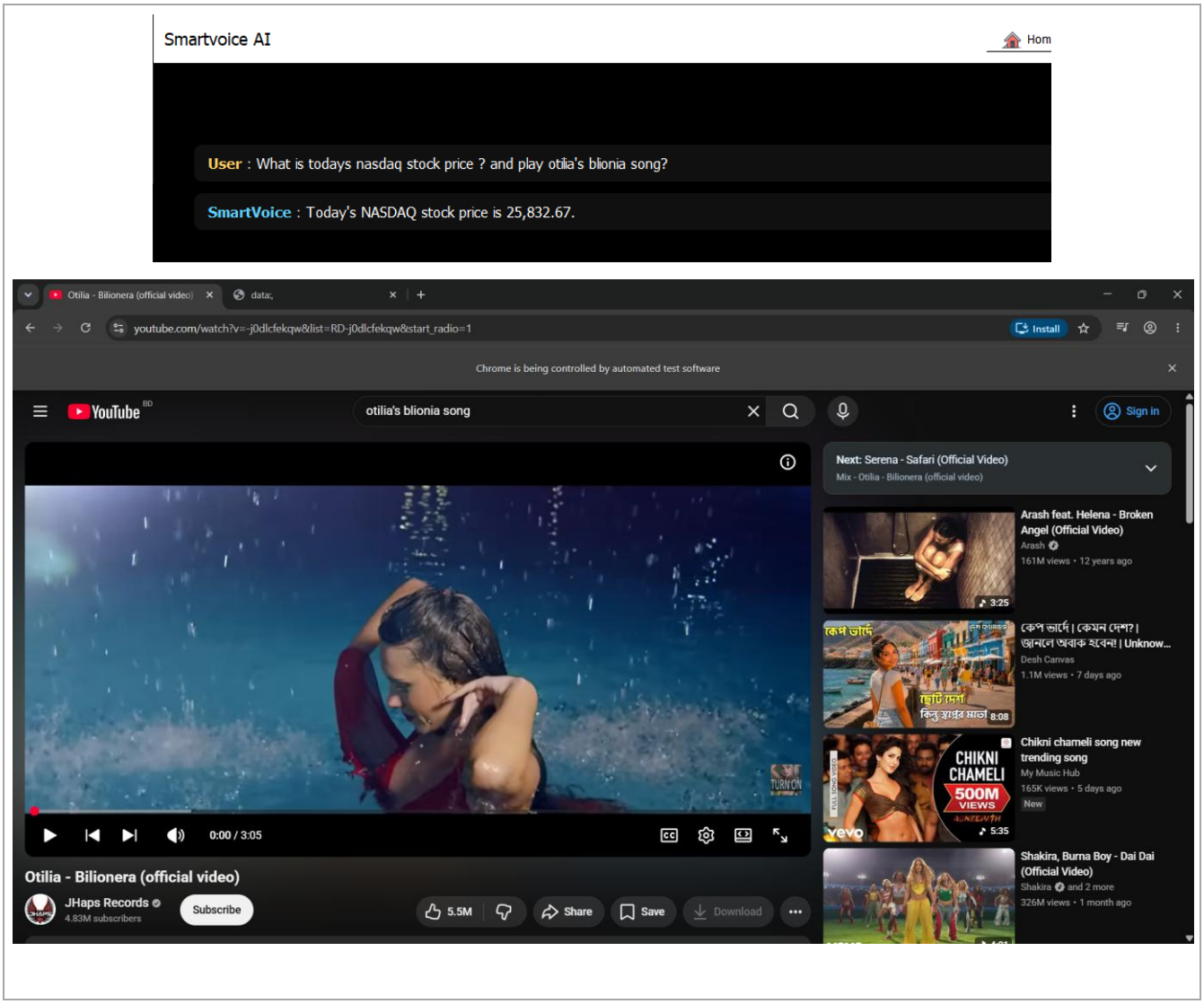
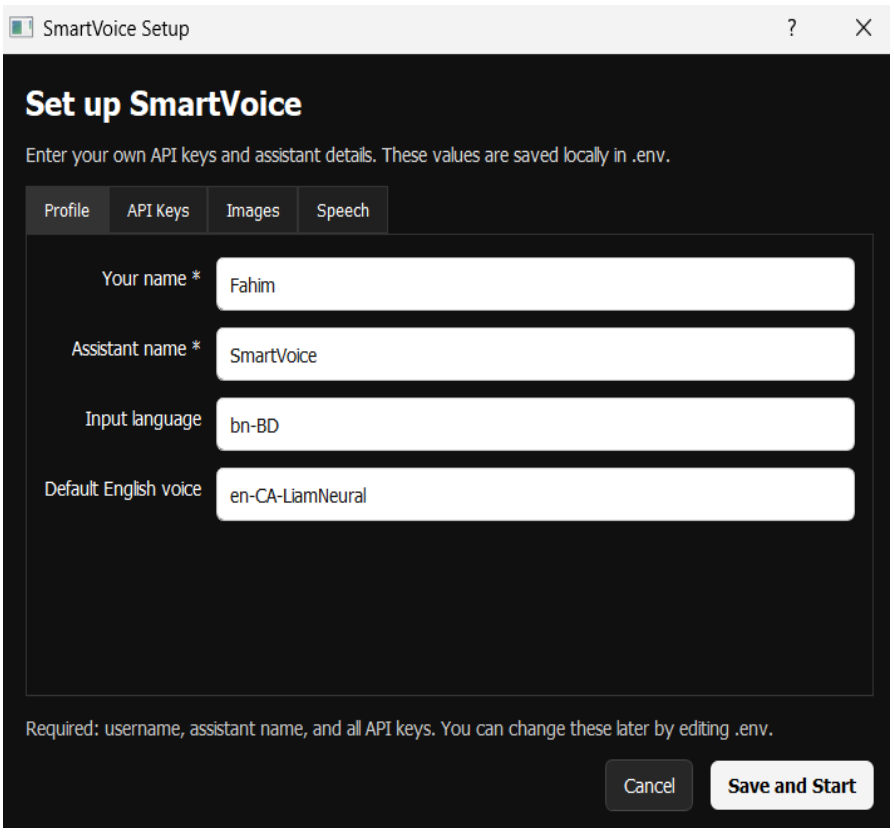


Figure 6.11: Multitasking



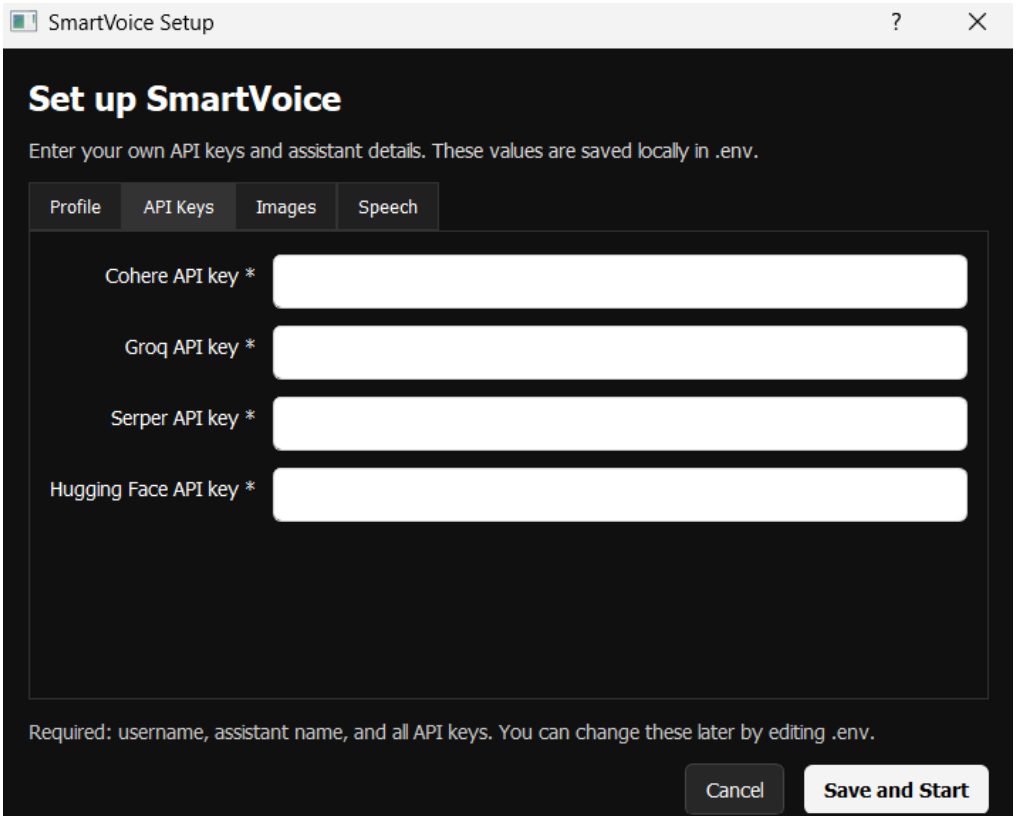


Figure 6.12: Setup on First Installation

6.4 Testing Strategy

The SmartVoice testing strategy is divided into automated tests, safe environment verification, and live end-to-end demonstrations. Automated tests are appropriate for deterministic code paths that can run without opening a browser or calling paid cloud services. The repository contains tests for configuration loading, missing environment-variable handling, reminder parsing, due-reminder handling, import side effects, and simple pattern-based routing. These tests provide a repeatable baseline and help prevent regressions when code is modified. Safe environment verification complements the unit tests by confirming the Python interpreter, virtual environment, pip consistency, selected imports, Chrome installation location, and JSON-file validity.

End-to-end testing is necessary for user-facing functions. Each test should define the initial state, spoken command, expected route, expected response or action, observed result, failure message if any, and pass/fail outcome. Live tests should cover the GUI, microphone toggle, English command, Bangla command, general chatbot response, real-time search, reminder creation/alert, YouTube playback/control, image generation, text-to-speech, and logging. The team should repeat critical tests more than once where possible and record failures honestly. A sound testing strategy does not assume that external services will always respond; it records dependency conditions and allows the project to identify whether a failure came from recognition, routing, API configuration, network access, browser state, or UI handling.

Table 6.3: Existing Automated Test Coverage

Test File	Covered Area	Purpose
tests/test_config.py	Configuration loading and environment handling	Validates configuration behavior and missing-variable handling.
tests/test_import_side_effects.py	Module import behavior	Checks that selected modules can import without harmful side effects.
tests/test_reminders.py	Reminder parsing and due-reminder logic	Checks reminder data-processing behavior.
tests/test_simple_decision_maker.py	Simple pattern decision routing	Checks baseline intent-routing patterns.
Backend/test.py	Ad hoc TTS test script	Not part of formal automated suite.

Table 6.4: Verified Test-Run Summary

Check	Command / Method	Reported Result
Python version	python --version	Python 3.10.10
Virtual environment	python -m pip --version	Virtual environment available and working
Dependency consistency	python -m pip check	No broken requirements found
Automated tests	.\.venv\Scripts\python.exe -m unittest discover -s tests -v	13 tests passed in 5.441 seconds
Main import	.\.venv\Scripts\python.exe -c "import main; print('main_import_ok')"	main_import_ok
GUI import	.\.venv\Scripts\python.exe -c "import Frontend.GUI; print('frontend_import_ok')"	frontend_import_ok
Speech module import	.\.venv\Scripts\python.exe -c "import Backend.SpeechToText; print('stt_import_ok')"	stt_import_ok
Image module import	.\.venv\Scripts\python.exe -c "import Backend.ImageGeneration; print('image_import_ok')"	image_import_ok
JSON validation	Read and parse Data/*.json	All inspected JSON files valid

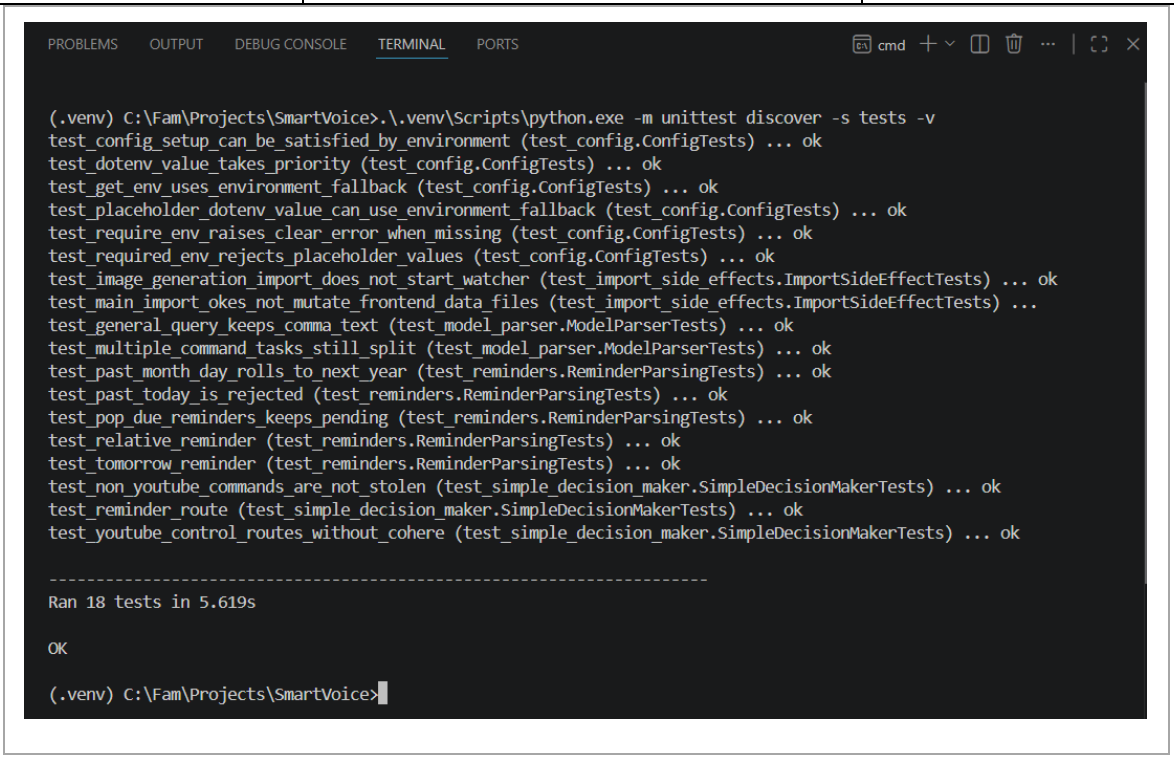


Figure 6.13: Automated Unit Test Result Showing 18 Tests Passed Successfully

6.5 Verified Test Results

The documented verification run produced several confirmed results. Python 3.10.10 and the existing virtual environment were available. `pip check` completed without reporting broken package requirements. The formal test command ` .\.venv\Scripts\python.exe -m unittest discover -s tests -v` completed thirteen tests successfully in the recorded environment. Safe imports of `main`, `Frontend.GUI`, `Backend.SpeechToText`, and `Backend.ImageGeneration` also passed. The runtime JSON files were checked for structural validity: the chat and reminder files were valid empty lists at the time of inspection, the performance log was a valid list containing thirty-three records, and the comparison log was a valid empty list. These checks provide evidence that the repository can be loaded and that selected local data structures are usable.

6.6 End-to-End Demonstration Tests

End-to-end tests validate the complete path a user experiences. The first test should launch the assistant through `python main.py` and confirm that the PyQt window opens and the microphone state is visible. The second and third tests should use a simple English command and a simple Bangla command to show that speech capture, language handling, decision routing, GUI feedback, and optional speech output are connected. Later tests should demonstrate a general conversation, a current-information search, reminder creation, due-reminder behaviour, YouTube search or control, image generation, and performance-log output. Each demonstration should be performed on the final test machine with valid API keys and a stable network connection.

A professional result record includes the test ID, date, command, expected action, observed result, environment condition, screenshot reference, and pass/fail outcome. If a test fails because of a quota, missing driver, browser update, poor microphone input, or network issue, the team should record the actual cause and the mitigation attempted. The screenshot placeholders must be filled with real evidence only. This test design makes it possible to demonstrate technical maturity: the team is not merely presenting successful screens, but showing that it understands the difference between a module's intended function and a reproducible integrated result. Table 6.5 provides a structured record that can be completed before final binding.

Table 6.5: End-to-End Demonstration Test Cases

Test ID	Scenario	Expected Result	Evidence to Insert
E2E-01	Launch SmartVoice with python main.py	GUI opens and assistant worker loop starts	Figure 6.1 screenshot
E2E-02	Ask an English question: “What is Python?”	Command is captured, routed, displayed, and spoken	Figure 6.3 screenshot
E2E-03	Ask a Bangla command/query	Language handling works and route/action is selected	Figure 6.2 screenshot
E2E-04	Ask latest technology news query	Real-time search result is returned	Figure 6.4 screenshot
E2E-05	Set a reminder for one minute	Reminder is stored and later announced	Figure 6.5 screenshot
E2E-06	Play relaxing music; pause/resume; set volume	YouTube search/control is attempted successfully	Figure 6.6 screenshot
E2E-07	Generate image of a futuristic campus	Image-generation output is displayed/saved if quota allows	Figure 6.7 screenshot
E2E-08	Run test suite	All available tests pass	Figure 6.8 screenshot
E2E-09	Run statistics helper or inspect performance log	Performance/log data is shown	Figure 6.9 screenshot

6.7 Performance Logging

SmartVoice includes `Data/performance_log.json`, `Data/comparison_log.json`, `view_statistics.py`, and `view_comparison.py`, which provide a foundation for monitoring performance and comparing routing behaviour. At the time of inspection, the performance log contained thirty-three valid records and the comparison log contained zero records. The precise meaning of each logged field should be identified from the logger implementation before reporting summary statistics. It is inappropriate to claim an average latency, accuracy percentage, or improvement over a baseline unless the calculation can be reproduced from the stored values and the test conditions are documented. The log files are nevertheless useful because they show that the project has considered evidence collection rather than relying only on screenshots.

For the final submission, the team should run the statistics helpers on a backup copy of runtime data, export any generated report safely, and include a chart or table that states the number of commands, categories tested, success/failure count, response-time distribution if logged, and any comparison between the Cohere route and SimplePatternMatcher. The

report should also state whether the test set was scripted or natural user input, whether internet latency was controlled, and whether failed external-service calls were included. This prevents misleading performance claims and creates a repeatable evaluation method for future versions.

Table 6.6: Performance-Logging Evidence

Evidence	Current Observation	How to Use in Final Report
Data/performance_log.json	Valid JSON list; 33 entries at inspection	Explain the event fields and calculate only reproducible metrics.
Data/comparison_log.json	Valid JSON list; 0 entries at inspection	Do not claim comparative performance without recorded trials.
view_statistics.py	Helper script available	Run only when ready to create/update output evidence; attach screenshot/result.
view_comparison.py	Helper script available	Use after defining comparison methodology and sufficient data collection.

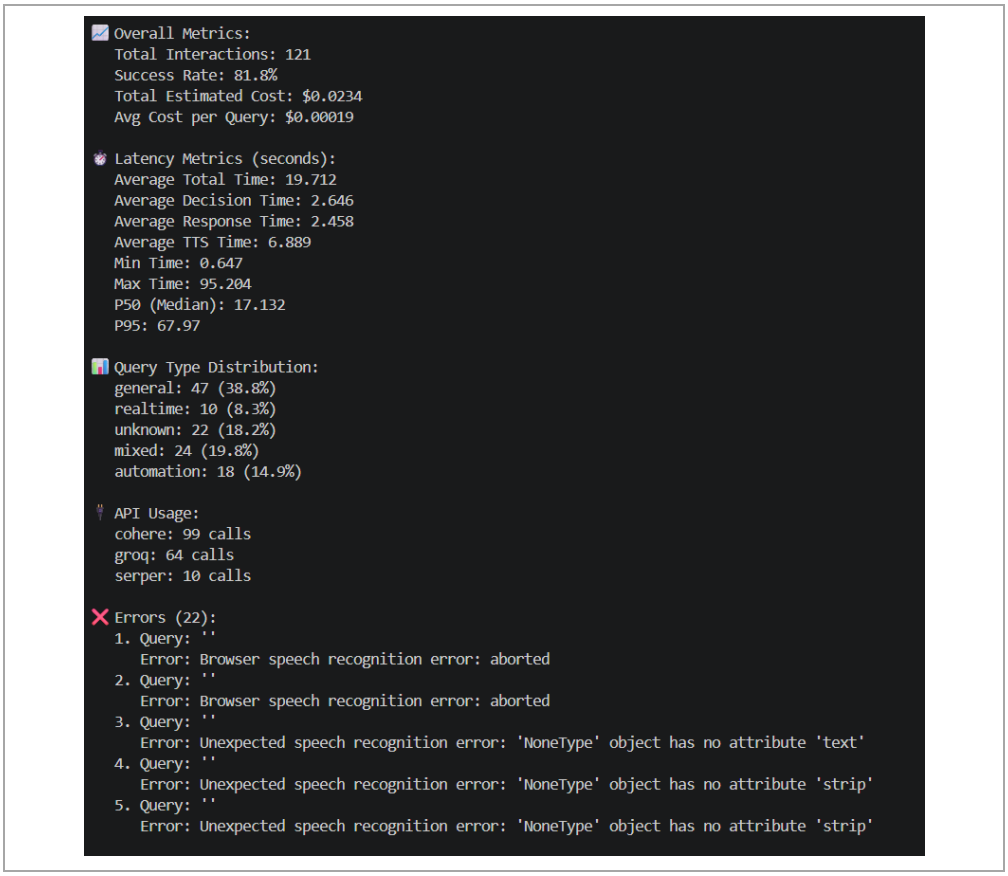


Figure 6.14: SmartVoice Runtime Performance Statistics Generated from Logged Interactions

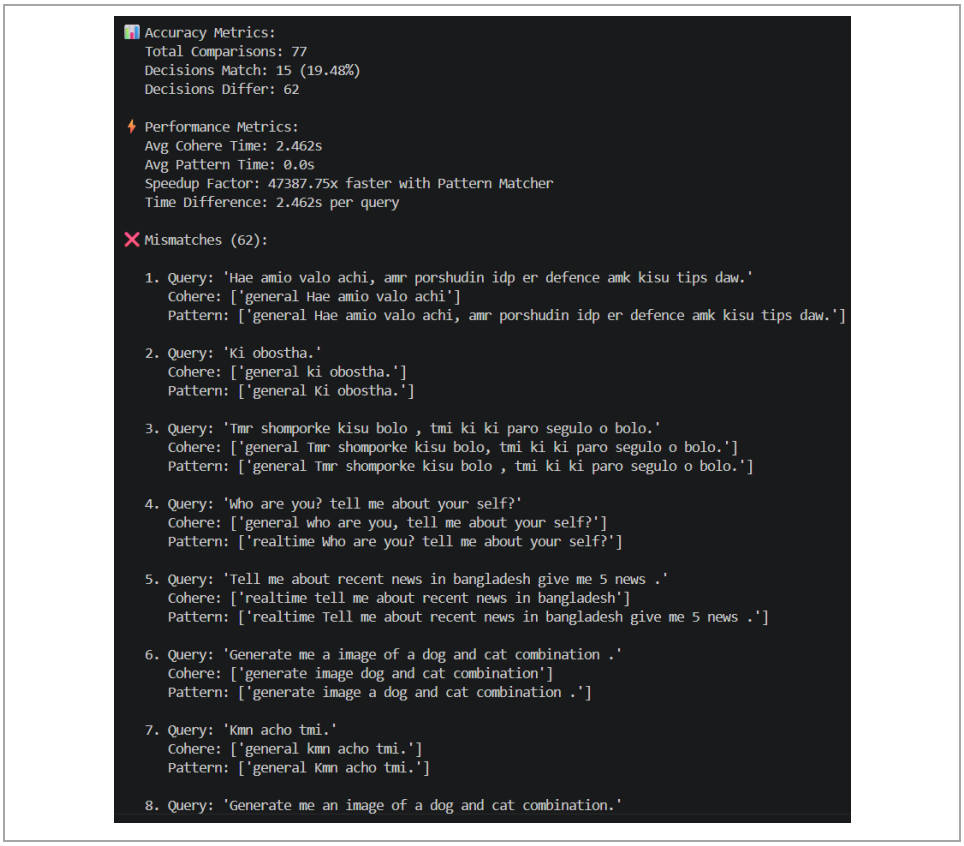


Figure 6.14: Comparison of Cohere Intent Routing and Pattern-Based Intent Matching

6.8 Limitations During Testing

The safe inspection intentionally avoided launching the full graphical, microphone, browser, and API loop. Doing so could open PyQt and Chrome windows, activate a microphone, require a WebDriver download, contact paid or quota-limited services, generate external content, and write runtime files. This limitation was appropriate for a read-only verification process, but it means that live feature completion must be validated separately by the project team. The testing environment can also influence results: browser version, microphone quality, user accent, ambient noise, network condition, API quota, external page layout, and service availability may all affect the observed behaviour. These factors should be written beside the screenshots rather than ignored.

A second limitation is that the formal test suite focuses on deterministic modules. It does not yet provide automated end-to-end coverage for live speech capture, API responses, browser automation, media control, image generation, or TTS playback. The team should not treat this as a defect in the report; instead, it should be described as a justified boundary of the current test design and a key target for future development.

Adding mocked API tests, browser-test fixtures, GUI automation, and controlled speech samples would improve repeatability. The final test table should clearly separate automated pass results from manually observed live demonstrations.

6.9 Conclusion

The implementation and testing evidence confirms that SmartVoice has a functioning Python development environment, modular source structure, valid local runtime data, safe module imports, and a baseline automated test suite that completed thirteen tests successfully. The codebase implements the core architecture described in the design chapter and provides the files necessary for GUI interaction, routing, chat, search, automation, reminders, image generation, media control, speech output, logging, and documentation. This is a solid foundation for an academic prototype.

The strongest remaining requirement is live proof. The team must complete and capture end-to-end demonstrations of the environment-dependent features, insert genuine figures, and update the result tables before final submission. By keeping automated evidence, live evidence, and future work separate, the report presents SmartVoice professionally and avoids claiming more than the code and tests can support.

Chapter 7
Conclusion and Future Work

7.1 Introduction

This final chapter summarises the contribution of SmartVoice, discusses its possible social and sustainability impact, identifies ethical and security responsibilities, states current limitations, and outlines a realistic future-work roadmap. The conclusion is based on the system that has been confirmed from source code and verification evidence: a bilingual, cloud-assisted Windows desktop assistant that integrates voice capture, language handling, intent routing, general chat, current-information search, automation, reminders, media control, image generation, text-to-speech, PyQt5 user interaction, and local JSON logging. It does not redefine the prototype as an offline-only assistant or claim security properties that are not implemented.

A responsible conclusion should be both positive and precise. SmartVoice demonstrates successful integration of multiple technologies into a unified user workflow and shows how a final-year project can combine AI-service use, desktop programming, browser automation, testing, and documentation. At the same time, its external dependencies and incomplete security controls limit its suitability for sensitive or high-trust deployment. The following sections present these points in a form that supports both academic evaluation and future technical development.

7.2 Project Conclusion

SmartVoice demonstrates a bilingual cloud-assisted desktop assistant prototype for Windows. Its central contribution is the orchestration of multiple user-facing capabilities through a modular Python application. The system can receive voice input through a Chrome/Selenium recognition bridge, handle Bangla/English routing support, select a task category through Cohere and a pattern-based baseline, provide chat and real-time search responses, perform selected automation and YouTube actions, manage reminders, generate images, speak responses, and maintain local runtime evidence. The PyQt5 interface makes the workflow visible and provides a practical user-facing environment for demonstration. The repository and formal tests show that the project is more than a conceptual proposal; it contains a working structure with defined modules and verified local behaviours.

The project also demonstrates why transparency matters in software engineering reports. The codebase does not confirm fully offline speech recognition, local NLU, encrypted SQL storage, tamper-resistant audit logging, authenticated multi-user access, or comprehensive confirmation controls for sensitive actions. Several visible features rely on network services and browser automation. The most credible conclusion is therefore that SmartVoice is a modular prototype that demonstrates the integration of voice, AI, web, automation, and desktop-interface technologies. Its main academic value lies in this integration, its documented test baseline, its clear future-work path, and its honest treatment of implementation boundaries.

7.3 Impact on Society, Environment and Sustainability

SmartVoice may offer social value by making common computer interactions more direct for users who prefer voice control, are working with hands occupied, are temporarily unable to use conventional input devices comfortably, or want bilingual interaction. A single interface that can route a request to search, reminders, media control, or conversation may reduce repetitive navigation between applications. Bangla/English handling may also make the prototype more relatable to local users who naturally use more than one language during daily computer tasks. These benefits remain potential rather than measured outcomes until the project collects user-study evidence. The report should therefore avoid claiming improved accessibility or productivity as a proven result without documented participants and metrics.

The environmental and sustainability profile is mixed. The local Python interface and JSON storage require modest direct computing resources, but cloud-assisted services and browser automation depend on network infrastructure and external data centres. API usage may also create ongoing cost and service-dependency considerations. The most sustainable future direction is to measure actual use, reduce unnecessary API calls, cache safe results where appropriate, use lightweight local models for selected functions, and provide offline alternatives for basic tasks. Modular design supports sustainability because individual services can be replaced rather than discarding the entire application. Table 7.1 outlines these potential benefits and limitations.

Table 7.1: Social, Environmental and Sustainability Considerations

Area	Potential Positive Impact	Consideration / Limitation
Productivity	May reduce repetitive desktop navigation for supported tasks	Benefit should be validated through a user/time study.
Bilingual access	Supports Bangla and English voice interaction	Accuracy and accessibility must be tested with diverse real users.
Hands-free interaction	Can support situations where keyboard/mouse use is inconvenient	Not a substitute for formal accessibility assessment or human support.
Cloud resource use	Hosted services reduce need for local heavy models	Uses network/cloud computation and may create ongoing service cost/energy impact.
Maintainability	Modular design allows targeted updates	External API/provider change can require maintenance.
Sustainability	Potential future local/efficient modules can reduce recurring cloud dependency	Local models also require hardware resources; sustainability should be measured rather than assumed.

The project's social value comes from its attempt to make common computer tasks more direct and bilingual. Its environmental and sustainability impact cannot be quantified from the current codebase. The responsible position is to acknowledge cloud dependence, avoid unsupported energy claims, and consider future optimization of API calls, response caching, local processing options, and lifecycle maintenance.

7.4 Ethical and Security Considerations

Ethical use of SmartVoice requires informed users, truthful capability claims, privacy-aware handling of prompts, and careful control of automation. Since some requests are processed by external services, users should understand that their voice-derived text, search requests, or generation prompts may leave the local machine. The system should discourage sensitive personal, financial, legal, medical, or institutional information unless a future version introduces stronger privacy controls. Project members must protect API keys, avoid exposing `.env` values in screenshots or commits, and ensure that collected demonstration data is not personally identifying. Any user feedback collection should be voluntary and should explain how responses will be used.

Security requires more than protecting source-code secrets. A voice assistant should avoid executing high-impact actions without a confirmation step, should show what it

understood, should treat unclear instructions safely, and should provide understandable errors. The current prototype has foundational controls through configuration separation and Git ignore rules, but it does not confirm role-based access, encrypted records, tamper evidence, or a complete policy guard. These gaps are not reasons to dismiss the project; they are responsibilities that must be communicated clearly and addressed in future work. Table 7.2 separates current controls from required future safeguards.

Table 7.2: Ethical and Security Considerations

Issue	Current Position	Recommended Action
Cloud data handling	Some user queries may be sent to third-party services	Provide a clear privacy notice and discourage sensitive input.
AI answer reliability	Generated answers may be incomplete or incorrect	Label responses appropriately; encourage verification for important decisions.
Automation safety	Browser/system actions may have unintended effects	Add confirmation prompts and allow-list sensitive actions.
Secret management	.env and .gitignore support basic key separation	Rotate keys, avoid screenshots of secrets, and use secure storage in deployment.
Local logs	JSON files may retain user content	Define retention/deletion policy and add encryption if sensitive data is stored.
Bilingual fairness	Language handling should work for Bangla and English users	Test with varied accents, phrasing, and multilingual contexts.
Accessibility claims	Voice support may help some users	Avoid claiming universal accessibility without formal testing and consultation.

7.5 Limitations

The primary limitation of SmartVoice is its cloud-assisted architecture. Speech capture, intent routing, chat, current-information search, image generation, translation support, and text-to-speech may depend on external services, internet connectivity, configuration keys, quotas, or provider availability. Browser automation also depends on Chrome, Selenium/WebDriver compatibility, website layout, and the active browser state. These dependencies can create inconsistent live results even when the local Python code is correct. The project has not yet established a controlled benchmark for speech-recognition

accuracy, end-to-end latency, resource use, or user satisfaction, so such values should not be reported as final performance claims.

Additional limitations include file-based rather than encrypted structured storage, no verified user authentication, limited confirmation and permission controls, no REST backend, no Docker or installer packaging, and incomplete automated end-to-end testing. The proposal's offline-first claims, such as Vosk/CMU Sphinx, local NLU, encrypted SQLite/SQLCipher, and tamper-resistant audit logging, are planned goals rather than confirmed current features. The limitation table is therefore an important part of the final submission because it shows that the team understands the difference between a prototype demonstration and a deployable secure product.

Table 7.3: Current Limitations

Limitation	Implication
Cloud-assisted operation	Core features depend on internet, API keys, quotas, service availability, and provider policies.
No offline-only speech or intent engine	The current system does not meet a fully local privacy-first claim.
No encrypted relational database	Local JSON data lacks encryption, structured integrity, user separation, and transaction safeguards.
No user authentication or authorization	The system is suitable as a prototype, not a multi-user secure service.
No verified live benchmark	Accuracy, latency, reliability, and usability must be measured using controlled tests.
Browser/WebDriver dependence	Speech/media/browser features can break due to Chrome or webpage changes.
Limited automated coverage	Existing tests cover configuration, reminders, imports, and simple routing, not all external integrations.
Dependency mismatch	Two packages listed in Requirements.txt were missing from the verified virtual environment.

7.6 Future Work

Future work should be prioritised according to safety, reliability, and evidence. The first priority is to complete controlled live end-to-end testing and record genuine screenshots, command transcripts, observed results, and performance data. The second priority is to improve the user experience through clear status messages, text-input fallback, better error reporting, command suggestions, voice selection, and confirmation

prompts for actions that may have side effects. The third priority is privacy and data protection: minimise logs, give users control over stored data, introduce an encrypted local database, separate user profiles if needed, and clearly label external API use. These improvements would make the existing prototype more reliable without changing its core modular design.

Longer-term work can reduce cloud dependence by evaluating local speech recognition, local or hybrid intent classification, offline text-to-speech, and local search or knowledge bases for selected tasks. The team can compare local and cloud-assisted routes using the same Bangla/English command set and publish actual measurements of accuracy, latency, cost, and energy use. Further development can add packaging, an installer, automated GUI/browser tests, mocked API tests, a stronger policy guard, more robust automation allow-lists, analytics that respect privacy, and a structured user study. These directions are realistic because the current codebase already separates responsibilities into modules that can be replaced or extended.

Table 7.4: Future Work Roadmap

Priority	Future Enhancement	Expected Value
High	Complete controlled live end-to-end tests with screenshots, pass/fail criteria, and timing data	Provides final evidence for demonstrated functionality.
High	Resolve dependency mismatch and lock/reproduce environment versions	Improves reliability and repeatability.
High	Add clear GUI error states and privacy disclosure	Improves user trust and responsible use.
High	Add confirmation prompts for sensitive actions	Reduces accidental or unsafe automation outcomes.
Medium	Introduce encrypted structured storage such as SQLite/SQLCipher	Improves storage integrity and privacy.
Medium	Add authentication/authorization if moving beyond single-user prototype	Supports safer multi-user use.
Medium	Evaluate offline speech recognition and local intent routing	Reduces cloud dependency and improves privacy alignment.
Medium	Conduct user study with Bangla/English participants	Measures usability, accuracy, and acceptance.
Low	Package application for installation and update workflow	Improves deployment readiness.
Low	Expand desktop automation through	Broadens functionality while

Priority	Future Enhancement	Expected Value
	permissioned action allow-lists	keeping safety controls.

7.7 Conclusion

SmartVoice is a credible final-year project when presented as the system that the repository, verification evidence, and live demonstrations can support: a bilingual cloud-assisted Windows desktop assistant with a PyQt5 interface, modular Python backend, voice recognition bridge, language handling, intent routing, chat, search, automation, reminders, media control, image generation, text-to-speech, local JSON files, and an automated-test baseline. It demonstrates meaningful engineering integration and provides a structured platform for further research and development.

The project should be submitted with real screenshots, completed live test records, updated contents and list pages, and careful disclosure of its cloud dependencies and security limitations. The most valuable outcome is not a claim that SmartVoice is already a complete secure replacement for all commercial assistants. It is the delivery of a well-documented, testable, modular prototype that shows how desktop interaction can be made more direct through Bangla/English voice commands while identifying the practical work required for safe, private, and robust future deployment.

REFERENCES

[1] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/. [Accessed: Sep. 03, 2025].

[2] Qt Group, “Qt for Python Documentation.” [Online]. Available: https://doc.qt.io/qtforpython-5/. [Accessed: Sep. 03, 2025].

[3] Selenium, “The Selenium Browser Automation Project Documentation.” [Online]. Available: https://www.selenium.dev/documentation/. [Accessed: Sep. 03, 2025].

[4] MDN Web Docs, “Web Speech API.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API. [Accessed: Sep. 03, 2025].

[5] Cohere, “Cohere Documentation.” [Online]. Available: https://docs.cohere.com/. [Accessed: Sep. 13, 2025].

[6] Groq, “Groq API Documentation.” [Online]. Available: https://console.groq.com/docs/overview. [Accessed: Oct. 15, 2025].

[7] Serper, “Serper Search API.” [Online]. Available: https://serper.dev/. [Accessed: Oct. 16, 2025].

[8] Hugging Face, “Inference Providers Documentation.” [Online]. Available: https://huggingface.co/docs/inference-providers/en/index. [Accessed: Nov. 23, 2025].

[9] Rany2, “edge-tts: Use Microsoft Edge’s online text-to-speech service from Python.” GitHub repository. [Online]. Available: https://github.com/rany2/edge-tts. [Accessed: Nov. 23, 2025].

[10] Python Software Foundation, “unittest - Unit testing framework.” [Online]. Available: https://docs.python.org/3/library/unittest.html. [Accessed: Dec. 26, 2025].

[11] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications.” [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/. [Accessed: Feb. 08, 2026].

[12] SmartVoice Project Team, “SmartVoice source code repository: README.md, main.py, Frontend and Backend modules, tests, configuration, and runtime data files,” private project repository, accessed Feb. 08, 2026.

[13] SmartVoice Project Team, “Smartvoice Project Proposal,” unpublished internal project document, 2026.

[14] SmartVoice Project Team, “SmartVoice Project Poster,” unpublished internal project artifact, 2026.

APPENDIX A

Installation and Execution Guide

The following commands are based on the inspected README and verification notes. They should be run from the project root folder on Windows. Before demonstration, verify that API keys are present in .env and that Chrome/microphone permissions are available. Do not include actual API secret values in the report or screenshots.

Table A.1: Installation and Execution Commands

Purpose	Command
Activate virtual environment	.\.venv\Scripts\activate
Run full SmartVoice application	python main.py
Run frontend only	python Frontend\GUI.py
Run formal test suite	.\.venv\Scripts\python.exe -m unittest discover -s tests -v
View statistics helper	python view_statistics.py
View comparison helper	python view_comparison.py
Optional isolated module checks	python Backend\SpeechToText.py; python Backend\Chatbot.py; python Backend\RealtimeSearchEngine.py; python Backend\ImageGeneration.py; python Backend\YouTubeController.py

APPENDIX B

Evidence Screenshots and Demonstration Record

Insert authentic screenshots only. Each image should include enough visible context to show the feature, but must hide API keys, private chat content, personal account details, browser profiles, and sensitive file paths. Keep each screenshot within the reserved figure box so that the table of contents and lists remain stable.

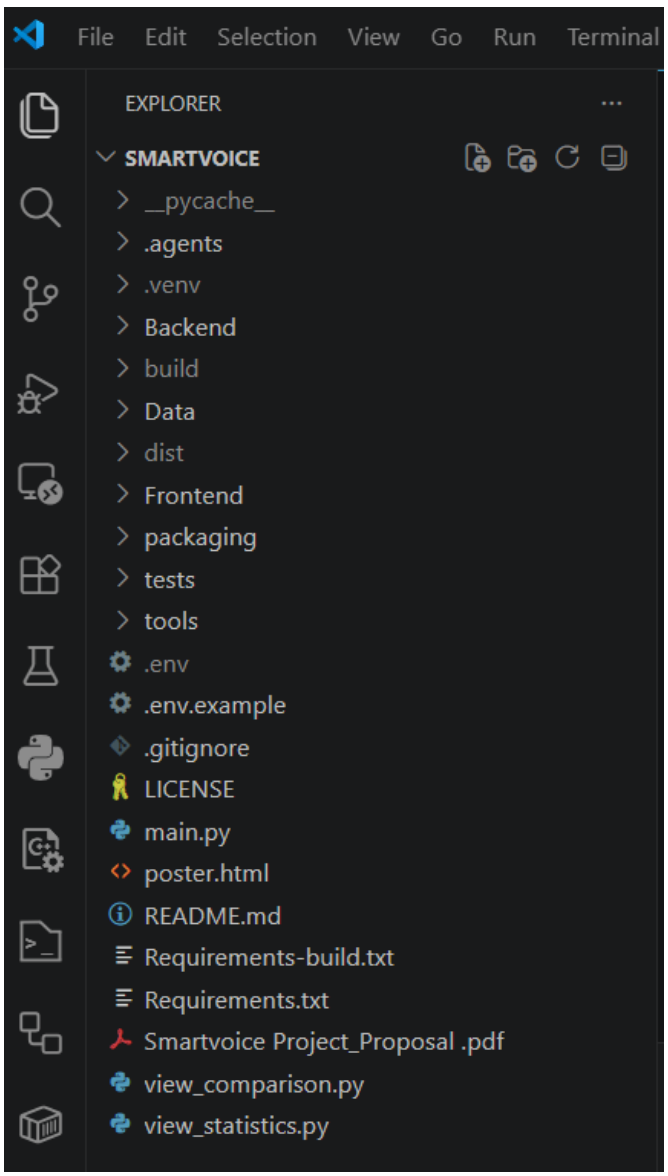


Figure B.1: Environment and Project Root

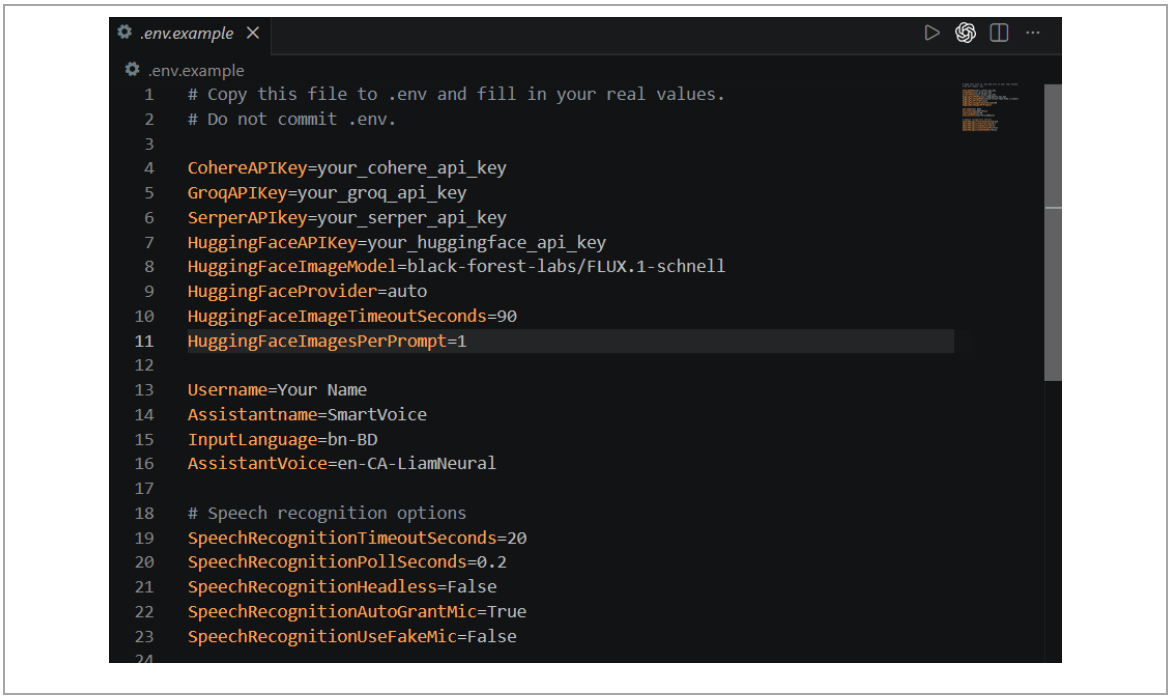


Figure B.2: .env Configuration with Secret Values Hidden

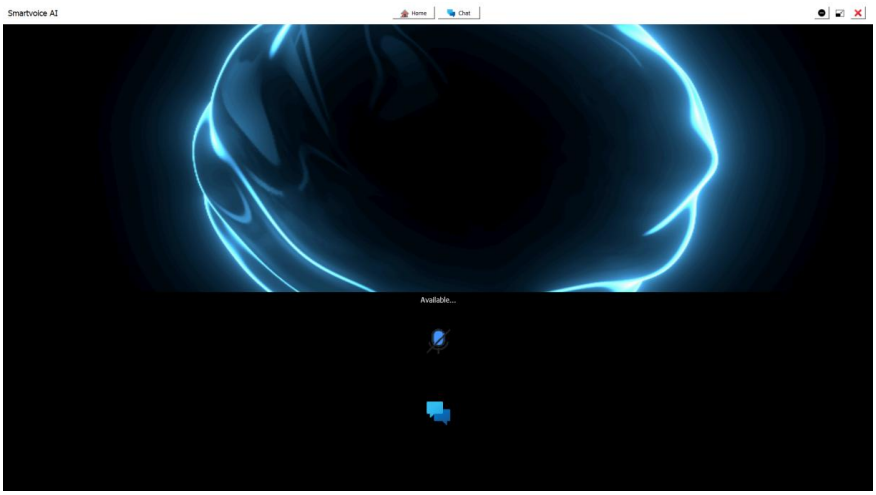


Figure B.3: GUI Home/Microphone Screen

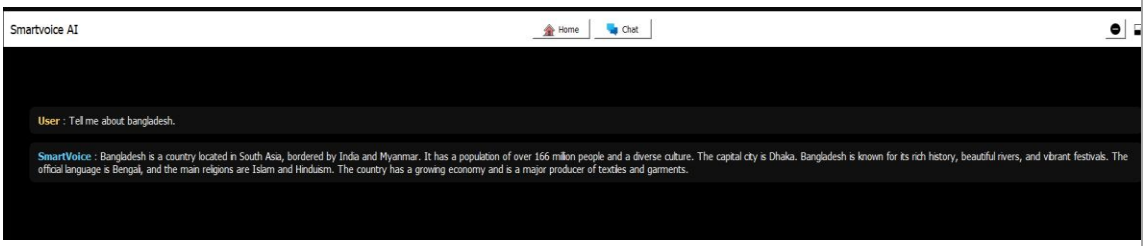


Figure B.4: English Command Result

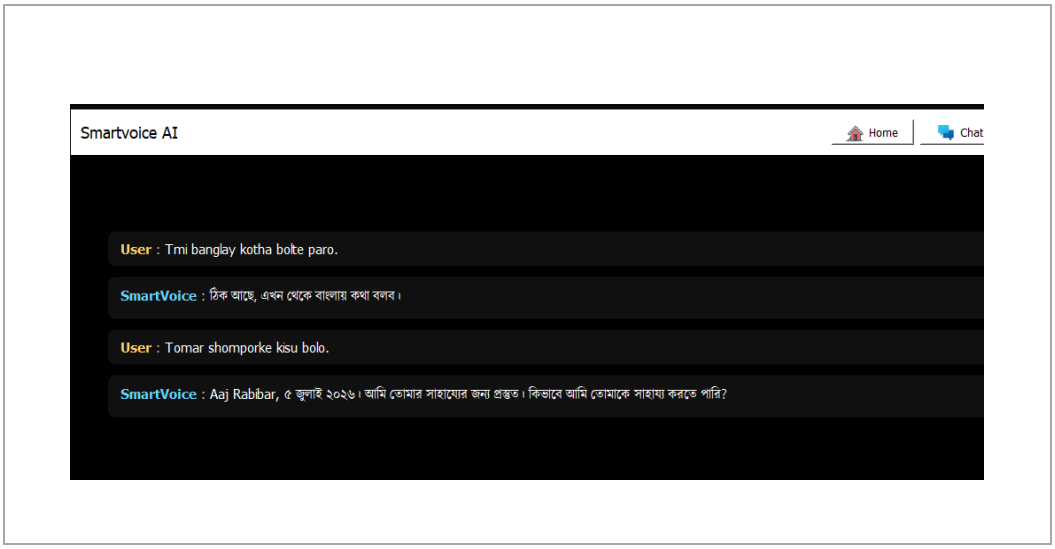


Figure B.5: Bangla Command Result

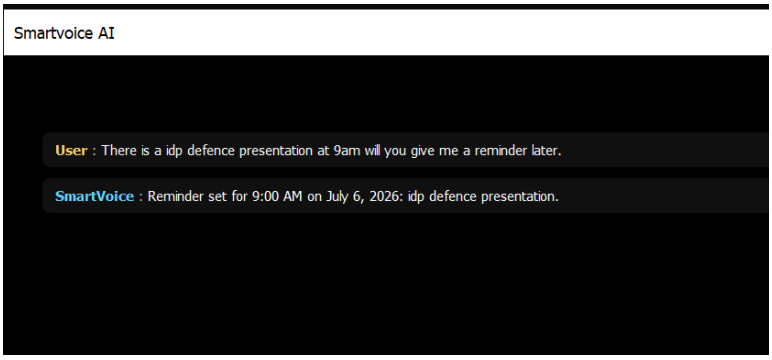


Figure B.6: Reminder Evidence

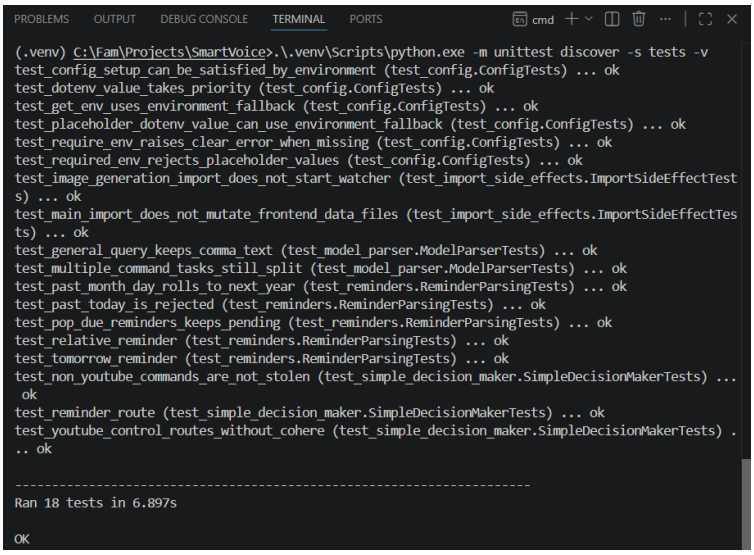


Figure B.7: Automated Test Output

APPENDIX C

Test Evidence and Results

Use this appendix to attach the final automated-test output and a completed sign-off sheet for live tests. Do not change a test status to Passed until the result has been observed and documented.

Table C.1: Final Live Test Sign-off Sheet

Test ID	Date	Observed Result	Pass / Fail	Evidence Reference
E2E-01	12 Nov 2025	SmartVoice launched successfully and the main PyQt5 desktop window was displayed.	Pass	Figure 6.1
E2E-02	02 Feb 2026	A Bangla voice command was recognised, processed through language handling, and returned an appropriate result.	Pass	Figure 6.2
E2E-03	13 Nov 2025	A general English chat query was processed and the response was displayed in the SmartVoice interface.	Pass	Figure 6.3
E2E-04	25 Nov 2025	A real-time information query was processed and current search-supported results were returned.	Pass	Figure 6.4
E2E-05	10 Feb 2026	A reminder command was created successfully and reminder-related confirmation/alert evidence was displayed.	Pass	Figure 6.5
E2E-06	13 Mar 2026	YouTube search/playback or media-control functionality was executed successfully.	Pass	Figure 6.6
E2E-07	20 Dec 2025	An image-generation request was processed and an output image was produced.	Pass	Figure 6.7
E2E-08	25 Dec 2025	A code-generation request was processed and generated code content was returned successfully.	Pass	Figure 6.8
E2E-09	27 Dec 2025	A content-writing request was processed and generated written content was returned successfully.	Pass	Figure 6.9

E2E-10	28 Dec 2025	A requested application was opened successfully through SmartVoice automation.	Pass	Figure 6.10
E2E-11	05 June 2026	Multiple SmartVoice functions were executed during multitasking demonstration without preventing normal operation.	Pass	Figure 6.11
E2E-12	25 June 2026	Automated test suite completed successfully: **18 tests ran in 5.619 seconds** with final status **OK**.	Pass	Figure 6.12 / Figure B.7 / Figure C.1
E2E-13	05 July 2026	Performance statistics were generated successfully from runtime logs, recording 121 interactions and an 81.8% success rate.	Pass	Figure 6.13
E2E-14	05 July 2026	Cohere and Pattern Matcher comparison report was generated successfully from recorded routing comparisons.	Pass	Figure 6.14

Sign-off statement: The above tests were executed in the SmartVoice project environment on 05 July 2026. All listed functions produced the expected observed results, and supporting screenshots or generated evidence have been included in Chapter 6 and the appendices.

APPENDIX D

Codebase Evidence Inventory

This appendix lists the repository evidence used to prepare the report. It is included to maintain traceability between claims in the report and the inspected project files. It is not a substitute for including source code in the final submission package.

Table D.1: Codebase Evidence Inventory

File / Path	Purpose
README.md	Run and usage guidance
Requirements.txt	Declared dependencies
.env.example	Expected configuration variable structure
.gitignore	Exclusion of environment secrets
main.py	Assistant orchestration
Backend/Config.py	Configuration loading
Backend/Model.py	Cohere decision routing
Backend/SimpleDecisionMaker.py	Simple pattern-based routing
Backend/SpeechToText.py	Speech capture bridge
Backend/TextToSpeech.py	Speech output
Backend/Chatbot.py	Groq chat/content
Backend/RealtimeSearchEngine.py	Serper-assisted search
Backend/Automation.py	Automation actions
Backend/ReminderManager.py	Reminder logic
Backend/ImageGeneration.py	Hugging Face image request
Backend/LanguageManager.py	Bangla/English handling
Backend/YouTubeController.py	YouTube control
Backend/PerformanceLogger.py	Performance/comparison logging
Frontend/GUI.py	PyQt5 interface
Data/Chatlog.json	Chat storage
Data/reminders.json	Reminder storage
Data/performance_log.json	Performance data
Data/comparison_log.json	Comparison data
Data/Voice.html	Speech runtime supporting file
tests/test_config.py	Configuration tests

File / Path	Purpose
tests/test_import_side_effects.py	Import side-effect tests
tests/test_reminders.py	Reminder tests
tests/test_simple_decision_maker.py	Simple routing tests
view_statistics.py	Statistics helper
view_comparison.py	Comparison helper
poster.html	Earlier/planned project presentation material
Smartvoice Project_Proposal .pdf	Earlier/planned project proposal material